



Đồ án tốt nghiệp Nguyễn Phúc Linh 20216846

Database (Trường Đại học Bách khoa Hà Nội)



messages.pdf_cover_qr_code_label

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN

**XÂY DỰNG NỀN TẢNG DATA LAKEHOUSE
CHO TIN TỨC VÀ ỨNG DỤNG PHÂN TÍCH**

NGUYỄN PHÚC LỊNH

Linh.NP216846@sis.hust.edu.vn

Chuyên ngành: Toán - Tin

Định hướng: Toán - Tin

Giảng viên hướng dẫn: ThS. Nguyễn Danh Tú

Chữ ký của GVHD

HÀ NỘI – 2025

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

1. Mục tiêu và nội dung của đề án:

Đề án tập trung nghiên cứu, thiết kế và triển khai một hệ thống Data Lakehouse hoàn chỉnh để thu thập và phân tích dữ liệu tin tức. Nội dung đề án có tính thời sự, phù hợp với chuyên ngành và mang tính ứng dụng cao.

2. Kết quả đạt được:

Sinh viên đã hoàn thành tốt các mục tiêu đề ra. Đề án đã xây dựng thành công một hệ thống End-to-End hoạt động được, cụ thể:

- Tích hợp thành công hệ sinh thái công nghệ mã nguồn mở phức tạp.
- Tự động hóa toàn bộ luồng dữ liệu từ thu thập, xử lý đến lưu trữ.
- Cung cấp khả năng truy vấn và trực quan hóa dữ liệu hiệu quả, sẵn sàng cho việc khai thác.

3. Ý thức làm việc của sinh viên:

Hà Nội, ngày ... tháng ... năm 2025

Giảng viên hướng dẫn

ThS. Nguyễn Danh Tú

PHIẾU BÁO CÁO TIẾN ĐỘ

Danh sách đánh giá đồ án

Ngày đánh giá	Lần	Nội dung kế hoạch	Nội dung đã thực hiện	Điểm tích cực	Điểm nội dung	Ghi chú
11/04/2025	1	Nghiên cứu công cụ	Nghiên cứu công cụ	9	9	
13/05/2025	2	Xây dựng báo cáo	đạt	9	9	

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn sâu sắc tới những người đã hỗ trợ em hoàn thành đồ án tốt nghiệp. Trước hết, em xin trân trọng cảm ơn ThS. Nguyễn Danh Tú. Thầy không chỉ tin tưởng giao cho em một đề tài thực tiễn mà còn tận tình chỉ bảo, định hướng và đưa ra những phản biện chuyên môn sắc bén giúp em hoàn thiện đồ án. Em cũng xin gửi lời cảm ơn đến Khoa Toán - Tin, Trường Đại học Bách khoa Hà Nội đã tạo ra môi trường học tập lý tưởng và trang bị cho em nền tảng kiến thức vững chắc. Xin cảm ơn trang tin VnExpress đã cung cấp nguồn dữ liệu công khai, phong phú. Cảm ơn những người bạn đã luôn đồng hành, trao đổi và giúp đỡ để dự án được hoàn thiện hơn.

Em xin trân trọng cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Đồ án được chia thành 3 chương, tập trung vào việc xây dựng hệ thống Lakehouse toàn diện.

- **Chương 1 – Tổng quan và khảo sát:** Khảo sát tổng quan về Data Lakehouse và bài toán phân tích dữ liệu tin tức.
- **Chương 2 – Phân tích và thiết kế hệ thống:** Phân tích các yêu cầu và thiết kế chi tiết kiến trúc hệ thống.
- **Chương 3 – Lập trình xây dựng hệ thống Lakehouse:** Xây dựng và triển khai hệ thống Data Lakehouse trên thực tế.

Hà Nội, ngày ... tháng ... năm 2025

Tác giả đồ án

Nguyễn Phúc Linh

Mục lục

Lời cảm ơn	iii
Tóm tắt nội dung đồ án	iii
Danh sách bảng	1
Danh sách hình vẽ	2
Mở đầu	4
Chương 1 Tổng quan và khảo sát	6
1.1 Tổng quan về kiến trúc dữ liệu hiện đại	6
1.1.1 Các thành phần cốt lõi của một nền tảng dữ liệu	6
1.1.2 Kiến trúc Truyền thông: Data Warehouse và Data Lake	9
1.1.3 Kiến trúc Data Lakehouse	13
1.2 Khảo sát về dữ liệu tin tức, thời sự	25
1.2.1 Nhu cầu xây dựng hệ thống phân tích dữ liệu tin tức, thời sự	25
1.2.2 Đặc điểm dữ liệu tin tức, thời sự	26
1.2.3 Khảo sát một số nguồn tin tức điện tử uy tín tại Việt Nam	27
1.2.4 Lựa chọn kiến trúc Lakehouse cho dữ liệu tin tức	29
1.2.5 Các ứng dụng tiềm năng của hệ thống phân tích dữ liệu tin tức	31
Chương 2 Phân tích và thiết kế hệ thống	32
2.1 Nguồn dữ liệu	32
2.1.1 Lựa chọn nguồn lấy dữ liệu	32
2.1.2 Những thách thức và giải pháp trong quá trình thu thập dữ liệu	33
2.1.3 Hiểu về cấu trúc dữ liệu cần thu thập	34
2.2 Thiết kế kiến trúc hệ thống	36
2.2.1 Bản thiết kế kiến trúc tổng thể	37

2.2.2	Data Ingestion	38
2.2.3	Data Storage	41
2.2.4	Data procesing	68
2.2.5	Data Consumption and Delivery	71
2.2.6	Common Services	77
Chương 3	Lập trình xây dựng hệ thống Lakehouse	84
3.1	Kiến trúc toàn bộ dự án	84
3.2	Lập trình và xây dựng quá trình thu thập dữ liệu	86
3.2.1	Crawl dữ liệu trong 1 ngày bất kì	87
3.2.2	Crawl dữ liệu trong một khoảng ngày bất kì	89
3.3	Lập trình và xây dựng phần lưu trữ cho Lakehouse	90
3.3.1	Kiến trúc lưu trữ đa vùng	91
3.3.2	Chi tiết cấu trúc và định dạng tệp qua các vùng	91
3.4	Lập trình và xây dựng phần ETL dữ liệu	93
3.4.1	Kiến trúc và Công nghệ sử dụng	94
3.4.2	Các luồng xử lý ETL chính	95
3.4.3	Các kỹ thuật tối ưu hóa Spark được áp dụng	96
3.5	Lập trình và xây dựng các dịch vụ chung khác	97
3.5.1	Project Nessie - Quản lý phiên bản dữ liệu	97
3.5.2	Giám sát hệ thống với Prometheus và Grafana	99
3.6	Lập trình và xây dựng các dịch vụ khai thác dữ liệu	102
3.6.1	Trino - Cổng truy vấn SQL phân tán	102
3.6.2	Metabase - Nền tảng Business Intelligence	105
Kết luận		114
Danh mục tài liệu tham khảo		118

Danh sách bảng

2.1	Mô tả các trường của bảng authors	49
2.2	Mô tả các trường của bảng topics	50
2.3	Mô tả các trường của bảng subtopic	50
2.4	Mô tả các trường của bảng keywords	51
2.5	Mô tả các trường của bảng references_table	51
2.6	Mô tả các trường của bảng articles	51
2.7	Mô tả các trường của bảng article_keywords	53
2.8	Mô tả các trường của bảng article_references	53
2.9	Mô tả các trường của bảng comments	53
2.10	Mô tả các trường của bảng comment_interactions	54
2.11	Mô tả các trường của bảng dim_date	58
2.12	Mô tả các trường của bảng dim_author	58
2.13	Mô tả các trường của bảng dim_topic	59
2.14	Mô tả các trường của bảng dim_sub_topic	59
2.15	Mô tả các trường của bảng dim_keyword	60
2.16	Mô tả các trường của bảng dim_reference_source	60
2.17	Mô tả các trường của bảng dim_interaction_type	61
2.18	Mô tả các trường của bảng fact_article_publication	61
2.19	Mô tả các trường của bảng fact_article_keyword	63
2.20	Mô tả các trường của bảng fact_article_reference	64
2.21	Mô tả các trường của bảng fact_top_comment_activity	65
2.22	Mô tả các trường của bảng fact_top_comment_interaction_detail	67

Danh sách hình vẽ

1.1	Các thành phần cốt lõi của một nền tảng dữ liệu [11]	7
1.2	Kiến trúc cơ bản Kho dữ liệu	10
1.3	Kiến trúc cơ bản Hồ dữ liệu	12
1.4	Các tính năng của Lakehouse	14
1.5	Kiến trúc của Lakehouse	16
1.6	Các vùng lưu trữ trong kiến trúc lakehouse	19
1.7	Sơ đồ tổ chức tiêu thụ dữ liệu [11]	22
1.8	Logo trang báo VnExpress	28
1.9	Logo trang báo Dân Trí	28
1.10	Logo trang báo VietNamNet	29
2.1	Kiến trúc cấp cao cho nền tảng dữ liệu	38
2.2	Quy trình Thu thập và Nhập liệu trong dự án	39
2.3	Mô hình Selenium Grid	40
2.4	Các thành phần cốt lõi cấu thành vùng lưu trữ của Lakehouse trong dự án	41
2.5	Kiến trúc phân tán MinIO (Nguồn: MinIO Documentation)	42
2.6	Kiến trúc file parquet	44
2.7	Kiến trúc định dạng bảng mở Iceberg	46
2.8	Các phân vùng lưu trữ	47
2.9	ERD của mô hình dữ liệu OLTP vùng clean	49
2.10	Mô hình cho fact article publication	55
2.11	Mô hình cho fact article keyword	56
2.12	Mô hình cho fact article reference	56

2.13	Mô hình cho fact comment activity	57
2.14	Mô hình cho fact top comment interaction detail	57
2.15	Kiến trúc spark cluster ở Standalone mode	70
2.16	Kiến trúc phân tán Trino	72
2.17	Công cụ Metabase	74
2.18	Công cụ Docker	77
2.19	Công cụ quản lý catalog tập trung Nessie	79
2.20	Công cụ điều phối luồng Airflow	80
2.21	Công cụ Monitoring hiệu quả	82
3.1	Kiến trúc tổng thể dự án	84
3.2	Cấu trúc cây thư mục	85
3.3	Mô hình Selenium Grid với 1 Hub và 3 Node đang hoạt động	87
3.4	Cấu hình <i>Selenium Pool</i> trên Airflow giới hạn 3 tác vụ song song	87
3.5	Pipeline trên Airflow cho tác vụ crawl dữ liệu theo ngày	88
3.6	Tài liệu cho dag crawl dữ liệu cho một ngày	89
3.7	Luồng DAG xử lý song song nhiều ngày trong một khoảng thời gian	89
3.8	Tài liệu cho dag crawl dữ liệu khoảng ngày bất kì	90
3.9	Ba bucket trên MinIO tương ứng với 3 vùng của Lakehouse	91
3.10	Cấu trúc thư mục và định dạng tệp tại vùng Raw	92
3.11	Đại diện cho các bảng có trong 2 vùng clean và curated	92
3.12	Cấu trúc bảng Iceberg tại vùng Clean và Curated với thư mục data và metadata	93
3.13	Giao diện quản lý của Spark Master cho thấy 2 Worker đã sẵn sàng	94
3.14	DAG trên Airflow cho pipeline ETL từ vùng Raw sang Clean	95
3.15	DAG trên Airflow cho pipeline ETL từ vùng Clean sang Curated	96
3.16	Giao diện người dùng của Nessie hiển thị các nhánh	97
3.17	Giao diện người dùng của Nessie hiển thị các commit	98
3.18	Tương tác với Nessie thông qua giao diện dòng lệnh (CLI)	99
3.19	Dashboard trên Grafana trực quan hóa tài nguyên các container	100

3.20	Dashboard trên Grafana theo dõi các chỉ số chất lượng của pipeline lịch sử	101
3.21	Dashboard trên Grafana theo dõi các chỉ số chất lượng của pipeline hàng ngày	101
3.22	Giao diện quản lý của Trino hiển thị lịch sử truy vấn	103
3.23	Cây thực thi được Trino tối ưu hóa cho một câu lệnh SQL phức tạp . .	104
3.24	Sử dụng DBeaver để kết nối đến Lakehouse thông qua Trino	105
3.25	Giao diện chính của Lakehouse Dashboard trên Metabase	106
3.26	Dashboard Tổng Quan Hiệu Suất Nội Dung	107
3.27	Dashboard Phân Tích Hiệu Suất Tác Giả	109
3.28	Dashboard Phân Tích Chủ Đề Và Từ Khóa	111
3.29	Dashboard Phân Tích Tương Tác Cộng Đồng	113

Mở đầu

Trong kỷ nguyên số hiện nay, sự bùng nổ của dữ liệu đã và đang đặt ra những thách thức to lớn cho các hệ thống lưu trữ và xử lý truyền thống. Đặc biệt, trong lĩnh vực tin tức và thời sự, dữ liệu được tạo ra mỗi ngày với khối lượng khổng lồ, ở nhiều định dạng đa dạng từ có cấu trúc, bán cấu trúc đến phi cấu trúc. Việc khai thác hiệu quả nguồn dữ liệu này để nắm bắt xu hướng xã hội, phân tích dư luận hay đưa ra các quyết định chiến lược đòi hỏi một nền tảng dữ liệu hiện đại, linh hoạt và có khả năng mở rộng.

Các kiến trúc truyền thống như Data Warehouse tỏ ra cứng nhắc và tốn kém khi xử lý dữ liệu phi cấu trúc, trong khi Data Lake tuy linh hoạt nhưng lại thiếu các tính năng quản trị và đảm bảo chất lượng dữ liệu. Để giải quyết những nhược điểm này, kiến trúc Data Lakehouse đã ra đời, kết hợp những ưu điểm tốt nhất của cả hai thế giới.

Xuất phát từ những yêu cầu thực tiễn đó, đề tài "Xây dựng nền tảng Data Lakehouse cho dữ liệu tin tức và ứng dụng phân tích" được thực hiện nhằm nghiên cứu và triển khai một giải pháp toàn diện để giải quyết bài toán dữ liệu trong lĩnh vực này.

Nội dung chính của đồ án được trình bày trong 3 chương:

- **Chương 1 – Tổng quan và khảo sát:** Trình bày tổng quan về các hệ thống dữ liệu, đi sâu vào kiến trúc Data Lakehouse, và khảo sát bài toán phân tích dữ liệu trong lĩnh vực tin tức, thời sự.
- **Chương 2 – Phân tích và thiết kế hệ thống:** Phân tích các yêu cầu và đưa ra bản thiết kế chi tiết cho toàn bộ hệ thống, từ kiến trúc tổng thể đến thiết kế cho từng thành phần con như lưu trữ, xử lý, và khai thác.
- **Chương 3 – Lập trình xây dựng hệ thống Lakehouse:** Trình bày chi tiết quá trình lập trình, triển khai và tích hợp các thành phần công nghệ để xây dựng nên một hệ thống Data Lakehouse hoàn chỉnh, hoạt động được.

Chương 1

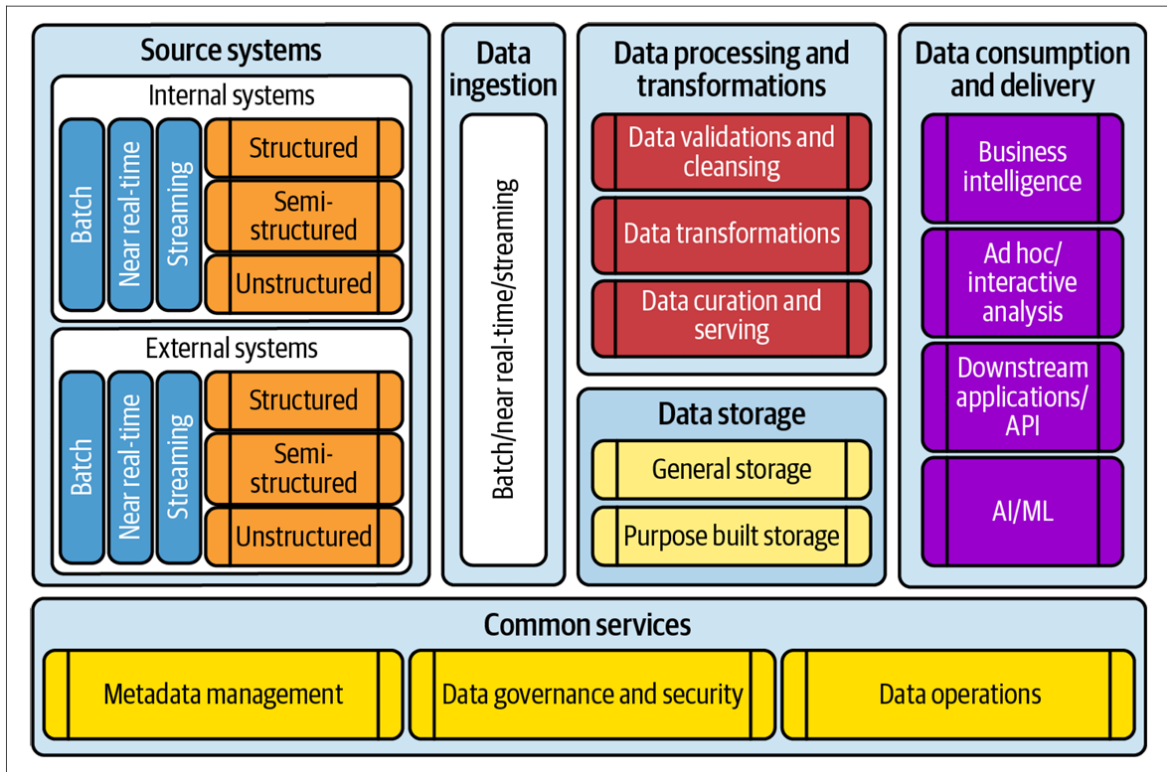
Tổng quan và khảo sát

1.1 Tổng quan về kiến trúc dữ liệu hiện đại

Trong bối cảnh Trí tuệ Nhân tạo (AI) đang phát triển như vũ bão, các doanh nghiệp ngày càng nhận thức sâu sắc vai trò trung tâm của dữ liệu. Khả năng khai thác hiệu quả nguồn tài sản này không còn là một lợi thế cạnh tranh mà đã trở thành yêu cầu cấp thiết. Điều này đòi hỏi các tổ chức phải xây dựng và triển khai một nền tảng dữ liệu (data platform) đủ mạnh mẽ và linh hoạt, không chỉ để vận hành các ứng dụng AI tiên tiến mà còn để đưa ra quyết định dựa trên dữ liệu, tối ưu hóa quy trình và tạo ra những giá trị đột phá. Việc xây dựng một nền tảng dữ liệu hiệu quả, do đó, trở thành một bài toán chiến lược, quyết định năng lực thích ứng và phát triển bền vững của doanh nghiệp trong kỷ nguyên số.

1.1.1 Các thành phần cốt lõi của một nền tảng dữ liệu

Kiến trúc dữ liệu đóng vai trò là bản thiết kế giúp định nghĩa các thành phần cốt lõi của một nền tảng dữ liệu và thiết lập các nguyên tắc thiết kế để đưa nó vào thực tế. Các thành phần cốt lõi này là nền tảng để xây dựng một nền tảng dữ liệu mạnh mẽ, có thể mở rộng và dễ tiếp cận.



Hình 1.1: Các thành phần cốt lõi của một nền tảng dữ liệu [11]

Để xây dựng một nền tảng dữ liệu toàn diện, các thành phần cốt lõi sau đây cần được xem xét:

1. **Hệ thống nguồn:** Đây là nơi cung cấp dữ liệu cho nền tảng dữ liệu, phục vụ cho các mục đích phân tích, BI, và ML. Hệ thống nguồn rất đa dạng, có thể bao gồm các hệ thống cũ, hệ thống OLTP backend, thiết bị IoT, dữ liệu clickstream, và mạng xã hội. Chúng có thể là các hệ thống nội bộ hoặc từ bên ngoài tổ chức, với dữ liệu được gửi theo lô (batch), gần thời gian thực, hoặc dưới dạng luồng liên tục (streaming). Dữ liệu từ các nguồn này cũng có thể ở nhiều định dạng khác nhau: có cấu trúc, bán cấu trúc, hoặc phi cấu trúc. Do đó, một nền tảng dữ liệu hiệu quả cần có khả năng hỗ trợ tất cả các loại hệ thống nguồn này, cùng với sự đa dạng về định dạng và tần suất cập nhật dữ liệu.
2. **Thu thập dữ liệu:** Thu thập dữ liệu là quá trình trích xuất dữ liệu từ các hệ thống nguồn đa dạng và tải nó vào nền tảng dữ liệu. Một khung công tác thu thập dữ liệu cần được triển khai để xây dựng các hệ thống xử lý dữ liệu theo lô, gần thời gian thực, hoặc theo luồng, tùy thuộc vào khả năng tạo và gửi dữ liệu của từng hệ thống nguồn cụ thể.

3. **Lưu trữ dữ liệu:** Thành phần lưu trữ dữ liệu đóng vai trò cốt lõi trong việc duy trì và bảo quản dữ liệu. Khi triển khai một nền tảng dữ liệu, việc thiết kế một lớp lưu trữ bền bỉ, đáng tin cậy và có tính sẵn sàng cao là vô cùng quan trọng. Lớp lưu trữ này phải có khả năng lưu giữ một lượng lớn dữ liệu lịch sử thuộc mọi loại. Các giải pháp lưu trữ dữ liệu phổ biến bao gồm các dịch vụ lưu trữ đối tượng trên đám mây (như Amazon S3, ADLS, GCS), kho dữ liệu, hệ thống quản lý cơ sở dữ liệu quan hệ (RDBMS), cơ sở dữ liệu trong bộ nhớ, và cơ sở dữ liệu đồ thị. Trong kiến trúc Lakehouse hiện đại, lưu trữ dữ liệu thường được phân chia thành các vùng hoặc lớp, theo một mô hình được gọi là kiến trúc huy chương, bao gồm vùng thô, vùng đã làm sạch, và vùng đã được sắp xếp và làm giàu. Đặc biệt, dữ liệu nhạy cảm cần được che giấu hoặc mã hóa trước khi lưu trữ, tuân thủ các yêu cầu về bảo mật và quy định.
4. **Xử lý và biến đổi dữ liệu:** Đây là thành phần chịu trách nhiệm cho các hoạt động quan trọng như làm sạch, xác thực, biến đổi và sắp xếp dữ liệu. Mục tiêu của các hoạt động này là đảm bảo rằng dữ liệu được lưu trữ trong nền tảng luôn sạch, hợp lệ, chính xác và đầy đủ. Điều này giúp người dùng có thể tin cậy vào dữ liệu và dễ dàng sử dụng nó cho các mục đích phân tích và ra quyết định. Quá trình này bao gồm nhiều bước cụ thể như xác thực và làm sạch dữ liệu để loại bỏ lỗi và sự không nhất quán, tích hợp dữ liệu từ nhiều nguồn khác nhau, biến đổi và làm giàu dữ liệu để tăng giá trị và tính hữu dụng, cuối cùng là sắp xếp và phục vụ dữ liệu cho các hệ thống và người dùng cuối.
5. **Sử dụng và phân phối dữ liệu:** Các thành phần sử dụng và phân phối dữ liệu trong nền tảng cho phép người dùng cuối và các ứng dụng khác nhau truy cập, phân tích, truy vấn và khai thác dữ liệu đã được xử lý. Các công cụ và phương thức tiêu thụ dữ liệu rất đa dạng, bao gồm các công cụ báo cáo BI để trực quan hóa và tạo báo cáo, các mô hình ML được sử dụng để dự đoán và dự báo xu hướng, hoặc các ứng dụng downstream và API cung cấp dữ liệu cho các hệ thống khác. Một đặc điểm quan trọng của các nền tảng dữ liệu hiện đại là khả năng hỗ trợ đa dạng các loại khối lượng công việc như BI, AI/ML, ETL và các ứng dụng thời gian thực.
6. **Các dịch vụ chung:** Các dịch vụ chung cung cấp các chức năng thiết yếu,

xuyên suốt toàn bộ nền tảng dữ liệu. Chúng đóng vai trò quan trọng trong việc làm cho dữ liệu trở nên dễ dàng khám phá, luôn sẵn sàng và được bảo vệ an toàn cho người dùng. Các dịch vụ này bao gồm:

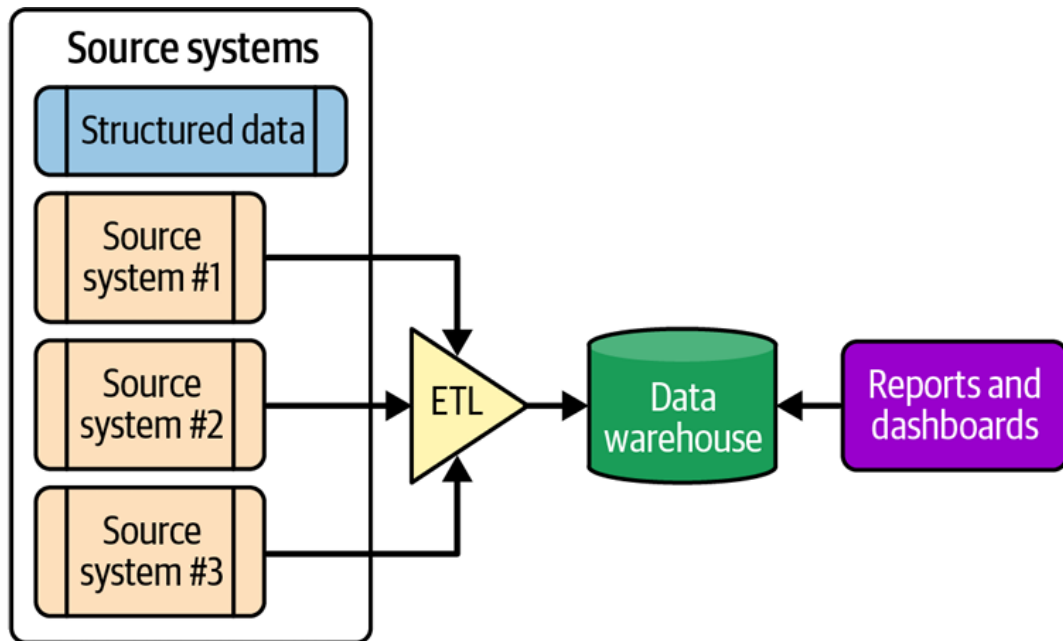
- **Quản lý siêu dữ liệu:** Giúp người dùng dễ dàng hiểu rõ hơn về dữ liệu thông qua việc sử dụng một danh mục dữ liệu. Danh mục này cung cấp thông tin chi tiết về siêu dữ liệu kỹ thuật và các định nghĩa nghiệp vụ liên quan đến dữ liệu.
- **Quản trị dữ liệu và bảo mật:** Đảm bảo rằng dữ liệu được quản lý, quản trị một cách chặt chẽ và được truy cập an toàn. Điều này bao gồm việc triển khai các biện pháp kiểm soát truy cập, giám sát và đảm bảo chất lượng dữ liệu, thực hiện kiểm toán, theo dõi dòng dữ liệu, quản lý việc chia sẻ dữ liệu và tuân thủ các quy định pháp lý. Các nền tảng dữ liệu hiện đại ngày nay cho phép áp dụng các biện pháp quản trị và bảo mật dữ liệu nghiêm ngặt hơn.
- **Vận hành dữ liệu (DataOps):** Giúp quản lý hiệu quả các hoạt động khác nhau trong suốt vòng đời của dữ liệu. Các hoạt động này bao gồm việc điều phối các luồng xử lý dữ liệu, tự động hóa quy trình kiểm thử, tích hợp và triển khai (CI/CD), cũng như quản lý và giám sát "sức khỏe" của toàn bộ hệ sinh thái dữ liệu thông qua các kỹ thuật quan sát dữ liệu.

Tất cả các thành phần cốt lõi này cùng nhau tạo nên một nền tảng dữ liệu hoàn chỉnh, cho phép người dùng thực hiện nhiều hoạt động khác nhau liên quan đến dữ liệu một cách hiệu quả và an toàn.

1.1.2 Kiến trúc Truyền thống: Data Warehouse và Data Lake

1. Kiến trúc Kho dữ liệu

Kho dữ liệu là một kiến trúc truyền thống để xây dựng các nền tảng dữ liệu. Bill Inmon [11] định nghĩa kho dữ liệu là một tập hợp dữ liệu hướng đối tượng, tích hợp, biến thiên theo thời gian, và không biến đổi có thể giúp đưa ra các quyết định quản lý.



Hình 1.2: Kiến trúc cơ bản Kho dữ liệu

Đặc trưng:

- **Hướng đối tượng:** Dữ liệu được tổ chức xoay quanh các chủ đề nghiệp vụ chính.
- **Tích hợp:** Tích hợp dữ liệu từ nhiều hệ thống nguồn khác nhau.
- **Biến thiên theo thời gian và Không biến đổi:** Được xây dựng để lưu trữ khối lượng lớn dữ liệu lịch sử. Kho dữ liệu giữ lại các phiên bản cũ của dữ liệu cùng dấu thời gian và dữ liệu sau khi tải vào thường không bị thay đổi hoặc xóa trong các hoạt động thông thường.
- **Schema nghiêm ngặt:** Schema (cấu trúc dữ liệu) được áp dụng và tuân thủ chặt chẽ.
- **Chất lượng dữ liệu tốt:** Dữ liệu được xác thực và làm sạch nghiêm ngặt trước khi tải. Các trường hợp ngoại lệ bị từ chối.
- **Tuân thủ ACID:** Các hệ thống cơ sở dữ liệu quan hệ thường dùng cho kho dữ liệu tuân thủ ACID (Atomicity, Consistency, Isolation, Durability).
- **Hỗ trợ cập nhật và xóa:** Việc cập nhật hoặc xóa bản ghi rất dễ dàng bằng truy vấn SQL đơn giản.
- **Chủ yếu là dữ liệu có cấu trúc:** Được xây dựng để lưu trữ và xử lý chủ yếu dữ liệu có cấu trúc và bán cấu trúc. Không có phương pháp trực tiếp để lưu trữ dữ liệu phi cấu trúc.

- Mô hình hóa và tổ chức dữ liệu: Dữ liệu được tổ chức trong một mô hình dữ liệu tốt (mô hình ER hoặc dimensional).
- Kiểm soát truy cập tốt hơn: Dữ liệu lưu trữ trong bảng (hàng, cột) cho phép kiểm soát truy cập chi tiết. Có thể tạo Data Marts.
- Gắn chặt giữa Lưu trữ và Tính toán: Các sản phẩm truyền thống thường là các thiết bị đi kèm, có sự gắn kết chặt chẽ giữa lưu trữ và tính toán.
- Định dạng lưu trữ độc quyền: Sử dụng các định dạng độc quyền.

Ưu điểm:

- Hiệu suất vượt trội cho các công việc BI: như báo cáo, bảng điều khiển và phân tích ad-hoc. Các công cụ lưu trữ và xử lý độc quyền được tối ưu hóa cao.
- Chất lượng dữ liệu cao: nhờ quy trình xác thực và làm sạch nghiêm ngặt.
- Kiểm soát truy cập và quản trị tốt: ở cấp độ chi tiết.
- Hỗ trợ ACID và các thao tác cập nhật/xóa dễ dàng.
- Kiến trúc đơn giản, tầng lưu trữ duy nhất.

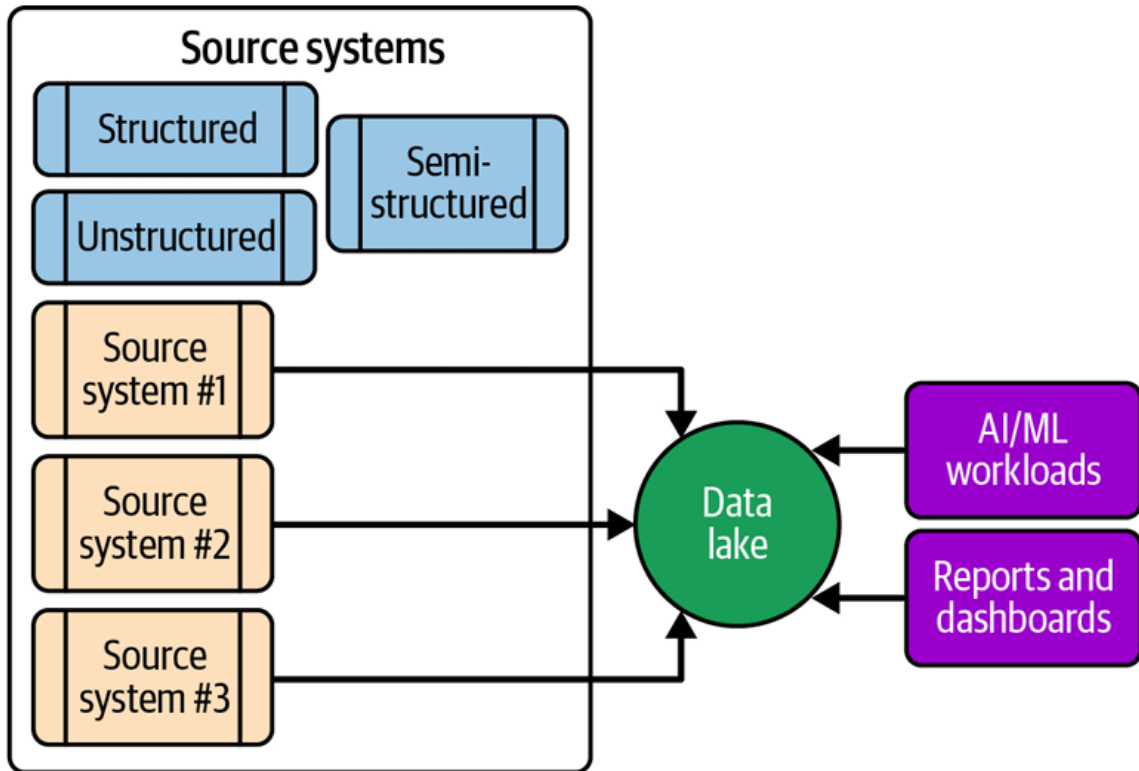
Nhược điểm:

- Chi phí cao: do sử dụng lưu trữ và công cụ độc quyền.
- Khóa nhà cung cấp: đáng kể do định dạng độc quyền và công cụ riêng của nhà cung cấp, khó di chuyển sang nền tảng khác.
- Hạn chế về định dạng dữ liệu: Chỉ hỗ trợ chủ yếu dữ liệu có cấu trúc và bán cấu trúc.
- Hạn chế chia sẻ dữ liệu: với người tiêu dùng sử dụng nền tảng khác do định dạng độc quyền.
- Thách thức với các trường hợp sử dụng dữ liệu biến động hoặc streaming: phù hợp hơn cho công việc theo lô.

Các sản phẩm kho dữ liệu phổ biến truyền thống bao gồm Teradata, IBM Netezza, Oracle Exadata.

2. Kiến trúc Hồ dữ liệu

Hồ dữ liệu [11] là một kho lưu trữ trung tâm duy nhất để lưu trữ tất cả dữ liệu của bạn ở quy mô lớn. Nó được xây dựng dựa trên lưu trữ đối tượng với chi phí thấp.



Hình 1.3: Kiến trúc cơ bản Hồ dữ liệu

Đặc trưng:

- Kho lưu trữ trung tâm duy nhất: Được thiết kế để lưu trữ tất cả dữ liệu.
- Dựa trên lưu trữ đối tượng chi phí thấp.
- Lưu trữ tất cả dữ liệu ở quy mô lớn: Bao gồm cả dữ liệu thô ở quy mô petabyte.
- Hỗ trợ đa dạng định dạng dữ liệu: Có thể lưu trữ dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc.
- Schema on Read: Schema được áp dụng khi dữ liệu được đọc để xử lý thay vì khi ghi vào.
- Không tuân thủ ACID theo truyền thống: Theo truyền thống, hồ dữ liệu không tuân thủ ACID. Cần các biện pháp bổ sung để xử lý tính đồng thời và tính nhất quán.
- Cần code tùy chỉnh cho cập nhật và xóa.
- Chất lượng dữ liệu thường thấp: Do thiếu xác thực và làm sạch trước khi lưu trữ.
- Sử dụng các định dạng tệp mở.

Ưu điểm:

- Chi phí thấp: nhờ sử dụng lưu trữ đối tượng chi phí thấp.
- Tính linh hoạt cao: do cách tiếp cận schema on read.
- Hỗ trợ đa dạng workloads: BI, AI/ML, ETL, và các trường hợp sử dụng theo thời gian thực. Đặc biệt cho phép triển khai AI/ML và streaming.
- Chia sẻ dữ liệu dễ dàng hơn: nhờ sử dụng các định dạng tệp mở.
- Khả năng lưu trữ tất cả dữ liệu thô.

Nhược điểm:

- Chất lượng dữ liệu thường thấp hơn.
- Nguy cơ trở thành "đầm lầy dữ liệu": nếu không được quản lý tốt.
- Hiệu suất cho các công việc BI có thể kém hơn: do schema on read và thiếu tối ưu hóa. Cần xử lý đáng kể trước khi dữ liệu thô dùng được cho BI.
- Thiếu tuân thủ ACID theo truyền thống.
- Thách thức trong việc cập nhật và xóa: cần code tùy chỉnh.
- Kiểm soát truy cập có thể khó khăn hơn.

1.1.3 Kiến trúc Data Lakehouse

Hành trình chinh phục và khai thác giá trị từ dữ liệu của các tổ chức từ lâu đã luôn đứng trước những lựa chọn khó khăn: hoặc là sự ổn định, có cấu trúc chặt chẽ của Kho dữ liệu (Data Warehouse) nhưng đi kèm với chi phí cao, tính linh hoạt hạn chế và rào cản với dữ liệu phi cấu trúc; hoặc là không gian lưu trữ rộng lớn, đa dạng của Hồ dữ liệu (Data Lake) nhưng lại thường xuyên đối mặt với bài toán về chất lượng, thiếu vắng các đảm bảo giao dịch ACID và phức tạp trong công tác quản trị.

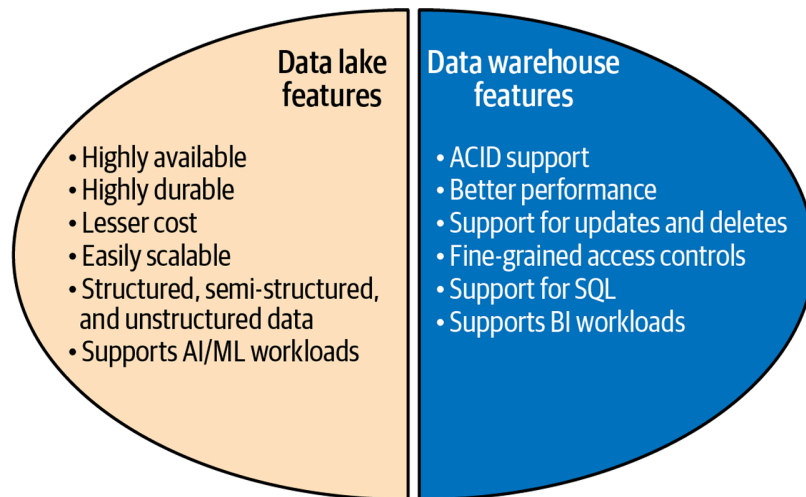
Tuy nhiên, một làn sóng mạnh mẽ từ các công cụ, sản phẩm và đặc biệt là công nghệ mã nguồn mở mới đã tạo nên một cuộc cách mạng, làm thay đổi sâu sắc cách chúng ta tiếp cận và xây dựng hệ sinh thái dữ liệu. Những tiến bộ vượt bậc này không chỉ giúp gỡ rối những kiến trúc phức tạp vốn có, mà còn mở đường cho sự ra đời của các nền tảng dữ liệu ưu việt hơn hẳn: đáng tin cậy hơn trong vận hành, cởi mở hơn trong kết nối và linh hoạt hơn để đáp ứng vô vàn các nhu cầu phân tích dữ liệu ngày càng đa dạng.

Chính từ sự giao thoa giữa nhu cầu thực tiễn cấp thiết và những đột phá công nghệ đó, kiến trúc Lakehouse đã hình thành và nhanh chóng khẳng định vị thế. Đây không chỉ đơn thuần là một mô hình kiến trúc mới mẻ, mà thực sự là một

giải pháp thông minh, kết tinh những thành tựu công nghệ nhằm kiến tạo nên một nền tảng dữ liệu vừa đơn giản trong triển khai, vừa mang tính thần cỏi mở.

1. Lakehouse: Những ưu điểm tốt nhất của cả hai thế giới

Lakehouse [11] kết hợp các tính năng của cả Kho dữ liệu (Data Warehouse) và Hồ dữ liệu (Data Lake).



Hình 1.4: Các tính năng của Lakehouse

a) Cách Lakehouse thừa hưởng các đặc tính của Hồ dữ liệu:

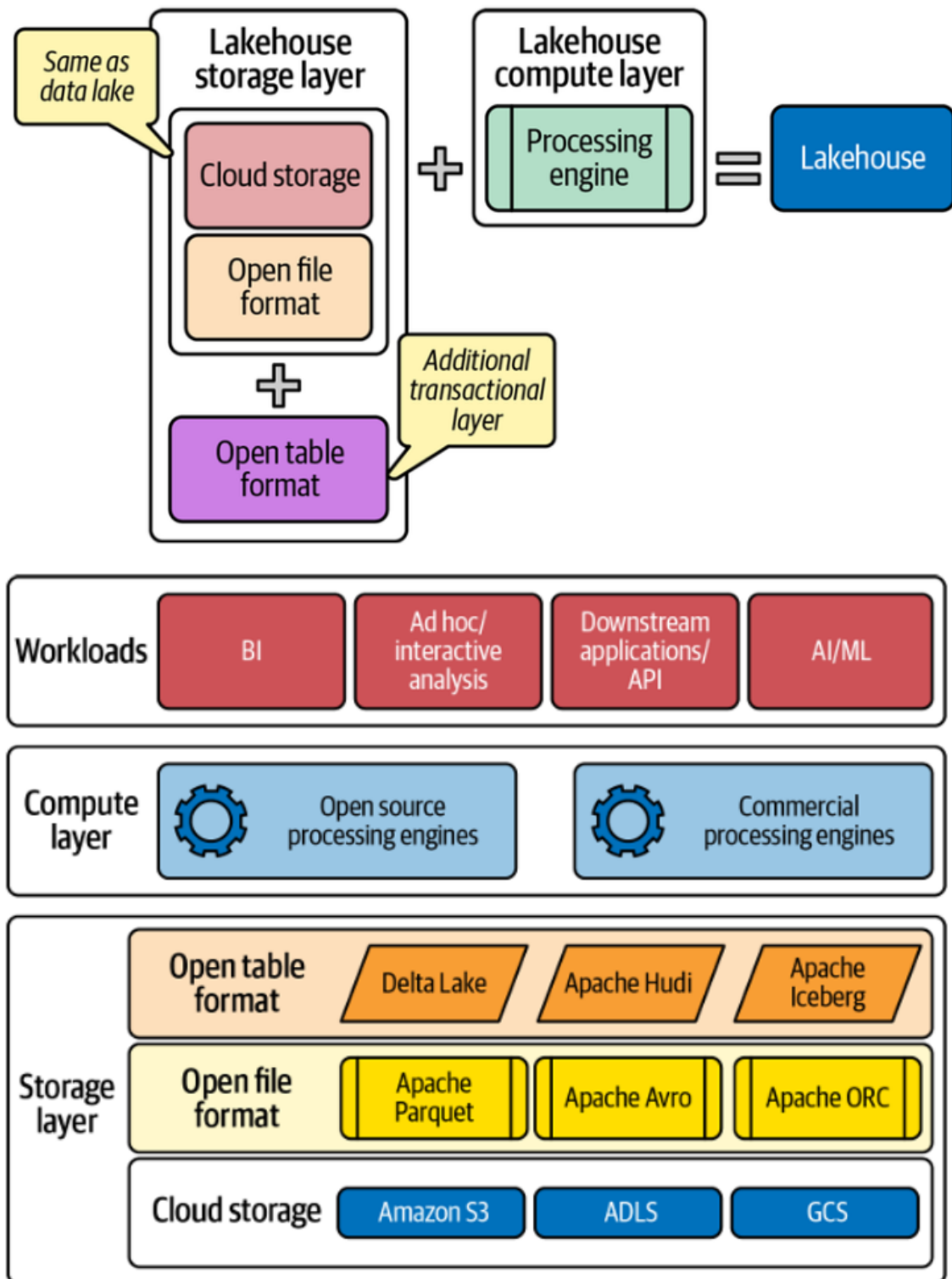
- Tương tự như một Hồ dữ liệu, Lakehouse thường tận dụng các giải pháp **lưu trữ đối tượng (object storage)** làm nền tảng. Các giải pháp này rất đa dạng, có thể là các dịch vụ trên nền tảng đám mây như Amazon S3, Azure Data Lake Storage (ADLS), Google Cloud Storage (GCS), hoặc các hệ thống **lưu trữ đối tượng tự triển khai (on-premise)** như MinIO.
- Dữ liệu trong Lakehouse được lưu trữ dưới các định dạng tệp mở (ví dụ: Apache Parquet, Apache Avro, hoặc Apache ORC).
- Việc sử dụng các giải pháp lưu trữ đối tượng này cho phép Lakehouse kế thừa những ưu điểm vượt trội của Hồ dữ liệu, bao gồm: tính sẵn sàng cao, độ bền dữ liệu xuất sắc, chi phí hiệu quả, khả năng mở rộng linh hoạt, hỗ trợ toàn diện cho mọi loại dữ liệu (từ có cấu trúc, bán cấu trúc đến phi cấu trúc), và đặc biệt là khả năng hỗ trợ mạnh mẽ cho các ứng dụng Trí tuệ Nhân tạo (AI) và Học Máy (ML). Mặc dù HDFS cũng có thể được sử dụng trong một số trường hợp, các giải pháp lưu trữ đối

tượng thường được ưu tiên nhờ tính linh hoạt và khả năng mở rộng, với xu hướng các giải pháp tiên tiến thường khai thác mạnh mẽ lợi thế của công nghệ đám mây khi phù hợp.

b) Cách Lakehouse tích hợp các năng lực của Kho dữ liệu:

- So với Hồ dữ liệu truyền thống, điểm cốt lõi làm nên sự khác biệt của Lakehouse là sự hiện diện của một thành phần bổ sung: lớp giao dịch (transactional layer). Lớp này đôi khi còn được gọi là "lớp siêu dữ liệu" (metadata layer), bởi nó cung cấp siêu dữ liệu cần thiết liên quan đến các giao dịch dữ liệu.
- Lớp giao dịch này hoạt động như một tầng quản lý thông minh nằm ngay trên các định dạng tệp mở của Hồ dữ liệu.
- Chính sự bổ sung này mang lại cho Lakehouse những khả năng mạnh mẽ tương tự Kho dữ liệu, điển hình là việc đảm bảo tuân thủ các nguyên tắc ACID, hỗ trợ hiệu quả các thao tác cập nhật và xóa dữ liệu, cải thiện đáng kể hiệu suất cho các tác vụ Phân tích Kinh doanh (BI), và cho phép thiết lập cơ chế kiểm soát truy cập chi tiết.
- Công nghệ được sử dụng để triển khai lớp giao dịch này được biết đến với tên gọi chung là các "định dạng bảng mở" (open table formats).

2. Hiểu về tổng quan kiến trúc Lakehouse



Hình 1.5: Kiến trúc của Lakehouse

Kiến trúc Lakehouse được xây dựng dựa trên hai lớp chính sau đây:

- Lớp Lưu trữ**: Lớp lưu trữ đóng vai trò trung tâm, được ví như trái tim của nền tảng dữ liệu Lakehouse. Nhiệm vụ quan trọng của lớp này là lưu trữ tất cả các loại dữ liệu một cách hiệu quả về chi phí và tối ưu hóa hiệu suất truy

vấn. Các thành phần chính của lớp lưu trữ trong kiến trúc Lakehouse bao gồm:

- **Bộ nhớ đối tượng (object storage):** Đây là thành phần cốt lõi nơi dữ liệu của Lakehouse được lưu trữ vật lý. Lakehouse có thể tận dụng các giải pháp lưu trữ đối tượng trên nền tảng đám mây – ví dụ như Amazon S3, ADLS (Azure Data Lake Storage), hoặc GCS (Google Cloud Storage) – hoặc các hệ thống lưu trữ đối tượng tự triển khai tại chỗ như MinIO hoặc các giải pháp tương tự. Việc sử dụng kiến trúc lưu trữ đối tượng này mang lại cho Lakehouse những lợi ích vốn có của Hồ dữ liệu như tính sẵn sàng cao, độ bền dữ liệu vượt trội, tiềm năng tối ưu chi phí, khả năng mở rộng linh hoạt và hỗ trợ toàn diện cho mọi loại dữ liệu (từ có cấu trúc, bán cấu trúc đến phi cấu trúc), cũng như tối ưu cho các ứng dụng AI và ML. Trong khi HDFS cũng có thể được sử dụng làm nền tảng lưu trữ, đặc biệt trong các môi trường tại chỗ đã có sẵn, các hệ thống lưu trữ đối tượng (cả trên đám mây lẫn tự triển khai) ngày càng trở nên phổ biến cho các triển khai Lakehouse hiện đại. Điều này là nhờ vào các ưu điểm như khả năng tách biệt hiệu quả giữa tài nguyên tính toán và lưu trữ, tính linh hoạt cao trong việc mở rộng, và trong nhiều trường hợp là chi phí tổng thể cạnh tranh hơn, đặc biệt khi các lợi thế của dịch vụ đám mây được khai thác.
- **Định dạng tệp mở:** Dữ liệu được lưu trữ trong bộ nhớ đối tượng dưới các định dạng tệp mở, ví dụ như Apache Parquet, Apache Avro, hoặc Apache ORC. Các định dạng này không chỉ hỗ trợ nén dữ liệu hiệu quả mà còn được thiết kế để tối ưu cho việc xử lý phân tán các tập dữ liệu lớn.
- **Định dạng bảng mở:** Đây là một thành phần then chốt, nằm phía trên các định dạng tệp mở. Có thể nói, định dạng bảng mở chính là "trái tim" mang tính quyết định của Lakehouse, bởi chúng bổ sung các khả năng giao dịch cần thiết cho Hồ dữ liệu, qua đó thực sự biến nó thành một Lakehouse hoàn chỉnh. Ba định dạng bảng mở phổ biến nhất hiện nay là Apache Iceberg, Apache Hudi, và Delta Lake (một dự án của Linux Foundation). Các tính năng quan trọng của định dạng bảng mở bao gồm:

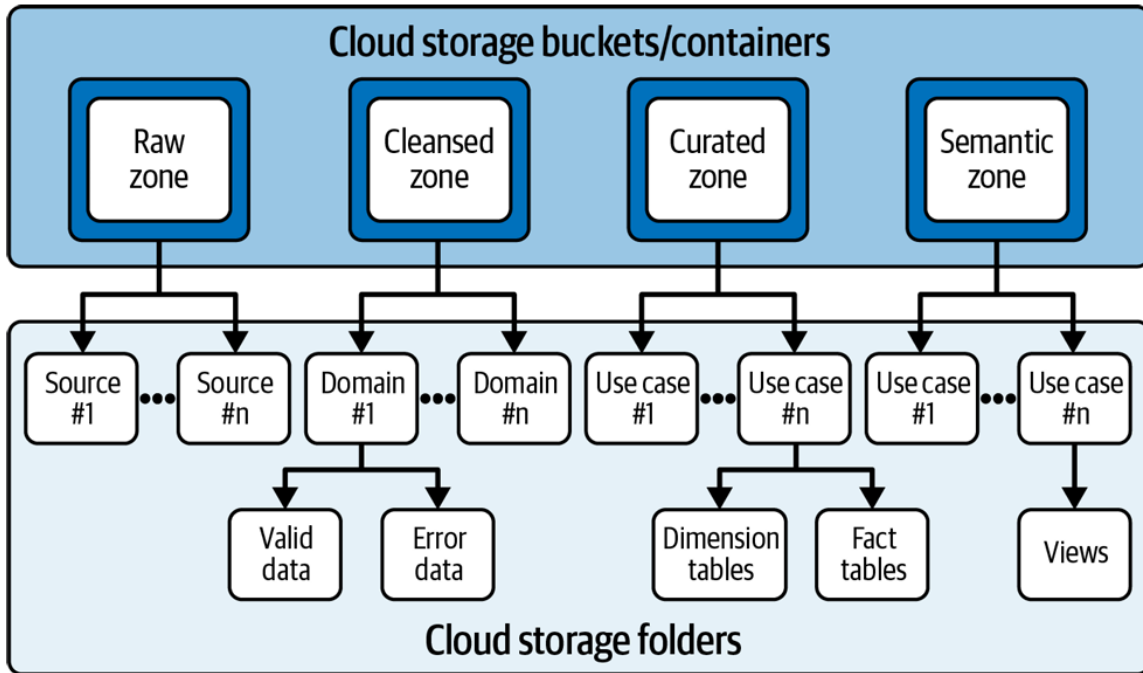
- Mang lại các khả năng của Kho dữ liệu như tuân thủ nguyên tắc ACID, hỗ trợ hiệu quả các thao tác cập nhật và xóa dữ liệu, cải thiện đáng kể hiệu suất cho các tác vụ Phân tích Kinh doanh (BI), và cho phép thiết lập cơ chế kiểm soát truy cập ở mức độ chi tiết. (Lớp này đôi khi được gọi là "lớp siêu dữ liệu" vì nó quản lý siêu dữ liệu liên quan đến các giao dịch).
- Cung cấp các tính năng quản lý dữ liệu tiên tiến như "time travel" (khả năng truy vấn và khôi phục các phiên bản dữ liệu cũ đã bị cập nhật hoặc xóa), "schema evolution" (khả năng thay đổi cấu trúc bảng như thêm, xóa, hoặc đổi tên cột một cách dễ dàng mà không cần phải ghi lại toàn bộ dữ liệu), và hỗ trợ truy vấn dữ liệu bằng SQL.

b) **Lớp Tính toán:** Nếu lớp lưu trữ được ví như trái tim, thì các công cụ và tài nguyên tính toán chính là bộ não của kiến trúc Lakehouse, chịu trách nhiệm thực thi tất cả các hoạt động tính toán và xử lý dữ liệu. Các đặc điểm chính của lớp tính toán bao gồm:

- Tính mở tự nhiên, cho phép dữ liệu được truy cập hoặc truy vấn trực tiếp bởi bất kỳ công cụ xử lý nào tương thích, mà không yêu cầu một công cụ độc quyền hay cụ thể nào để chạy các tác vụ BI hoặc phân tích tương tác.
- Có thể sử dụng các giải pháp mã nguồn mở phổ biến và mạnh mẽ như Spark, Presto, Trino, và Hive, hoặc cũng có thể là các công cụ thương mại được các nhà cung cấp xây dựng và tối ưu hóa riêng cho kiến trúc Lakehouse.
- Hoạt động dựa trên nguyên tắc tách biệt giữa tài nguyên lưu trữ và tài nguyên tính toán. Cách tiếp cận này mang lại nhiều lợi ích quan trọng cho các hoạt động xử lý dữ liệu, từ khâu thu thập, xử lý, chuyển đổi cho đến khâu tiêu thụ và phân tích dữ liệu, đồng thời cho phép việc mở rộng quy mô một cách độc lập cho cả hai.
- Lớp tính toán linh hoạt này cho phép nhiều vai trò người dùng khác nhau trong tổ chức (như kỹ sư dữ liệu, nhà phân tích dữ liệu, nhà khoa học dữ liệu, người dùng nghiệp vụ, v.v.) có thể truy cập và sử dụng dữ liệu một cách hiệu quả, phù hợp với nhu cầu và công cụ chuyên môn của họ. Ví dụ, kỹ sư dữ liệu có thể sử dụng Spark cho các tác vụ ETL

phức tạp, trong khi nhà phân tích dữ liệu có thể tận dụng các công cụ dựa trên SQL như Presto hoặc Trino để thực hiện các truy vấn tương tác trực tiếp trên dữ liệu lưu trữ.

3. Cách tổ chức lưu trữ trong Lakehouse



Hình 1.6: Các vùng lưu trữ trong kiến trúc lakehouse

Kiến trúc Medallion chia nhỏ dữ liệu trong Lakehouse thành các vùng (zones) dựa trên chất lượng và mức độ xử lý, thường bao gồm các vùng Bronze, Silver, và Gold, cùng với một lớp ngữ nghĩa tùy chọn:

a) **Vùng Thô (Raw Zone / Bronze Zone):** Vùng thô là khu vực đệm ban đầu, được thiết kế để duy trì dữ liệu gốc từ các hệ thống nguồn, chưa qua bất kỳ sự biến đổi đáng kể nào. Khi lập kế hoạch cho vùng thô, cần xem xét các điểm sau:

- Vì vùng thô chứa các tệp dữ liệu gốc, dữ liệu này thường không được sử dụng trực tiếp cho mục đích phân tích hoặc tạo ra thông tin chi tiết. Do đó, thông thường không cần thiết phải tạo bảng dữ liệu trên các tệp này ở giai đoạn này.
- Cấu trúc thư mục cơ bản bên trong vùng thô nên được tổ chức một cách logic dựa trên hệ thống nguồn. Nên tạo các thư mục riêng cho từng hệ thống nguồn và các thư mục con tương ứng với mỗi bảng hoặc loại dữ liệu.

liệu từ nguồn đó.

- Các tệp dữ liệu thô thường rất quan trọng cho việc xử lý lại dữ liệu khi cần thiết, hoặc trong tương lai, cho các mục đích kiểm toán và đảm bảo tuân thủ quy định. Do đó, việc duy trì các tệp thô này trong một khoảng thời gian dài là rất quan trọng.
- Để tối ưu chi phí lưu trữ, nên xem xét việc lưu trữ (archive) các tệp thô này vào các tầng lưu trữ lạnh (ví dụ: Amazon S3 Glacier, Azure Archive Storage, Google Cloud Archive Storage) sau khi chúng đã được xử lý và chuyển sang các vùng tiếp theo.

b) Vùng Đã Làm Sạch (Clean Zone / Silver Zone): Vùng này chứa dữ liệu đã được làm sạch, lọc, xác thực và có thể đã được làm giàu từ vùng thô. Dữ liệu ở đây có cấu trúc rõ ràng hơn và sẵn sàng cho các bước xử lý hoặc phân tích tiếp theo. Bên trong vùng đã làm sạch, có thể phân chia thành các cấp độ:

- *Cấp độ 1 (Làm sạch cơ bản):* Chứa các bảng dữ liệu đã qua các bước kiểm tra chất lượng cơ bản (ví dụ: kiểm tra kiểu dữ liệu, định dạng), đảm bảo tính hợp lệ và nhất quán hơn so với dữ liệu thô. Dữ liệu ở đây có thể vẫn giữ cấu trúc gần với nguồn.
- *Cấp độ 2 (Tích hợp và chuẩn hóa):* Bao gồm các bảng dữ liệu được tích hợp và hợp nhất từ nhiều nguồn khác nhau (sau khi đã được làm sạch ở cấp độ một). Các bảng này thường được tổ chức theo các thực thể hoặc miền nghiệp vụ chính, có thể được chuẩn hóa (ví dụ: sử dụng mô hình Quan hệ Thực thể - ER).

Khi thiết kế vùng đã làm sạch, cần lưu ý những điều sau:

- Nên chuyển đổi dữ liệu sang các định dạng bảng mở (ví dụ: Delta Lake, Apache Iceberg, Apache Hudi) khi lưu trữ trong vùng đã làm sạch để tận dụng các tính năng quản lý dữ liệu tiên tiến như giao dịch ACID, time travel, và schema evolution.
- Các nhà khoa học dữ liệu thường sử dụng dữ liệu ở vùng này cho các hoạt động phân tích dữ liệu khám phá (EDA). Người dùng có kỹ thuật cũng có thể tận dụng vùng này cho các truy vấn đặc biệt (ad hoc). Việc tạo bảng trên các tệp dữ liệu sẽ giúp người dùng dễ dàng khám phá và thực hiện các truy vấn SQL.

- Đối với các tổ chức lớn, việc xem xét sử dụng phương pháp mô hình hóa Data Vault cũng là một lựa chọn để tăng tính linh hoạt và khả năng kiểm toán.
- Nếu định dạng bảng cơ sở hỗ trợ, hãy cân nhắc việc định nghĩa các khóa chính và khóa ngoại (mang tính thông tin) cho các bảng để giúp xác định mối quan hệ giữa các thực thể.

c) **Vùng Đã Tuyển Chọn (Curated Zone / Gold Zone):** Vùng đã tuyển chọn, còn được gọi là vùng trình bày (presentation zone) hoặc vùng vàng (gold zone), chứa dữ liệu đã được tổng hợp, làm giàu và tối ưu hóa cao độ cho các mục đích phân tích và báo cáo nghiệp vụ cụ thể. Dữ liệu ở đây thường được phi chuẩn hóa (denormalized) và tổ chức theo các mô hình phục vụ người dùng cuối. Các cân nhắc thiết kế chính cho vùng đã tuyển chọn bao gồm:

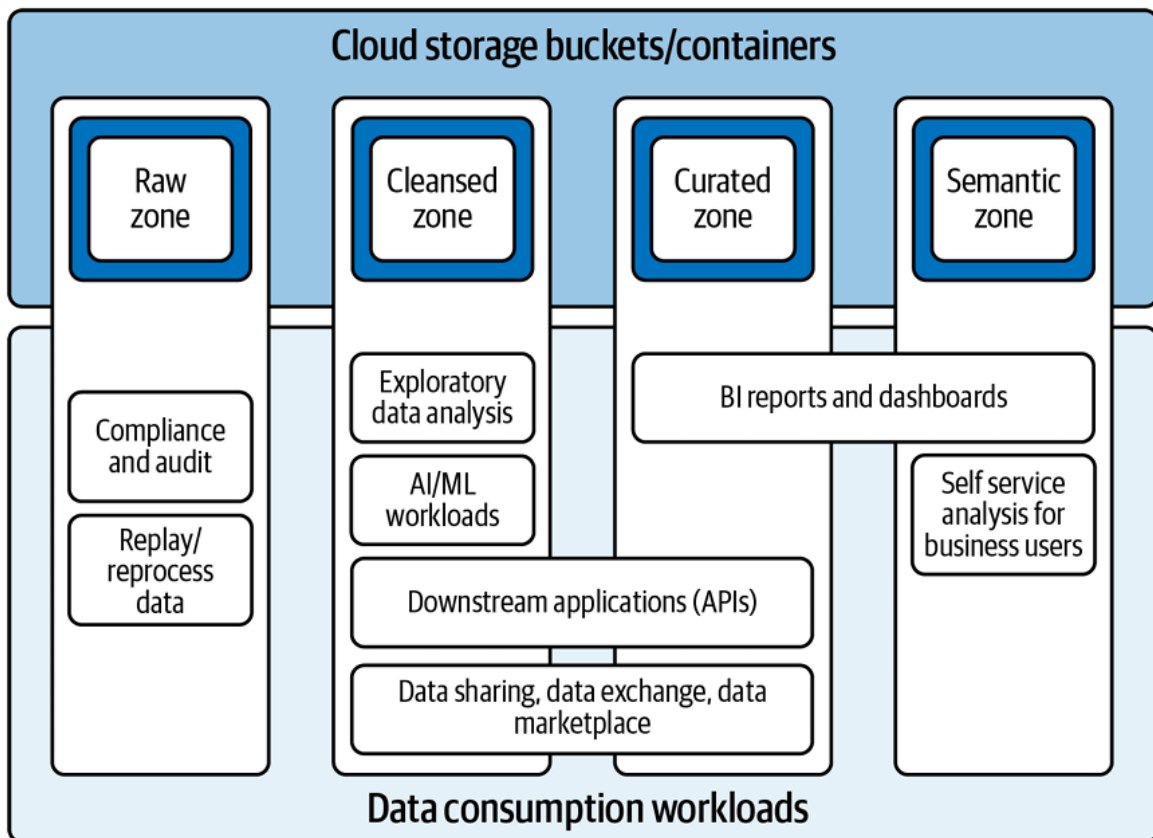
- Xem xét việc sử dụng phương pháp mô hình hóa chiều (dimensional modeling), điển hình là lược đồ hình sao (star schema) hoặc lược đồ bông tuyết (snowflake schema), để thiết kế các bảng trong vùng này. Các mô hình này rất hiệu quả cho việc cải thiện hiệu suất truy vấn BI.
- Khi tải dữ liệu, cần đảm bảo thiết lập các mối quan hệ phụ thuộc logic thích hợp giữa các bảng chiều (dimensions) và bảng sự kiện (facts).
- Nên lưu giữ toàn bộ dữ liệu lịch sử (bao gồm cả các phiên bản cũ hơn của bản ghi nếu cần) trong các bảng của vùng đã tuyển chọn. Điều này rất quan trọng để thực hiện các phân tích mô tả, phân tích xu hướng và tạo ra các thông tin chi tiết có giá trị theo thời gian.

d) **Vùng Ngữ Nghĩa (Semantic Zone):** Một số tổ chức có thể quyết định tạo thêm một vùng ngữ nghĩa như một lớp ảo hóa hoặc vật lý hóa nằm trên vùng đã tuyển chọn. Mục đích là để cung cấp dữ liệu theo một cách dễ hiểu và dễ truy cập hơn cho người dùng nghiệp vụ, thường ánh xạ trực tiếp tới các thuật ngữ và khái niệm kinh doanh. Khi thiết kế vùng ngữ nghĩa, cần lưu ý các điểm sau:

- Người dùng nghiệp vụ và những người dùng không chuyên sâu về kỹ thuật thường sử dụng các bảng hoặc view của vùng ngữ nghĩa cho các hoạt động phân tích tự phục vụ. Do đó, việc thiết kế các đối tượng dữ liệu này cần phải xem xét đến các hình mẫu người dùng dữ liệu (data personas) và nhu cầu cụ thể của họ.

- Lớp này có thể bao gồm các view logic, view tổng hợp sẵn (materialized views) chứa các kết quả tính toán trước (ví dụ: KPI, chỉ số tổng hợp) để tăng tốc độ truy vấn cho các báo cáo và dashboard thường dùng.
- Cần cân nhắc kỹ lưỡng các công cụ và công nghệ sẽ được sử dụng để thiết kế, mô hình hóa và quản lý lớp ngữ nghĩa này. Ví dụ, các công cụ BI hiện đại thường có khả năng xây dựng lớp ngữ nghĩa riêng, hoặc có thể sử dụng các giải pháp chuyên biệt như Atscale. Đối với các giải pháp triển khai tùy chỉnh, việc tận dụng các view (thường hoặc tổng hợp sẵn) là phổ biến.

4. Khối lượng công việc và đối tượng tiêu thụ dữ liệu trong hệ thống Lakehouse



Hình 1.7: Sơ đồ tổ chức tiêu thụ dữ liệu [11]

Trong kiến trúc lakehouse, các thành phần tiêu thụ dữ liệu cho phép người dùng truy cập, phân tích, truy vấn và sử dụng dữ liệu. Các hoạt động tiêu thụ dữ liệu bao gồm phân tích tương tác, Phân tích Kinh doanh (BI), Trí tuệ Nhân tạo/Học máy (AI/ML) và chia sẻ với các ứng dụng hạ nguồn. Các đối tượng

người dùng tiêu thụ dữ liệu có thể là các kỹ sư dữ liệu, nhà phân tích dữ liệu, kỹ sư BI, kỹ sư ML, nhà khoa học dữ liệu, người dùng nghiệp vụ, và các ứng dụng hạ nguồn.

Khi triển khai các quy trình tiêu thụ và cung cấp dữ liệu, có một số cân nhắc thiết kế và thực hành tốt cần lưu ý:

a) **Cân nhắc về Tải công việc:**

- Các loại tải công việc khác nhau và các đối tượng người dùng khác nhau sẽ sử dụng dữ liệu từ các vùng lưu trữ khác nhau trong lakehouse. Bạn nên tận dụng dữ liệu từ các vùng phù hợp cho từng loại tải công việc tương ứng. Cụ thể:
 - *Vùng Thô (Raw)*: Đối với các yêu cầu liên quan đến tuân thủ và kiểm toán, bạn có thể cần truy cập vào các tệp thô được gửi từ hệ thống nguồn. Nếu các tệp dữ liệu thô này đã được đưa vào kho lưu trữ dài hạn, bạn có thể sao chép chúng từ vùng lưu trữ dài hạn sang vùng thô. Bạn cũng có thể sử dụng các tệp dữ liệu thô để xử lý lại hoặc phát lại bất kỳ tệp nguồn nào khi cần.
 - *Vùng Đã làm sạch (Cleansed)*: Vùng này chứa dữ liệu đã được xác thực và làm sạch ở mức cơ bản. Dữ liệu từ vùng này có thể được sử dụng cho một số phân tích cơ bản hoặc làm đầu vào cho các mô hình ML cần dữ liệu gần với dạng thô nhưng đã qua kiểm tra chất lượng ban đầu.
 - *Vùng Đã tuyển chọn (Curated)*: Vùng này chứa dữ liệu đã được tổ chức theo các quy trình nghiệp vụ, thường sử dụng mô hình chiều. Đây là vùng lý tưởng cho các công cụ BI và người dùng nghiệp vụ thực hiện báo cáo và phân tích mô tả.
 - *Vùng Ngữ nghĩa (Semantic)*: Nếu có, vùng này cung cấp một lớp trừu tượng hóa dữ liệu đã được mô hình hóa, thường dưới dạng các view tổng hợp sẵn (materialized views) với các kết quả đã được tính toán trước. Điều này giúp người dùng nghiệp vụ dễ dàng hiểu và truy vấn dữ liệu hơn, đồng thời cải thiện hiệu suất cho các truy vấn BI.
- Các công cụ BI và các ứng dụng hạ nguồn thường sử dụng dữ liệu từ vùng đã tuyển chọn và vùng ngữ nghĩa. Các mô hình ML và các ứng dụng AI có thể sử dụng dữ liệu từ vùng đã làm sạch và vùng đã tuyển

chọn.

- Bạn có thể xem xét việc chia sẻ dữ liệu trong lakehouse với các đối tác và khách hàng. Tính năng chia sẻ dữ liệu cho phép các tổ chức triển khai một "thị trường dữ liệu" để bán và mua dữ liệu, cũng như tạo các "phòng sạch dữ liệu" để chia sẻ dữ liệu với các đối tác cho các mục đích cộng tác an toàn. Nên tận dụng các tính năng của định dạng bảng mở, như giao thức Delta Sharing của Delta Lake, để triển khai các trường hợp sử dụng chia sẻ dữ liệu này.

b) Thực hành Tốt:

- Không tạo các bản sao dữ liệu không cần thiết khi chia sẻ nó với người dùng. Khác với các kho dữ liệu truyền thống, bạn không nhất thiết phải trích xuất dữ liệu thành các tệp riêng biệt để chia sẻ với các ứng dụng hạ nguồn nếu có các phương thức chia sẻ trực tiếp hiệu quả hơn.
- Luôn sử dụng dữ liệu từ các vùng lưu trữ phù hợp của lakehouse cho từng mục đích tiêu thụ cụ thể để đảm bảo tính chính xác, hiệu quả và quản trị tốt.
- Tận dụng các bộ kết nối, API và các giao thức chia sẻ dữ liệu hiện đại để cung cấp dữ liệu một cách dễ dàng, an toàn và được kiểm soát cho các ứng dụng hạ nguồn.
- Tìm hiểu và tận dụng tối đa các tính năng liên quan của định dạng bảng mở đang sử dụng, ví dụ như giao thức Delta Sharing của Delta Lake, để tối ưu hóa việc chia sẻ và cộng tác dữ liệu.

1.2 Khảo sát về dữ liệu tin tức, thời sự

Trong kỷ nguyên thông tin số, tin tức và các sự kiện thời sự có vai trò ngày càng quan trọng, không chỉ cung cấp nhận thức về thế giới xung quanh mà còn tác động trực tiếp đến nhiều khía cạnh của đời sống kinh tế - xã hội. Việc hiểu rõ bản chất, nguồn gốc và các đặc điểm của loại dữ liệu này là bước đi tiên quyết để có thể xây dựng một hệ thống lưu trữ và phân tích hiệu quả.

1.2.1 Nhu cầu xây dựng hệ thống phân tích dữ liệu tin tức, thời sự

Sự bùng nổ của thông tin trực tuyến mang lại nguồn tài nguyên dữ liệu tin tức khổng lồ. Tuy nhiên, việc khai thác hiệu quả nguồn dữ liệu này vẫn còn nhiều hạn chế và đặt ra nhu cầu cấp thiết về một hệ thống phân tích chuyên biệt:

a) Hạn chế của các nền tảng tin tức hiện tại

- Hầu hết các trang web tin tức hiện nay chủ yếu tập trung vào việc cung cấp nội dung cho người đọc cuối mà thiếu đi các công cụ mạnh mẽ cho phép người dùng hoặc các tổ chức thực hiện thống kê, phân tích sâu hoặc theo dõi dữ liệu lịch sử một cách có hệ thống. Khả năng tìm kiếm và lọc tin thường giới hạn ở mức cơ bản, không đáp ứng được nhu cầu nghiên cứu hay phân tích xu hướng phức tạp.
- Việc trích xuất dữ liệu thủ công từ nhiều nguồn khác nhau để phục vụ phân tích là tốn kém thời gian, công sức và dễ gặp sai sót.

b) Tầm ảnh hưởng sâu rộng của thông tin thời sự

- Tin tức có tác động mạnh mẽ và trực tiếp đến nhiều lĩnh vực quan trọng. Ví dụ, trong lĩnh vực tài chính, một thông tin về chính sách kinh tế, tình hình doanh nghiệp, hoặc biến động thị trường thế giới có thể ngay lập tức ảnh hưởng đến giá cổ phiếu, tỷ giá hối đoái và các quyết định đầu tư. Các nhà đầu tư, nhà phân tích tài chính luôn cần cập nhật và phân tích tin tức để đưa ra những nhận định chính xác.
- Tương tự, tin tức về chính trị, xã hội, pháp luật cũng có thể tạo ra những thay đổi lớn trong dư luận, hành vi của người tiêu dùng, và các chiến lược hoạt động của doanh nghiệp.
- Việc thiếu một hệ thống tổng hợp và phân tích tin tức có thể dẫn đến việc bỏ lỡ các thông tin quan trọng, phản ứng chậm trễ trước các biến

động hoặc không nắm bắt được các xu hướng tiềm năng.

c) Lợi ích của việc phân tích dữ liệu tin tức một cách có hệ thống

- *Nắm bắt xu hướng và dự báo:* Phân tích dữ liệu tin tức lịch sử và theo thời gian thực giúp phát hiện các chủ đề nổi bật, xu hướng thông tin, và có thể hỗ trợ dự báo các sự kiện hoặc biến động trong tương lai.
- *Đánh giá tác động và quản lý rủi ro:* Hiểu được cách tin tức lan truyền và tác động đến các lĩnh vực cụ thể giúp các tổ chức đánh giá được rủi ro tiềm ẩn và xây dựng chiến lược ứng phó phù hợp.
- *Hỗ trợ ra quyết định dựa trên dữ liệu:* Cung cấp thông tin đầu vào khách quan và đa chiều cho các nhà quản lý, nhà hoạch định chính sách, và các nhà nghiên cứu.
- *Xây dựng kho tri thức lịch sử:* Lưu trữ và tổ chức dữ liệu tin tức một cách có hệ thống tạo ra một kho tri thức giá trị, phục vụ cho việc tra cứu và nghiên cứu lâu dài.

Chính vì những lý do trên, việc xây dựng một hệ thống chuyên biệt có khả năng thu thập, lưu trữ, xử lý và phân tích dữ liệu tin tức, thời sự là một nhu cầu thực tế và mang lại nhiều giá trị gia tăng.

1.2.2 Đặc điểm dữ liệu tin tức, thời sự

Dữ liệu tin tức, thời sự, với bản chất là các bài báo được đăng tải và cập nhật liên tục trên vô số trang web, mang những đặc điểm riêng biệt đòi hỏi sự cân nhắc kỹ lưỡng trong quá trình thu thập, xử lý và lưu trữ.

Đầu tiên, xét về cấu trúc, dữ liệu này thường được thu thập thông qua việc trích xuất thông tin từ mã nguồn HTML của các trang báo điện tử. Quá trình này dẫn đến định dạng dữ liệu bán cấu trúc (semi-structured). Cụ thể hơn, dữ liệu thường được lưu trữ dưới dạng các tệp JSON (JavaScript Object Notation). Tuy nhiên, do sự đa dạng trong thiết kế và cấu trúc của trang web, các tệp JSON này thường có cấu trúc schema phức tạp và lồng nhau. Mỗi trang báo có thể sử dụng các thẻ HTML, lớp (class), và định danh (ID) khác nhau để đánh dấu các thành phần của một bài viết như tiêu đề, tác giả, ngày đăng, nội dung chính, chuyên mục, từ khóa, v.v.

Thứ hai, một trong những đặc điểm nổi bật nhất của dữ liệu tin tức là khối lượng khổng lồ và tốc độ cập nhật chóng mặt. Tin tức được tạo ra và lan truyền

không ngừng, 24 giờ một ngày, 7 ngày một tuần, trên phạm vi toàn cầu. Mỗi sự kiện, dù lớn hay nhỏ, đều có thể trở thành một bản tin mới. Điều này đặt ra yêu cầu cao cho hệ thống thu thập dữ liệu. Hệ thống không chỉ cần có khả năng xử lý một lượng lớn dữ liệu ban đầu mà còn phải được thiết kế để có thể thu thập thông tin một cách thường xuyên, liên tục, thậm chí là theo thời gian thực hoặc gần thời gian thực đối với các nguồn tin quan trọng. Điều này đòi hỏi hạ tầng mạnh mẽ, băng thông lớn và các giải pháp lưu trữ có khả năng mở rộng linh hoạt để đáp ứng sự gia tăng không ngừng của dữ liệu. Các thuật toán và cơ chế thu thập cũng cần được tối ưu hóa để tránh tình trạng quá tải cho các máy chủ nguồn và đảm bảo việc cập nhật thông tin mới một cách nhanh chóng và hiệu quả.

Cuối cùng, mặc dù dự án hiện tại không tập trung vào việc lưu trữ các yếu tố đa phương tiện, không thể phủ nhận rằng tin tức, thời sự ngày nay ngày càng trở nên phong phú hơn với sự xuất hiện của hình ảnh, âm thanh (podcast, phỏng vấn), và video (clip tin tức, phóng sự). Đây là những thành phần quan trọng giúp tăng tính hấp dẫn, trực quan và khả năng truyền tải thông điệp của tin tức. Do đó, khi thiết kế hệ thống, việc tính đến khả năng mở rộng để đáp ứng nhu cầu lưu trữ và xử lý các định dạng dữ liệu này trong tương lai là một yếu tố cần thiết. Điều này có thể bao gồm việc lựa chọn các giải pháp lưu trữ có khả năng hỗ trợ các tệp dung lượng lớn, các cơ chế quản lý siêu dữ liệu (metadata) cho các tệp đa phương tiện, và khả năng tích hợp với các công cụ xử lý hình ảnh, âm thanh, video nếu cần. Một thiết kế hệ thống có tầm nhìn xa sẽ giúp tránh được những chi phí và sự phức tạp không cần thiết khi nhu cầu phát sinh trong tương lai.

Tóm lại, việc hiểu rõ các đặc điểm về cấu trúc bán cấu trúc phức tạp, khối lượng lớn và tốc độ cập nhật nhanh, cùng với sự đa dạng về định dạng bao gồm cả đa phương tiện, là nền tảng quan trọng để xây dựng một hệ thống quản lý và khai thác dữ liệu tin tức, thời sự hiệu quả và bền vững.

1.2.3 Khảo sát một số nguồn tin tức điện tử uy tín tại Việt Nam

Để đảm bảo chất lượng và độ tin cậy của dữ liệu đầu vào, dự án sẽ tập trung thu thập thông tin từ một trong số trang báo điện tử lớn, có uy tín và lượng thông tin phong phú tại Việt Nam. Dưới đây là ba trang báo lớn nhất Việt Nam:

a) VnExpress (vnexpress.net):



Hình 1.8: Logo trang báo VnExpress

- *Sơ lược:* Là một trong những báo điện tử tiếng Việt có lượng truy cập hàng đầu, thành lập năm 2001. VnExpress được biết đến với tốc độ cập nhật tin tức nhanh, đa dạng về chủ đề.
- *Nội dung chính:* Bao gồm các chuyên mục thời sự, kinh doanh, pháp luật, công nghệ (Số hóa), khoa học, thể thao, giải trí, đời sống. Các bài viết thường có độ dài vừa phải, cung cấp thông tin rõ ràng, kèm theo các siêu dữ liệu như ngày đăng, chuyên mục, tác giả (cho một số bài).
- *Định dạng dữ liệu thường gặp:* Chủ yếu là HTML khi thu thập từ website. Cấu trúc trang và bài viết tương đối ổn định, thuận lợi cho việc xây dựng bộ trích xuất dữ liệu.

b) **Dân trí (dantri.com.vn):**



Hình 1.9: Logo trang báo Dân Trí

- *Sơ lược:* Là cơ quan của Hội Khuyến học Việt Nam, báo điện tử Dân trí có lượng độc giả lớn, tập trung vào các vấn đề dân sinh, xã hội, giáo dục, y tế.
- *Nội dung chính:* Tin tức thời sự, các vấn đề xã hội được quan tâm, giáo dục, y tế, pháp luật, kinh doanh, thể thao. Các bài viết thường đi kèm với các thông tin như ngày đăng, chuyên mục.
- *Định dạng dữ liệu thường gặp:* HTML là định dạng chính khi truy cập qua web. Cấu trúc bài viết có những đặc điểm riêng cần được phân tích để trích xuất nội dung và siêu dữ liệu.

c) **VietnamNet (vietnamnet.vn):**



Hình 1.10: Logo trang báo VietNamNet

- *Sơ lược:* Thuộc Bộ Thông tin và Truyền thông, VietnamNet là một trong những trang báo điện tử có uy tín và lịch sử lâu đời.
- *Nội dung chính:* Cung cấp thông tin đa dạng các lĩnh vực như chính trị, kinh tế, xã hội, công nghệ thông tin (ICT), giáo dục, đời sống, văn hóa, quốc tế. Các bài viết thường có chất lượng, được biên tập kỹ lưỡng.
- *Định dạng dữ liệu thường gặp:* Dữ liệu chủ yếu dưới dạng HTML. Trang báo có nhiều chuyên mục con, cần xác định rõ cấu trúc để thu thập hiệu quả.

Việc lựa chọn các trang báo này dựa trên uy tín, mức độ phổ biến, sự đa dạng và khối lượng thông tin cung cấp hàng ngày, cũng như tính ổn định tương đối của cấu trúc website để thuận lợi cho việc thu thập dữ liệu bán cấu trúc.

1.2.4 Lựa chọn kiến trúc Lakehouse cho dữ liệu tin tức

Với các đặc điểm của dữ liệu tin tức đã phân tích, đặc biệt là tính bán cấu trúc của dữ liệu web và nhu cầu lưu trữ, xử lý linh hoạt, việc lựa chọn kiến trúc hệ thống là vô cùng quan trọng. Kiến trúc Lakehouse được đề xuất cho dự án này vì những lý do sau:

a) **Giải quyết hạn chế của các kiến trúc truyền thống đối với dữ liệu tin tức:**

- *Data Warehouse truyền thống:* Thường yêu cầu schema-on-write (định nghĩa cấu trúc trước khi ghi), điều này không linh hoạt với dữ liệu HTML từ các trang báo có thể có sự thay đổi nhỏ về cấu trúc hoặc khi muốn bổ sung các trường thông tin mới. Chi phí bản quyền và vận hành cũng có thể cao.

- *Data Lake đơn thuần*: Mặc dù rất tốt cho việc lưu trữ dữ liệu thô đa dạng (bao gồm cả HTML), Data Lake thuần túy thường thiếu các cơ chế quản lý giao dịch (ACID), khó đảm bảo chất lượng dữ liệu ở các bước xử lý sau, và có thể trở thành "data swamp" (đầm lầy dữ liệu) nếu không được quản trị tốt. Việc truy vấn và phân tích trực tiếp trên dữ liệu thô cũng kém hiệu quả.

b) Ưu điểm vượt trội của Lakehouse đối với dữ liệu tin tức bán cấu trúc và có cấu trúc:

- *Lưu trữ linh hoạt dữ liệu bán cấu trúc và có cấu trúc*: Lakehouse cho phép lưu trữ dữ liệu linh hoạt ở định dạng là file. Nó có thể là file csv, json, parquet, ...
- *Hỗ trợ giao dịch ACID*: Thông qua các định dạng bảng mở như Delta Lake, Apache Iceberg, hoặc Apache Hudi, Lakehouse mang lại các đảm bảo về tính toàn vẹn dữ liệu (ACID) cho các bảng đã qua xử lý. Điều này rất quan trọng khi thực hiện các thao tác cập nhật, làm giàu siêu dữ liệu hoặc xây dựng các bảng tổng hợp cho phân tích.
- *Khả năng Schema Enforcement và Schema Evolution*: Lakehouse cho phép định nghĩa và kiểm soát schema cho dữ liệu đã được cấu trúc hóa, đồng thời hỗ trợ việc thay đổi schema một cách linh hoạt khi có sự thay đổi từ nguồn hoặc yêu cầu nghiệp vụ mới mà không làm gián đoạn hệ thống.
- *Kết hợp lưu trữ chi phí thấp và truy vấn hiệu năng cao*: Tận dụng các giải pháp lưu trữ đối tượng hiệu quả về chi phí và các công cụ truy vấn mạnh mẽ (như Spark SQL, Trino) có thể hoạt động trực tiếp trên dữ liệu trong Lakehouse.
- *Nền tảng thống nhất cho xử lý và phân tích dữ liệu*: Lakehouse cung cấp một nền tảng duy nhất để thực hiện các quy trình từ thu thập, tiền xử lý, làm sạch dữ liệu bán cấu trúc đến việc lưu trữ dữ liệu có cấu trúc và thực hiện các phân tích dựa trên SQL hoặc các công cụ khác. Điều này giúp đơn giản hóa kiến trúc tổng thể.

Với việc dự án tập trung vào dữ liệu văn bản (HTML, metadata), kiến trúc Lakehouse cung cấp sự cân bằng lý tưởng giữa tính linh hoạt trong việc xử lý dữ liệu bán cấu trúc và độ tin cậy, khả năng quản trị của dữ liệu có cấu trúc, làm

cho nó trở thành lựa chọn phù hợp.

1.2.5 Các ứng dụng tiềm năng của hệ thống phân tích dữ liệu tin tức

Sau khi xây dựng hệ thống Lakehouse để lưu trữ và xử lý dữ liệu tin tức, một số ứng dụng phân tích tiềm năng có thể được triển khai bao gồm:

a) Thống kê và theo dõi xu hướng chủ đề:

- Xác định tần suất xuất hiện của các từ khóa, cụm từ, hoặc thực thể tên riêng (tên người, tổ chức, địa danh) trong các bài báo theo thời gian.
- Phát hiện các chủ đề đang nổi lên hoặc giảm sút mức độ quan tâm dựa trên phân tích nội dung tin tức từ nhiều nguồn.

b) Xây dựng kho dữ liệu tin tức lịch sử cho tra cứu và nghiên cứu:

- Lưu trữ có tổ chức các bài báo đã thu thập, cho phép tìm kiếm và truy xuất thông tin theo nhiều tiêu chí (ví dụ: từ khóa, ngày đăng, chuyên mục, nguồn báo).
- Cung cấp nguồn dữ liệu cho các nghiên cứu về lịch sử sự kiện, biến đổi ngôn ngữ báo chí, hoặc các phân tích xã hội học dựa trên nội dung truyền thông.

c) Phân tích sơ bộ mối liên hệ giữa tin tức và các yếu tố bên ngoài:

- Đối chiếu sự xuất hiện của các tin tức liên quan đến một ngành hoặc công ty cụ thể với biến động giá cổ phiếu của công ty đó để tìm kiếm các mối tương quan tiềm năng.
- Theo dõi mật độ tin tức về các chủ đề kinh tế vĩ mô (lạm phát, lãi suất, GDP) để có cái nhìn tổng quan về bối cảnh thông tin.

d) Cung cấp dữ liệu nền cho các ứng dụng phân tích nâng cao trong tương lai:

- Có thể cấp quyền hoặc viết api lấy dữ liệu từ vùng clean cho các tác vụ phân tích AI/ML. Họ sẽ theo dõi giá trị cảm xúc bài báo, tiêu cực, tích cực, thái độ người đọc,... để đưa ra các dự đoán về thị trường, những xu hướng hay nắm bắt được sở thích chủ đề người đọc quan tâm.

Các ứng dụng này, dù ở quy mô nào, đều hướng tới việc biến khối lượng lớn dữ liệu tin tức thành những thông tin hữu ích, hỗ trợ hiểu biết sâu sắc hơn về các sự kiện và xu hướng đang diễn ra.

Chương 2

Phân tích và thiết kế hệ thống

2.1 Nguồn dữ liệu

Việc thu thập dữ liệu là bước khởi đầu và đóng vai trò vô cùng quan trọng trong bất kỳ dự án phân tích dữ liệu hay xây dựng mô hình học máy nào. Chất lượng và tính phù hợp của dữ liệu nguồn sẽ ảnh hưởng trực tiếp đến kết quả cuối cùng. Trong khuôn khổ của nghiên cứu này, tôi tập trung vào việc thu thập dữ liệu tin tức từ các nguồn trực tuyến.

2.1.1 Lựa chọn nguồn lấy dữ liệu

Thế giới Internet hiện nay cung cấp một lượng lớn thông tin tin tức từ vô số các trang web khác nhau. Điều này tạo ra sự thuận lợi trong việc tìm kiếm nguồn dữ liệu. Tuy nhiên, không phải tất cả các nguồn đều đáng tin cậy và phù hợp cho mục đích nghiên cứu. Do đó, việc lựa chọn một nguồn dữ liệu đáp ứng các tiêu chí về tính chính thống, uy tín và đã được kiểm chứng là ưu tiên hàng đầu.

Các tiêu chí cụ thể để lựa chọn nguồn dữ liệu bao gồm:

- **Tính đa dạng về chủ đề:** Nguồn dữ liệu cần cung cấp thông tin bao quát nhiều lĩnh vực khác nhau của đời sống xã hội như chính trị, kinh tế, văn hóa, thể thao, công nghệ, v.v. Điều này giúp đảm bảo bộ dữ liệu thu thập được có tính đại diện cao.
- **Tính cập nhật liên tục:** Trang web phải thường xuyên đăng tải các bài viết mới, phản ánh kịp thời các sự kiện đang diễn ra.
- **Lượng người dùng lớn và tính tương tác cao:** Một trang web có lượng truy cập lớn và nhiều tương tác từ người đọc (bình luận, chia sẻ) thường cho

thấy mức độ uy tín và sự quan tâm của cộng đồng đối với nội dung. Dữ liệu về tương tác cũng có thể là một nguồn thông tin giá trị.

- **Khả năng lưu trữ bài viết lịch sử:** Việc truy cập được các bài báo đã xuất bản trong quá khứ là cần thiết để phân tích xu hướng hoặc các sự kiện theo dòng thời gian.
- **Cấu trúc trang web tương đối ổn định:** Mặc dù các trang web động luôn có sự thay đổi, một cấu trúc cơ bản ổn định sẽ giúp quá trình thu thập dữ liệu (crawling) diễn ra thuận lợi hơn và ít phải bảo trì mã nguồn hơn.

Dựa trên các tiêu chí trên, tôi đã tiến hành khảo sát và đánh giá một số trang tin tức điện tử lớn tại Việt Nam. Cuối cùng, **VnExpress** đã được lựa chọn làm nguồn dữ liệu chính. VnExpress là một trong những báo điện tử có lượng truy cập hàng đầu, nội dung phong phú, đa dạng, được cập nhật liên tục và có hệ thống lưu trữ bài viết tốt.

2.1.2 Những thách thức và giải pháp trong quá trình thu thập dữ liệu

1. Thách thức

Mặc dù VnExpress là một nguồn dữ liệu phong phú, quá trình trích xuất thông tin từ trang web này cũng gặp phải một số thách thức kỹ thuật đáng kể:

- **Trang web động:** VnExpress sử dụng nhiều JavaScript để tải nội dung một cách động. Điều này có nghĩa là nội dung không hiển thị đầy đủ ngay khi tải HTML ban đầu. Do đó, các thư viện chỉ dựa trên việc gửi yêu cầu HTTP và phân tích HTML tĩnh như `requests` kết hợp với `BeautifulSoup4` sẽ không đủ để lấy được toàn bộ dữ liệu mong muốn, đặc biệt là các phần tử được tải sau hoặc dựa trên hành động của người dùng.
- **Tải dữ liệu bình luận và tương tác:** Thông tin về các bình luận của độc giả, số lượt thích, và các loại tương tác khác thường được tải không đồng bộ khi người cần dùng cuộn trang, chờ tải hoặc nhấp vào các nút "xem thêm". Việc này đòi hỏi phải mô phỏng các hành động của người dùng để kích hoạt việc tải dữ liệu này.
- **Phân trang (Pagination):** Các bài báo trong cùng một chuyên mục hoặc kết quả tìm kiếm thường được chia thành nhiều trang khác nhau. Để thu thập toàn bộ dữ liệu, cần phải tự động chuyển qua các trang này, thường

bằng cách mô phỏng việc nhấp chuột vào các nút điều hướng trang.

- **Chính sách chống thu thập dữ liệu tự động:**

2. Giải pháp

Để vượt qua những thách thức nêu trên, tôi đã quyết định sử dụng kết hợp hai thư viện mạnh mẽ trong Python: **Selenium** và **BeautifulSoup4**.

- **Selenium:** Đóng vai trò như một trình duyệt web tự động. Selenium có khả năng tương tác với các trang web động bằng cách thực thi JavaScript, mô phỏng các hành động của người dùng như nhấp chuột, cuộn trang, điền vào biểu mẫu, và chờ đợi các phần tử trên trang được tải hoàn chỉnh. Nhờ đó, chúng tôi có thể truy cập được vào nội dung HTML đầy đủ của trang, bao gồm cả các phần được tải động.
- **BeautifulSoup4 (bs4):** Sau khi Selenium đã tải và xử lý xong trang web, trả về mã nguồn HTML hoàn chỉnh, BeautifulSoup4 sẽ được sử dụng để phân tích (parse) cây DOM (Document Object Model) của HTML đó. Với bs4, chúng tôi có thể dễ dàng tìm kiếm, trích xuất các thẻ HTML cụ thể và nội dung văn bản bên trong chúng dựa trên các thuộc tính như tên thẻ, class, id, v.v.

Quy trình tổng quát sẽ là: Selenium điều khiển trình duyệt để mở URL bài báo, thực hiện các hành động cần thiết (như cuộn trang để tải bình luận), sau đó lấy mã nguồn HTML đã được render đầy đủ. Mã nguồn này sau đó được chuyển cho BeautifulSoup4 để tiến hành bóc tách và thu thập các trường thông tin có chọn lọc.

2.1.3 Hiểu về cấu trúc dữ liệu cần thu thập

1. Các trường dữ liệu sẽ thu thập

Mỗi bài báo trên VnExpress [5] chứa đựng rất nhiều thông tin có giá trị, không chỉ giới hạn ở nội dung chính của bài viết mà còn bao gồm các siêu dữ liệu (metadata) quan trọng liên quan đến bài báo đó. Việc xác định rõ ràng các trường thông tin cần thiết sẽ giúp quá trình trích xuất diễn ra hiệu quả và dữ liệu thu thập được sẽ có cấu trúc, dễ dàng cho việc lưu trữ và phân tích sau này.

Dưới đây là danh sách các trường thông tin dự kiến sẽ được thu thập từ mỗi bài báo:

- `title`: Tiêu đề chính của bài báo.
- `url`: Đường dẫn URL duy nhất của bài báo.
- `description`: Đoạn mô tả ngắn (meta description) của bài báo, thường được hiển thị trong kết quả tìm kiếm.
- `topic`: Chủ đề chính của bài báo (ví dụ: "Thế giới", "Kinh doanh", "Thể thao").
- `sub_topic`: Chủ đề phụ hoặc chuyên mục con, nếu có (ví dụ: trong chủ đề "Thế giới" có thể có "Quân sự", "Phân tích").
- `keywords`: Danh sách các từ khóa liên quan đến nội dung bài báo, do tòa soạn gán thẻ hoặc được trích xuất tự động.
- `publish_date`: Ngày và giờ bài báo được xuất bản.
- `author`: Tên tác giả hoặc nhóm tác giả của bài báo.
- `references`: Danh sách các nguồn tin tham khảo (nếu có) được đề cập ở cuối bài báo.
- `main_content`: Toàn bộ nội dung văn bản chính của bài báo.
- `comment_count`: Tổng số lượng bình luận của độc giả cho bài báo đó.
- `top_comments`: Danh sách một số bình luận nổi bật hoặc được nhiều người tương tác nhất. Mỗi bình luận trong danh sách này sẽ bao gồm:
 - `commenter_name`: Tên người bình luận.
 - `comment_content`: Nội dung bình luận.
 - `total_likes`: Tổng số lượt thích hoặc tương tác tích cực cho bình luận đó.
 - `interaction_details`: Chi tiết các loại tương tác (ví dụ: "Thích", "Vui", "Buồn") và số lượng tương ứng cho mỗi loại (nếu có thể thu thập).

2. Định dạng cấu trúc dữ liệu của một bài báo

Sau khi thu thập, mỗi bài báo cùng với các thông tin liên quan sẽ được tổ chức và lưu trữ dưới dạng một đối tượng JSON (JavaScript Object Notation). JSON được chọn vì tính linh hoạt, dễ đọc bởi cả người và máy. Đồng thời, vì dữ liệu là các văn bản, nhiều metadata, thì Json được xem là một định dạng hợp lý nhất cho trường hợp này.

Dưới đây là một ví dụ minh họa cấu trúc JSON dự kiến cho một bài báo, bao gồm các trường dữ liệu chính đã được xác định. Những dữ liệu này đều được

chọn lọc và được xem là có ý nghĩa cho mục đích phân tích sau này:

```

1 {
2   "title": "...",
3   "url": "https://example.com/...",
4   "description": "...",
5   "topic": "...",
6   "sub_topic": "...",
7   "keywords": [
8     "keyword1",
9     "keyword2",
10    "...",
11  ],
12  "publish_date": "YYYY-MM-DDTHH:MM:SS+TZ",
13  "author": "...",
14  "references": [
15    "...",
16    "...",
17  ],
18  "main_content": "...",
19  "comment_count": 0,
20  "top_comments": [
21    {
22      "commenter_name": "...",
23      "comment_content": "...",
24      "total_likes": 0,
25      "interaction_details": {
26        "key_reaction_1": 0,
27        "key_reaction_2": 0
28      }
29    },
30    {
31      "commenter_name": "...",
32      "comment_content": "...",
33      "total_likes": 0,
34      "interaction_details": {
35        "key_reaction_1": 0,
36        "key_reaction_2": 0
37      }
38    }
39  ]
40 }

```

Listing 2.1: Ví dụ cấu trúc JSON cho một bài báo

Việc xác định rõ cấu trúc này từ đầu sẽ giúp đảm bảo tính nhất quán của dữ liệu thu thập được, tạo điều kiện thuận lợi cho các bước xử lý, phân tích và xây dựng mô hình ở giai đoạn sau.

2.2 Thiết kế kiến trúc hệ thống

Trong phần này, tôi sẽ đi sâu vào các bước phân tích và thiết kế hệ thống Lakehouse, hướng đến việc xây dựng một nền tảng dữ liệu (Data Platform) hoàn

chính và hiệu quả.

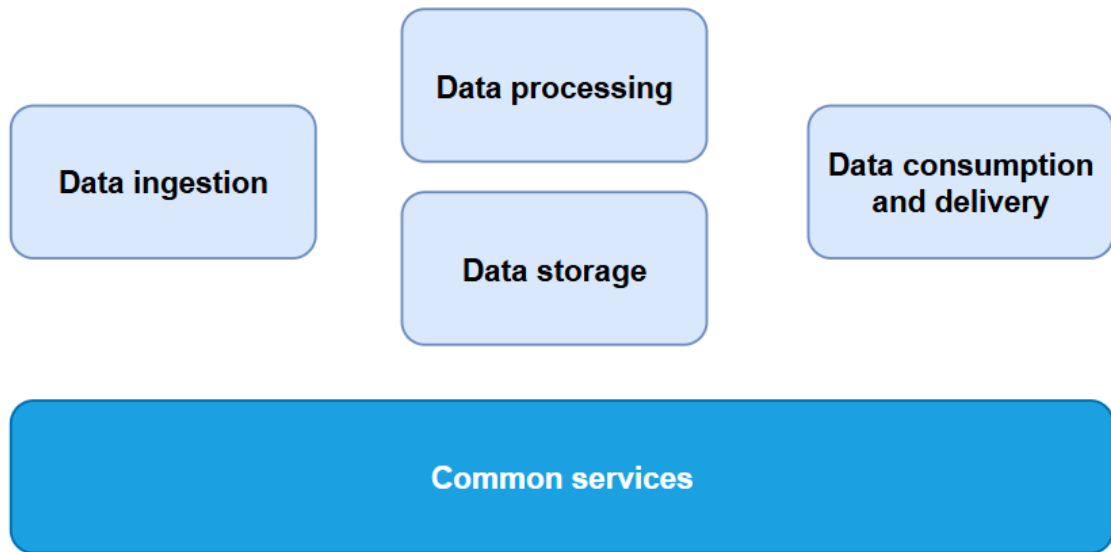
2.2.1 Bản thiết kế kiến trúc tổng thể

Trước khi xây dựng một hệ thống dữ liệu lớn và vững chắc, việc đầu tiên và quan trọng nhất là phải bắt đầu từ những thiết kế nền tảng. Trong quá trình thiết kế và triển khai một nền tảng lakehouse, việc tạo ra một bản Architecture Blueprint (bản thiết kế kiến trúc tổng thể) đóng vai trò là bước khởi đầu then chốt. Bản blueprint này chính là bản thiết kế chủ đạo, định hướng cho toàn bộ hệ thống mà chúng ta dự định xây dựng.

Ở bước đầu tiên, chúng ta cần phác thảo một bản blueprint ở cấp độ cao, minh họa các thành phần cốt lõi của nền tảng. Một sơ đồ đơn giản nhất về bất kỳ nền tảng dữ liệu hiện đại nào cũng sẽ thường bao gồm năm thành phần cốt lõi sau:

- **Data Ingestion (Thu thập dữ liệu):** Quá trình thu thập dữ liệu từ các nguồn khác nhau.
- **Data Storage (Lưu trữ dữ liệu):** Nơi lưu trữ dữ liệu đã thu thập, có thể bao gồm các lớp khác nhau như raw, processed, curated.
- **Data Processing (Xử lý dữ liệu):** Các tác vụ biến đổi, làm sạch, và làm giàu dữ liệu.
- **Data Consumption and Delivery (Tiêu thụ và Phân phối dữ liệu):** Cách thức người dùng và các ứng dụng khác truy cập và sử dụng dữ liệu.
- **Common Services (Các dịch vụ chung):** Bao gồm các dịch vụ hỗ trợ quản lý, giám sát, điều phối luồng công việc, quản trị danh mục dữ liệu, và đảm bảo an ninh, tuân thủ.

Sơ đồ dưới đây minh họa một kiến trúc cấp cao cho nền tảng dữ liệu, bao gồm các thành phần cốt lõi vừa được đề cập.



Hình 2.1: Kiến trúc cấp cao cho nền tảng dữ liệu

Dựa trên mẫu kiến trúc, tôi sẽ bổ sung các yếu tố kỹ thuật chi tiết hơn vào từng thành phần cốt lõi này. Bản Architecture Blueprint cho một nền tảng dữ liệu được xây dựng theo kiến trúc lakehouse sẽ cụ thể hóa các công nghệ, định dạng lưu trữ, và luồng dữ liệu đặc thù của mô hình này. Bản Architecture Blueprint này không chỉ giúp xác định rõ các thành phần cốt lõi và sự phụ thuộc lẫn nhau giữa chúng, mà còn là cơ sở vững chắc để lựa chọn ngăn xếp công nghệ (tech stack) phù hợp. Quan trọng hơn, nó giúp thiết lập các nguyên tắc hướng dẫn và tiêu chuẩn kỹ thuật nhằm đảm bảo nền tảng được xây dựng đồng bộ, nhất quán với chiến lược dữ liệu tổng thể và tầm nhìn của tổ chức.

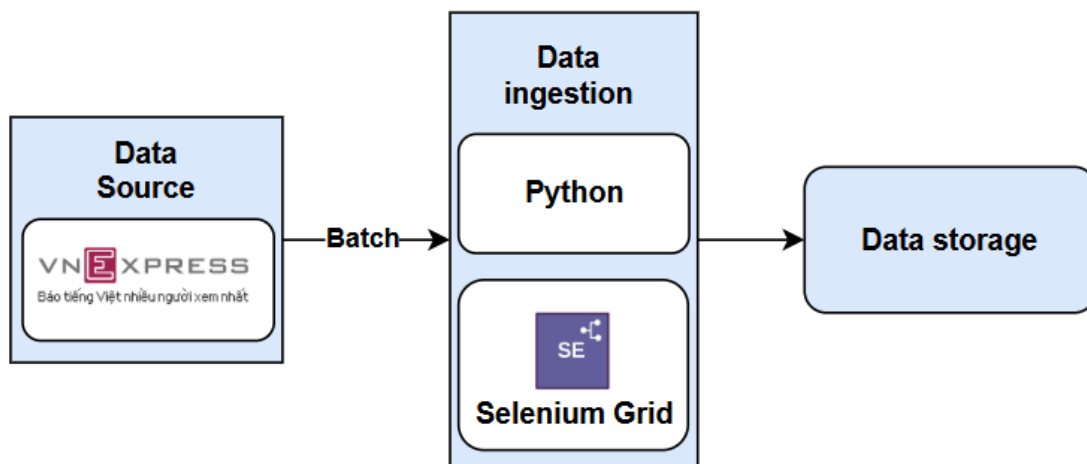
Từ bản thiết kế nền tảng này, tôi sẽ tiến hành lựa chọn các công nghệ cụ thể và phát triển kiến trúc chi tiết hơn cho dự án Lakehouse của mình. Sau khi có được bản blueprint tổng thể, bước tiếp theo là thiết kế và triển khai cẩn thận từng thành phần, dựa trên các yêu cầu nghiệp vụ, ràng buộc kỹ thuật và các cân nhắc về chi phí, hiệu năng cũng như khả năng mở rộng của hệ thống.

2.2.2 Data Ingestion

Như đã phân tích trong các phần trước, nguồn dữ liệu chính cho dự án này là nội dung tin tức từ các trang web cụ thể. Để thu thập dữ liệu này một cách hiệu quả, tôi đã lựa chọn phương pháp trích xuất dữ liệu tự động (web crawling/scraping). Ngôn ngữ lập trình **Python**, với sự hỗ trợ của thư viện

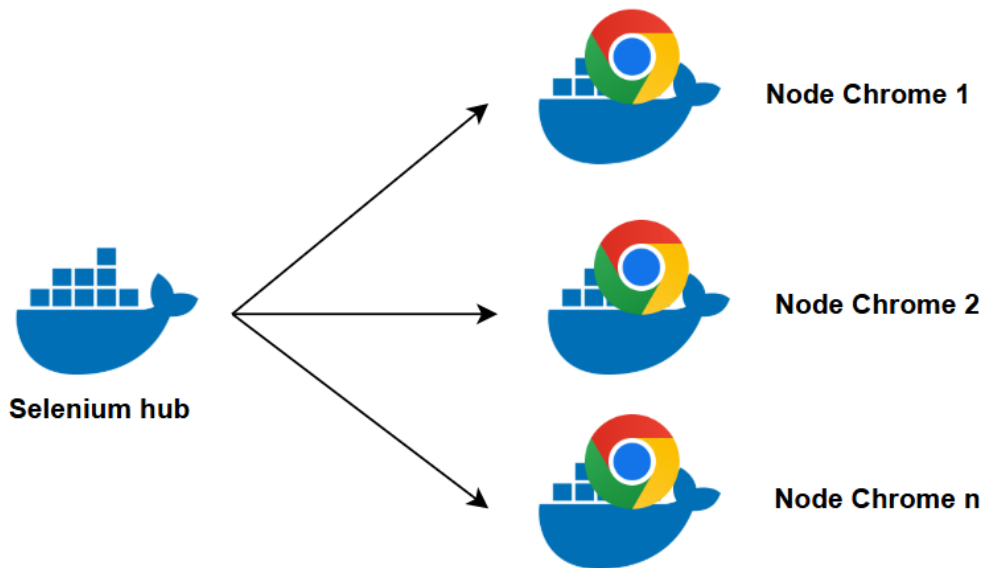
Selenium, được tôi sử dụng làm công cụ chính để thực hiện quá trình này. Selenium cho phép tương tác với các trang web động, xử lý JavaScript, và mô phỏng hành vi người dùng, nhờ đó tôi có thể thu thập được nội dung một cách đầy đủ nhất.

Dữ liệu sau khi thu thập sẽ được tôi cấu trúc hóa và chuẩn bị cho việc nhập vào hệ thống lưu trữ. Tôi đã thiết kế quy trình nhập liệu cho dự án theo phương thức **batch processing**. Cụ thể, dữ liệu sẽ được thu thập, xử lý sơ bộ và nạp vào hệ thống theo từng lô. Các lô này được tôi phân chia dựa trên các **chủ đề (topic)** của tin tức và theo **ngày xuất bản (publish date)**. Cách tiếp cận này không chỉ giúp tôi quản lý luồng dữ liệu dễ dàng hơn mà còn tối ưu hóa cho các bước xử lý và truy vấn dữ liệu về sau.



Hình 2.2: Quy trình Thu thập và Nhập liệu trong dự án

Để nâng cao hiệu suất và khả năng mở rộng của quá trình thu thập dữ liệu, đặc biệt khi cần xử lý một lượng lớn trang web hoặc cập nhật dữ liệu thường xuyên cho dự án, tôi đã quyết định triển khai giải pháp sử dụng **Selenium Grid** [10].



Hình 2.3: Mô hình Selenium Grid

Về cơ bản, Selenium Grid mà tôi thiết lập hoạt động như một hệ thống phân tán bao gồm một **Hub** (trung tâm điều phối) và nhiều **Nodes** (các máy thực thi).

- **Hub**: Trong hệ thống của tôi, Hub đóng vai trò là điểm trung tâm tiếp nhận các yêu cầu thực thi tác vụ duyệt web (cụ thể là các yêu cầu crawl dữ liệu từ các nguồn đã xác định). Hub sẽ quản lý một danh sách các Nodes đang sẵn sàng và phân phối các yêu cầu này đến Node phù hợp để thực thi.
- **Nodes**: Là các máy tính (hoặc container) riêng lẻ trong cụm của tôi, nơi các phiên trình duyệt thực sự được khởi tạo và chạy để thực hiện việc crawl dữ liệu. Mỗi Node được tôi cấu hình để có thể chạy nhiều phiên trình duyệt song song, tùy thuộc vào tài nguyên của nó.

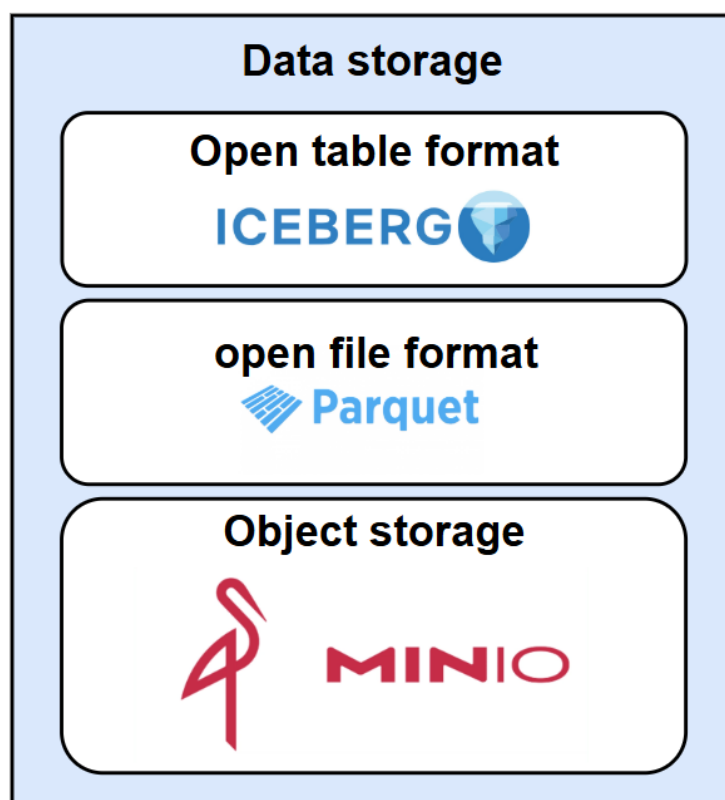
Với kiến trúc này, các tác vụ thu thập dữ liệu (ví dụ: crawl một danh sách các URL hoặc một chuyên mục cụ thể theo kế hoạch) sẽ được tôi đưa vào một hàng đợi (queue). Hub sẽ liên tục kiểm tra hàng đợi này và khi có Node rảnh, nó sẽ tự động gán tác vụ tiếp theo cho Node đó thực thi. Cơ chế này cho phép tôi thực hiện việc thu thập dữ liệu **song song** trên nhiều luồng, giúp rút ngắn đáng kể thời gian cần thiết so với việc crawl tuần tự. Toàn bộ logic điều khiển Hub, phân phối tác vụ cho Nodes, và bản thân các script crawl trên từng Node đều được tôi lập trình và quản lý bằng Python, tận dụng tối đa các tính năng của thư viện Selenium. Giải pháp Selenium Grid mà tôi áp dụng không chỉ giúp tăng tốc độ thu thập dữ liệu mà còn cải thiện đáng kể tính ổn định và khả năng chịu lỗi của

toàn bộ hệ thống thu thập dữ liệu cho dự án.

2.2.3 Data Storage

1. Tổng quan thành phần chính trong vùng lưu trữ

Trong kiến trúc Lakehouse mà tôi đang xây dựng cho dự án này, thành phần lưu trữ dữ liệu (Data Storage) đóng vai trò trung tâm, có thể coi là "trái tim" của toàn bộ hệ thống. Sự tiến bộ và khác biệt của kiến trúc Lakehouse so với các giải pháp truyền thống phần lớn đến từ cách tiếp cận hiện đại trong việc lưu trữ và quản lý dữ liệu. Cụ thể, nền tảng lưu trữ trong Lakehouse của tôi được xây dựng dựa trên sự kết hợp của ba trụ cột công nghệ chính: **Object Storage**, **Open File Format**, và **Open Table Format**.



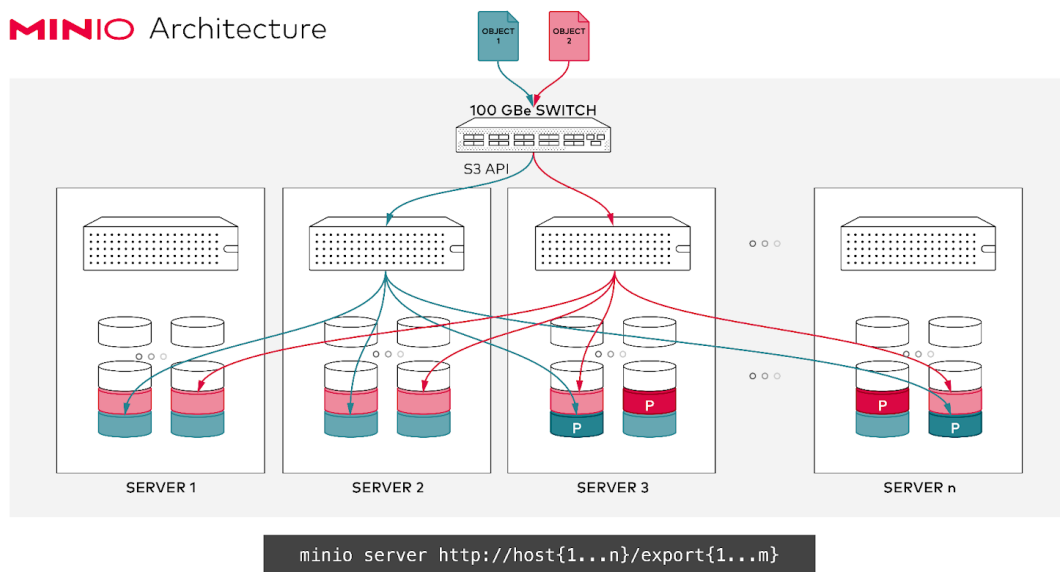
Hình 2.4: Các thành phần cốt lõi cấu thành vùng lưu trữ của Lakehouse trong dự án

Dưới đây, tôi sẽ trình bày chi tiết hơn về lựa chọn công nghệ cho từng thành phần trong giải pháp lưu trữ của dự án:

a) **Object Storage - MinIO:**

Đối với lớp lưu trữ vật lý, tôi đã quyết định sử dụng **MinIO** [8]. MinIO là một hệ thống lưu trữ đối tượng mã nguồn mở, hiệu năng cao và có khả năng tương thích API hoàn toàn với Amazon S3. Lựa chọn này mang lại cho dự án của tôi một số lợi thế quan trọng:

- **Khả năng mở rộng theo chiều ngang:** MinIO cho phép tôi dễ dàng mở rộng dung lượng lưu trữ khi khối lượng dữ liệu của dự án tăng lên.
- **Chi phí hiệu quả (Cost-effectiveness):** Là một giải pháp mã nguồn mở, MinIO giúp tôi tiết kiệm chi phí bản quyền so với các dịch vụ lưu trữ đối tượng thương mại, trong khi vẫn đảm bảo được hiệu suất và độ tin cậy.
- **Tính linh hoạt (Flexibility):** Việc tương thích với API S3 giúp tôi dễ dàng tích hợp MinIO với nhiều công cụ và dịch vụ khác trong hệ sinh thái dữ liệu lớn mà không gặp nhiều trở ngại. Tôi có thể triển khai MinIO trên hạ tầng tại chỗ (on-premises) hoặc trên bất kỳ nền tảng đám mây nào, mang lại sự chủ động trong việc quản lý hạ tầng.



Hình 2.5: Kiến trúc phân tán MinIO (Nguồn: MinIO Documentation)

MinIO phân tán dữ liệu dựa trên cơ chế **erasure coding** (mã hóa xóa). Như minh họa trong Hình 2.5, khi một đối tượng (Object 1, Object 2) được ghi vào cụm MinIO, nó sẽ được chia thành các *K data shards* và *M parity shards* được sinh ra từ *K* mảnh bằng công thức toán học. Các mảnh dữ liệu

này sau đó được phân phối đều trên nhiều ổ đĩa của các máy chủ khác nhau trong cụm. Và khi gặp sự cố mất M mảnh bất kì, chỉ cần còn K trong số K + M mảnh ban đầu, vẫn có thể phục hồi lại dữ liệu đã mất. Cơ chế này cung cấp khả năng chịu lỗi cao và độ bền dữ liệu vượt trội mà không cần nhân bản toàn bộ dữ liệu, giúp tiết kiệm không gian lưu trữ hiệu quả.

Câu lệnh khởi tạo một cụm MinIO thường có dạng:

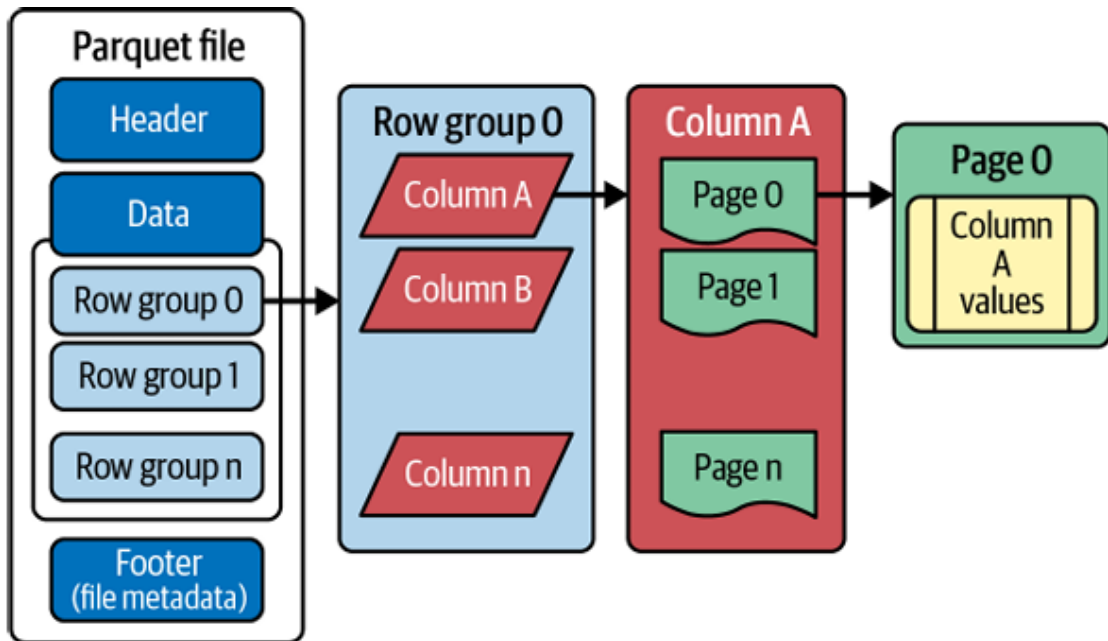
```
minio server http://host{1...N}/export{1...M}
```

Trong đó `host{1...N}` chỉ định N máy chủ trong cụm và `export{1...M}` chỉ định M thư mục (tương ứng với các ổ đĩa) trên mỗi máy chủ sẽ được sử dụng để lưu trữ dữ liệu.

b) **Open File Format - Parquet:**

Để lưu trữ dữ liệu thực tế trên MinIO, tôi đã chọn sử dụng định dạng tệp **Apache Parquet**. Parquet là một định dạng lưu trữ dạng cột (columnar storage format) mã nguồn mở, được tối ưu hóa cho các workload phân tích dữ liệu lớn. Việc tôi lựa chọn Parquet dựa trên các ưu điểm nổi bật của nó:

- **Hiệu suất truy vấn cao:** Lưu trữ theo cột cho phép các công cụ truy vấn chỉ đọc những cột cần thiết cho một câu lệnh cụ thể, giảm đáng kể lượng I/O và tăng tốc độ xử lý, điều này rất quan trọng cho các tác vụ phân tích trong dự án của tôi.
- **Khả năng nén dữ liệu hiệu quả:** Parquet hỗ trợ nhiều thuật toán nén mạnh mẽ (ví dụ: Snappy, Gzip, ZSTD), giúp tôi giảm dung lượng lưu trữ và chi phí liên quan.
- **Hỗ trợ schema evolution:** Parquet cho phép tôi thêm, bớt hoặc thay đổi kiểu dữ liệu của các cột theo thời gian mà không làm hỏng dữ liệu cũ, đảm bảo tính linh hoạt khi cấu trúc dữ liệu của dự án có sự thay đổi.
- **Khả năng phân tách và xử lý song song (Splittable):** Các tệp Parquet có thể được chia thành nhiều phần nhỏ để xử lý song song bởi các framework như Spark, Trino, nâng cao hiệu suất xử lý dữ liệu lớn trong hệ thống của tôi.



Hình 2.6: Kiến trúc file parquet

Toàn bộ **Tệp Parquet** bao gồm *Header*, phần *Data* được chia thành nhiều **Nhóm Hàng** (ví dụ: Row group 0, Row group 1, ..., Row group n), và kết thúc bằng *Footer* chứa siêu dữ liệu của tệp. Bên trong mỗi **Nhóm Hàng** (ví dụ: Row group 0), dữ liệu được tổ chức theo cột, nơi mỗi cột (ví dụ: Column A, Column B, ..., Column n) thực chất là một **Khối Cột** (Column Chunk) chứa tất cả giá trị của cột đó cho các hàng thuộc nhóm hàng này. Tiếp tục đi sâu vào một **Khối Cột** cụ thể, nó được chia nhỏ hơn thành các **Trang** (ví dụ: Page 0, Page 1, ..., Page n). Cuối cùng, mỗi **Trang** chứa các giá trị dữ liệu thực tế đã được mã hóa và nén cho một phần của cột đó.

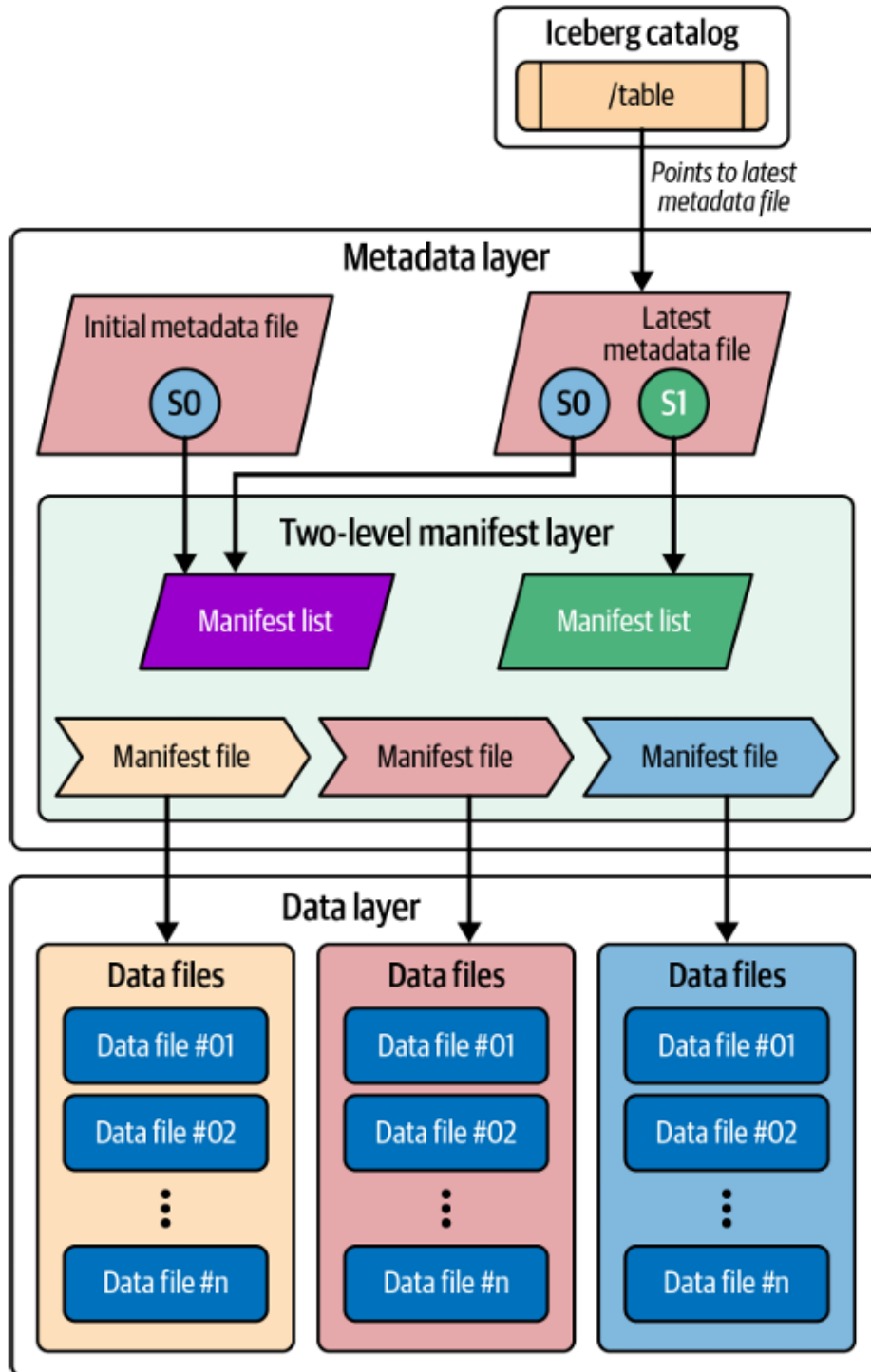
c) Open Table Format - Iceberg:

Đây là lớp trừu tượng quan trọng nằm phía trên Open File Format, mang lại các tính năng quản lý dữ liệu giống như trong một data warehouse truyền thống ngay trên data lake. Cho dự án này, tôi đã lựa chọn **Apache Iceberg** làm định dạng bảng mở. Iceberg cung cấp một lớp metadata mạnh mẽ để quản lý các tệp dữ liệu (ví dụ: các tệp Parquet) và mang lại nhiều lợi ích then chốt cho Lakehouse của tôi:

- **ACID Transactions:** Iceberg hỗ trợ các giao dịch ACID (Atomicity, Consistency, Isolation, Durability) trên data lake, cho phép tôi thực hiện các thao tác ghi, cập nhật, xóa dữ liệu một cách an toàn và nhất quán.
- **Schema Evolution Toàn diện:** Iceberg hỗ trợ các thay đổi schema phức

tập (đổi tên cột, thay đổi thứ tự cột, thay đổi kiểu dữ liệu một cách an toàn) mà không cần viết lại toàn bộ dữ liệu của bảng, một tính năng cực kỳ hữu ích cho tôi trong quá trình phát triển dự án.

- **Time Travel và Versioning:** Iceberg hoạt động dựa trên cơ chế **snapshots**. Mỗi thay đổi đối với bảng (thêm, xóa, cập nhật dữ liệu) sẽ tạo ra một snapshot mới, trở đến trạng thái của dữ liệu tại thời điểm đó. Điều này cho phép tôi dễ dàng quay lại các phiên bản dữ liệu trước đó (time travel), kiểm tra lịch sử thay đổi của bảng, hoặc thực hiện các truy vấn nhất quán tại một thời điểm cụ thể (snapshot isolation).
- **Partition Evolution:** Iceberg cho phép tôi thay đổi chiến lược phân vùng (partitioning scheme) của bảng mà không cần viết lại dữ liệu cũ, giúp tối ưu hóa hiệu suất truy vấn khi yêu cầu thay đổi.
- **Tích hợp rộng rãi:** Iceberg được hỗ trợ bởi nhiều công cụ xử lý dữ liệu phổ biến như Spark, Trino, Flink, Hive, giúp tôi linh hoạt trong việc lựa chọn công cụ để tương tác với dữ liệu.



Hình 2.7: Kiến trúc định dạng bảng mở Iceberg

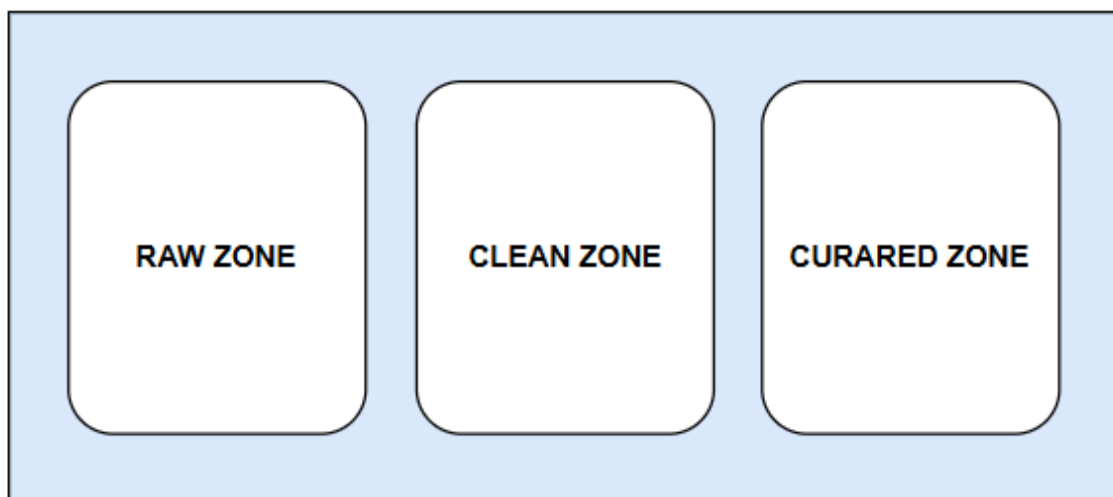
Bắt đầu từ **Iceberg catalog**, một con trỏ sẽ chỉ đến tệp siêu dữ liệu (metadata file) mới nhất cho một bảng cụ thể. Lớp **Metadata layer** (Lớp siêu dữ liệu) chứa các tệp siêu dữ liệu được phiên bản hóa (như *Initial metadata file* với snapshot S_0 và *Latest metadata file* với snapshot S_1 trong hình). Mỗi tệp siêu dữ liệu này đại diện cho một snapshot (trạng thái) của bảng tại một thời

điểm, bao gồm schema (lược đồ), thông tin phân vùng (partition spec), và tham chiếu đến một danh sách manifest. Tiếp theo là **Two-level manifest layer** (Lớp manifest hai cấp). Mỗi tệp siêu dữ liệu trỏ tới một **Manifest list**. **Manifest list** này lại chứa các mục, mỗi mục trỏ đến một tệp **Manifest file** riêng lẻ. Các tệp **Manifest file** này có nhiệm vụ theo dõi các tệp dữ liệu, lưu trữ đường dẫn đến tệp dữ liệu, định dạng tệp, thông tin phân vùng của tệp và các thống kê mức cột. Cuối cùng, **Data layer** (Lớp dữ liệu) là nơi chứa các **Data files** (ví dụ: Data file #01, Data file #02,...) – đây là các tệp dữ liệu thực tế (thường ở định dạng Parquet, ORC, hoặc Avro) chứa nội dung của bảng.

Sự kết hợp của MinIO cho lớp lưu trữ đối tượng, Parquet cho định dạng tệp hiệu quả, và Iceberg cho lớp quản lý bảng thông minh chính là nền tảng vững chắc cho vùng lưu trữ dữ liệu trong kiến trúc Lakehouse mà tôi đang xây dựng, đảm bảo tính mở, hiệu suất cao và khả năng quản lý dữ liệu tiên tiến.

2. Thiết kế phân vùng dữ liệu

Trong kiến trúc lưu trữ Lakehouse, việc phân vùng dữ liệu thành các khu vực riêng biệt là một thực hành quan trọng để quản lý vòng đời dữ liệu, tối ưu hóa hiệu suất và kiểm soát truy cập. Tôi đã quyết định chia dữ liệu thành ba vùng chính: vùng Raw, vùng Clean, và vùng Curated.



Hình 2.8: Các phân vùng lưu trữ

a) Vùng Raw

Đây là điểm nhập liệu đầu tiên vào Lakehouse. Tại vùng này, mục tiêu chính

là lưu trữ dữ liệu thô ngay sau quá trình thu thập, **giữ nguyên định dạng gốc nhất có thể** để đảm bảo không mất mát thông tin và phục vụ cho việc xử lý lại khi cần.

- **Định dạng lưu trữ:** Đối với dữ liệu bài báo, vốn ban đầu ở dạng các đối tượng JSON riêng lẻ, tôi sẽ lưu trữ một khối lượng các bài báo thành một tệp, dưới định dạng **JSON Lines (JSONL)**. Mỗi dòng trong tệp JSONL là một đối tượng JSON độc lập đại diện dữ liệu của một bài báo, giúp việc đọc và xử lý song song bằng các công cụ như Apache Spark trở nên dễ dàng và hiệu quả, đồng thời vẫn giữ được tính linh hoạt của dữ liệu JSON gốc.
- **Cấu trúc lưu trữ trên MinIO:** Để tổ chức dữ liệu một cách khoa học và dễ quản lý, các đối tượng trong MinIO sẽ được lưu trữ theo cấu trúc key phân cấp, dựa trên thời gian và chủ đề:

```
zone/yyyy/mm/dd/topic/source_file.jsonl
```

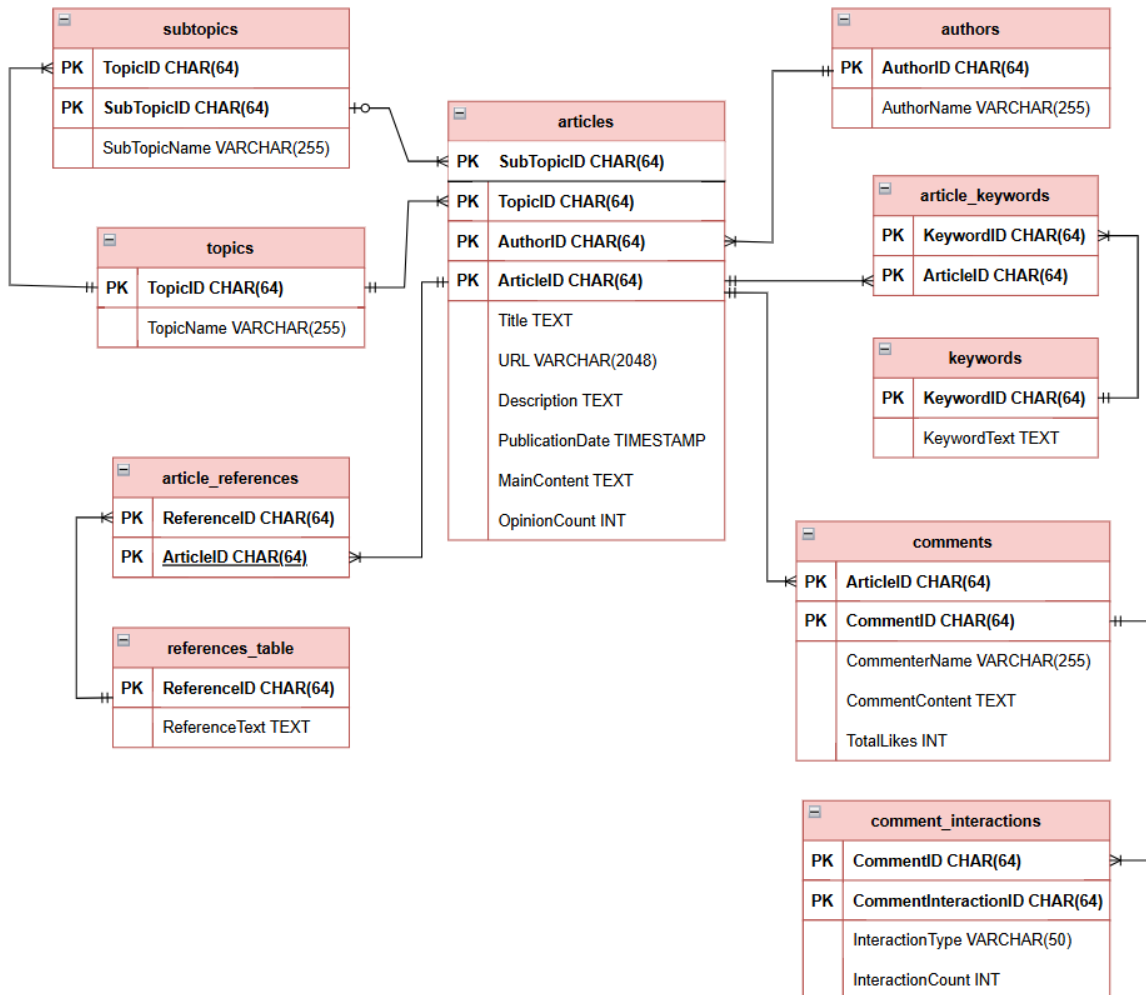
Cấu trúc này không chỉ giúp dễ dàng truy vấn dữ liệu theo ngày, tháng, năm hoặc chủ đề mà còn thuận tiện cho việc quản lý vòng đời dữ liệu và áp dụng các chính sách lưu trữ.

b) Vùng Clean

Dữ liệu từ vùng Raw sau đó sẽ được chuyển đến vùng Clean. Tại đây, dữ liệu trải qua các bước **xử lý, làm sạch, chuẩn hóa, và tổ chức lại cấu trúc**. Mục tiêu là tạo ra một nguồn dữ liệu đáng tin cậy, nhất quán và sẵn sàng cho các bước phân tích sâu hơn hoặc cho các ứng dụng khác.

- **Chuyển đổi định dạng và quản lý bảng:** Dữ liệu sẽ được chuyển đổi từ định dạng JSONL sang **Apache Parquet**, một định dạng tệp dạng cột (columnar) hiệu quả cao cho các tác vụ phân tích. Đồng thời, **Apache Iceberg** sẽ được sử dụng làm định dạng bảng mở (open table format) để quản lý các tệp Parquet này. Iceberg cung cấp các tính năng quan trọng như giao dịch ACID, schema evolution, time travel, và quản lý metadata hiệu quả trên data lake.
- **Mô hình dữ liệu:** Dữ liệu ở vùng Clean sẽ được tổ chức theo một mô hình dữ liệu có cấu trúc rõ ràng. Mô hình này có thể hướng tới sự **chuẩn hóa ở mức độ nhất định** nhằm đảm bảo tính toàn vẹn, nhất quán và

giảm thiểu dư thừa dữ liệu. Vùng này được thiết kế để chứa toàn bộ dữ liệu đã làm sạch từ vùng thô, sẵn sàng cho các AI Engineer hoặc Data Scientist truy cập, khám phá, trích xuất đặc trưng (feature engineering) và xây dựng mô hình máy học.



Hình 2.9: ERD của mô hình dữ liệu OLTP vùng clean

Bảng 2.1: Mô tả các trường của bảng authors

Tên trường	Kiểu dữ liệu	Ý nghĩa
AuthorID	STRING	Khóa chính nhân tạo (surrogate key) cho tác giả, tạo bằng hàm băm SHA256 từ AuthorName.

Tên trường	Kiểu dữ liệu	Ý nghĩa
AuthorName	STRING	Tên của tác giả, lấy từ trường `tac_gia` của dữ liệu thô và đã được chuẩn hóa (distinct).

Bảng authors: Lưu trữ thông tin về các tác giả duy nhất của bài báo. Dữ liệu được chuẩn hóa từ trường `tac\_gia`.

Bảng 2.2: Mô tả các trường của bảng topics

Tên trường	Kiểu dữ liệu	Ý nghĩa
TopicID	STRING	Khóa chính nhân tạo cho chủ đề, tạo bằng hàm băm SHA256 từ TopicName.
TopicName	STRING	Tên của chủ đề chính, lấy từ trường `topic` của dữ liệu thô và đã được chuẩn hóa (distinct).

Bảng topics: Lưu trữ các chủ đề chính duy nhất của bài báo. Dữ liệu được chuẩn hóa từ trường `topic`.

Bảng 2.3: Mô tả các trường của bảng subtopic

Tên trường	Kiểu dữ liệu	Ý nghĩa
SubTopicID	STRING	Khóa chính nhân tạo cho chủ đề phụ, tạo bằng hàm băm SHA256 từ SubTopicName và TopicID của chủ đề cha.
SubTopicName	STRING	Tên của chủ đề phụ, lấy từ trường `sub_topic` của dữ liệu thô.
TopicID	STRING	Khóa ngoại tham chiếu đến TopicID trong bảng `topics`, xác định chủ đề cha cho chủ đề phụ này.

Bảng subtopics: Lưu trữ các chủ đề phụ duy nhất của bài báo, liên kết với chủ đề chính. Dữ liệu được chuẩn hóa từ `sub\_topic` và `topic`.

Bảng 2.4: Mô tả các trường của bảng keywords

Tên trường	Kiểu dữ liệu	Ý nghĩa
KeywordID	STRING	Khóa chính nhân tạo cho từ khóa, tạo bằng hàm băm SHA256 từ KeywordText.
KeywordText	STRING	Nội dung của từ khóa. Dữ liệu được chuẩn hóa sau khi `explode` từ mảng `tu_khoa` của dữ liệu thô và lấy giá trị distinct.

Bảng keywords: Lưu trữ danh mục các từ khóa duy nhất được tìm thấy trong các bài báo.

Bảng 2.5: Mô tả các trường của bảng references_table

Tên trường	Kiểu dữ liệu	Ý nghĩa
ReferenceID	STRING	Khóa chính nhân tạo cho nguồn tham khảo, tạo bằng hàm băm SHA256 từ ReferenceText.
ReferenceText	STRING	Nội dung/tên của nguồn tham khảo. Dữ liệu được chuẩn hóa sau khi `explode` từ mảng `tham_khao` của dữ liệu thô và lấy giá trị distinct.

Bảng references_table: Lưu trữ danh mục các nguồn tham khảo duy nhất được trích dẫn trong các bài báo.

Bảng 2.6: Mô tả các trường của bảng articles

Tên trường	Kiểu dữ liệu	Ý nghĩa
ArticleID	STRING	Khóa chính nhân tạo cho bài báo, tạo bằng hàm băm SHA256 từ trường URL duy nhất của bài báo.
Title	STRING	Tiêu đề của bài báo. Lấy từ trường `title` của dữ liệu thô.

Tên trường	Kiểu dữ liệu	Ý nghĩa
URL	STRING	Đường dẫn URL gốc, duy nhất của bài báo. Lấy từ trường `url` của dữ liệu thô.
Description	STRING	Mô tả ngắn gọn của bài báo. Lấy từ trường `description` của dữ liệu thô.
PublicationDate	TIMESTAMP	Ngày giờ bài báo được xuất bản. Chuyển đổi từ trường `ngay_xuat_ban` của dữ liệu thô. Dùng để phân vùng bảng này.
MainContent	STRING	Nội dung chính của bài báo. Lấy từ trường `noi_dung_chinh` của dữ liệu thô.
OpinionCount	INT	Số lượng ý kiến ban đầu (nếu có) được báo cáo cho bài báo. Chuyển đổi từ trường `y_kien`, mặc định là 0 nếu giá trị không hợp lệ hoặc thiếu.
AuthorID	STRING	Khóa ngoại tham chiếu đến AuthorID trong bảng `authors`. Liên kết dựa trên `tac_gia` và `AuthorName`.
TopicID	STRING	Khóa ngoại tham chiếu đến TopicID trong bảng `topics`. Liên kết dựa trên `topic` và `TopicName`.
SubTopicID	STRING	Khóa ngoại tham chiếu đến SubTopicID trong bảng `subtopics`. Có thể là NULL nếu bài báo không có chủ đề phụ hoặc không tìm thấy mapping.

Bảng articles: Bảng trung tâm lưu trữ thông tin chi tiết đã được làm sạch và chuẩn hóa của từng bài báo.

Bảng 2.7: Mô tả các trường của bảng article_keywords

Tên trường	Kiểu dữ liệu	Ý nghĩa
ArticleID	STRING	Khóa ngoại tham chiếu đến ArticleID trong bảng `articles`.
KeywordID	STRING	Khóa ngoại tham chiếu đến KeywordID trong bảng `keywords`.

Bảng article_keywords: Bảng nối thể hiện mối quan hệ nhiều-nhiều giữa bài báo và từ khóa, đảm bảo các cặp ArticleID-KeywordID là duy nhất.

Bảng 2.8: Mô tả các trường của bảng article_references

Tên trường	Kiểu dữ liệu	Ý nghĩa
ArticleID	STRING	Khóa ngoại tham chiếu đến ArticleID trong bảng `articles`.
ReferenceID	STRING	Khóa ngoại tham chiếu đến ReferenceID trong bảng `references_table`.

Bảng article_references: Bảng nối thể hiện mối quan hệ nhiều-nhiều giữa bài báo và nguồn tham khảo, đảm bảo các cặp ArticleID-ReferenceID là duy nhất.

Bảng 2.9: Mô tả các trường của bảng comments

Tên trường	Kiểu dữ liệu	Ý nghĩa
CommentID	STRING	Khóa chính nhân tạo cho bình luận, tạo bằng hàm băm SHA256 từ RawRecordUID (ID tạm thời của bản ghi thô), CommenterName và CommentContent.
ArticleID	STRING	Khóa ngoại tham chiếu đến ArticleID trong bảng `articles` mà bình luận này thuộc về.

Tên trường	Kiểu dữ liệu	Ý nghĩa
CommenterName	STRING	Tên người bình luận. Lấy từ `nguoi_binh_luan` trong cấu trúc `top_binh_luan` của dữ liệu thô.
CommentContent	STRING	Nội dung của bình luận. Lấy từ `noi_dung` trong cấu trúc `top_binh_luan` của dữ liệu thô.
TotalLikes	INT	Tổng số lượt thích cho bình luận. Lấy từ `tong_luot_thich` trong `top_binh_luan`, mặc định là 0 nếu giá trị không hợp lệ hoặc thiếu.

Bảng comments: Lưu trữ các bình luận của người dùng về bài báo. Dữ liệu được trích xuất, làm sạch và chuẩn hóa từ trường `top\_binh\_luan`.

Bảng 2.10: Mô tả các trường của bảng comment_interactions

Tên trường	Kiểu dữ liệu	Ý nghĩa
CommentInteractionID	STRING	Khóa chính nhân tạo cho một loại tương tác cụ thể của bình luận, tạo bằng hàm băm SHA256 từ CommentID và InteractionType.
CommentID	STRING	Khóa ngoại tham chiếu đến CommentID trong bảng `comments`.
InteractionType	STRING	Loại tương tác (ví dụ: `Thích`, `Vui`). Lấy từ key trong map `chi_tiet_tuong_tac` của mỗi bình luận trong dữ liệu thô.
InteractionCount	INT	Số lượng của loại tương tác này. Lấy từ value trong map `chi_tiet_tuong_tac`, mặc định là 0 nếu giá trị không hợp lệ hoặc thiếu.

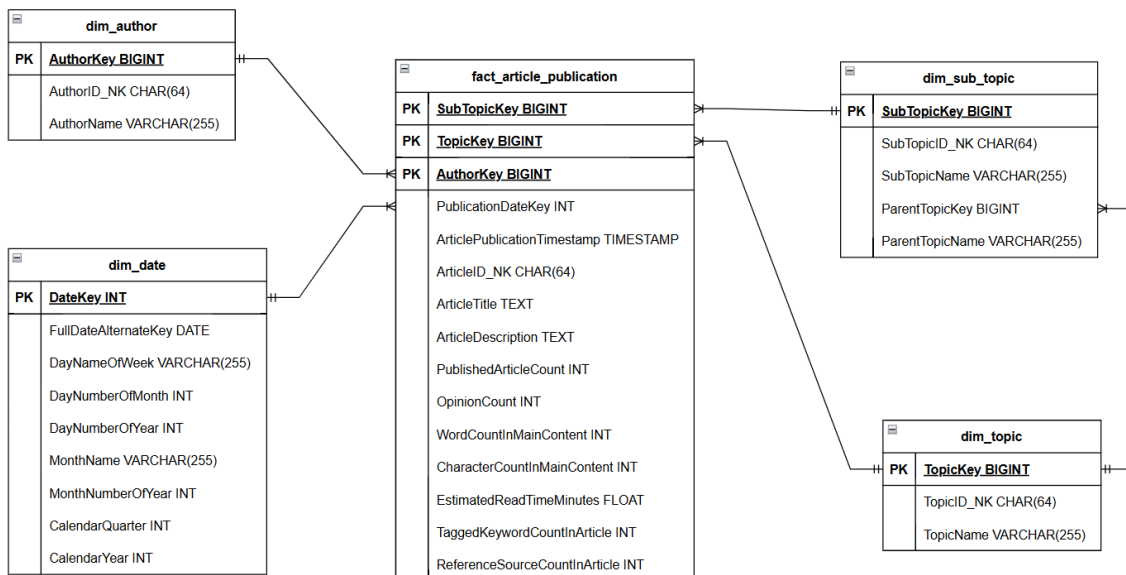
Bảng comment_interactions: Lưu trữ số lượng chi tiết cho từng loại tương tác của mỗi bình luận. Dữ liệu được `explode` từ map `chi\_tiet\_tuong\_tac`.

c) Vùng Curated

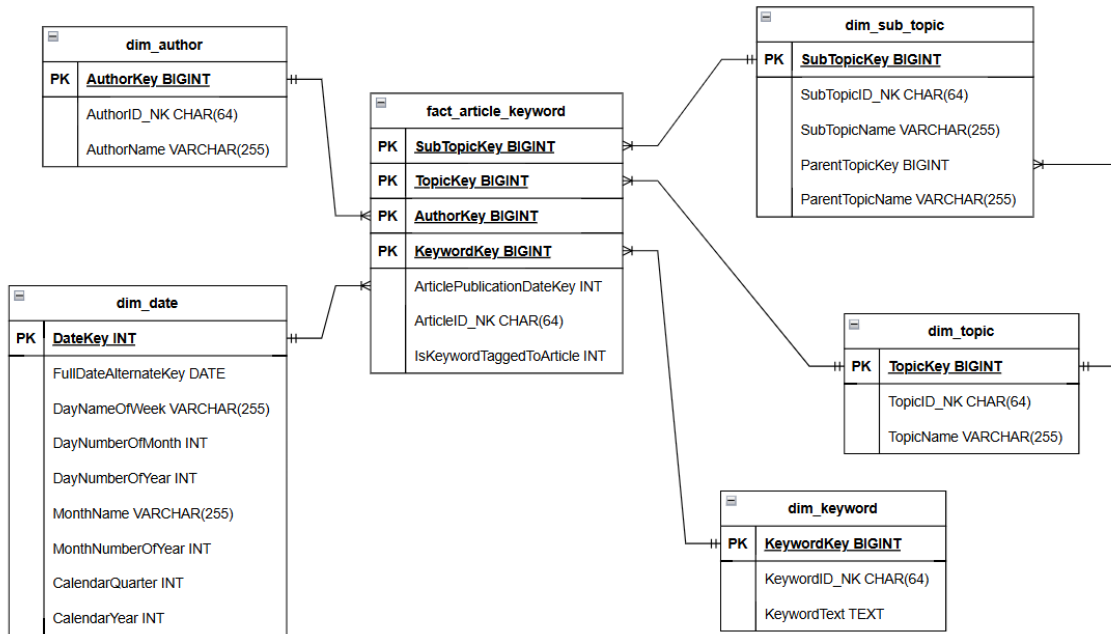
Vùng Curated là khu vực chứa dữ liệu đã được biến đổi, tổng hợp và tối ưu hóa để phục vụ trực tiếp cho các mục đích **phân tích nghiệp vụ (Business Intelligence - BI), báo cáo, và các ứng dụng phân tích dữ liệu chuyên sâu** của người dùng cuối.

- **Mô hình dữ liệu cho phân tích:** Dữ liệu tại đây được tổ chức theo các mô hình dữ liệu đa chiều (dimensional models) **galaxy schema**. Mô hình này được thiết kế để hỗ trợ các truy vấn phân tích (OLAP queries) một cách hiệu quả, cho phép người dùng dễ dàng "cắt lát"(slice), "khoan sâu"(drill-down) và tổng hợp dữ liệu theo nhiều chiều khác nhau.
- **Định dạng và quản lý:** Tương tự như vùng Clean, dữ liệu ở vùng Curated cũng được lưu trữ dưới định dạng Apache Parquet và quản lý bởi Apache Iceberg để tận dụng các lợi ích về hiệu suất và quản lý.

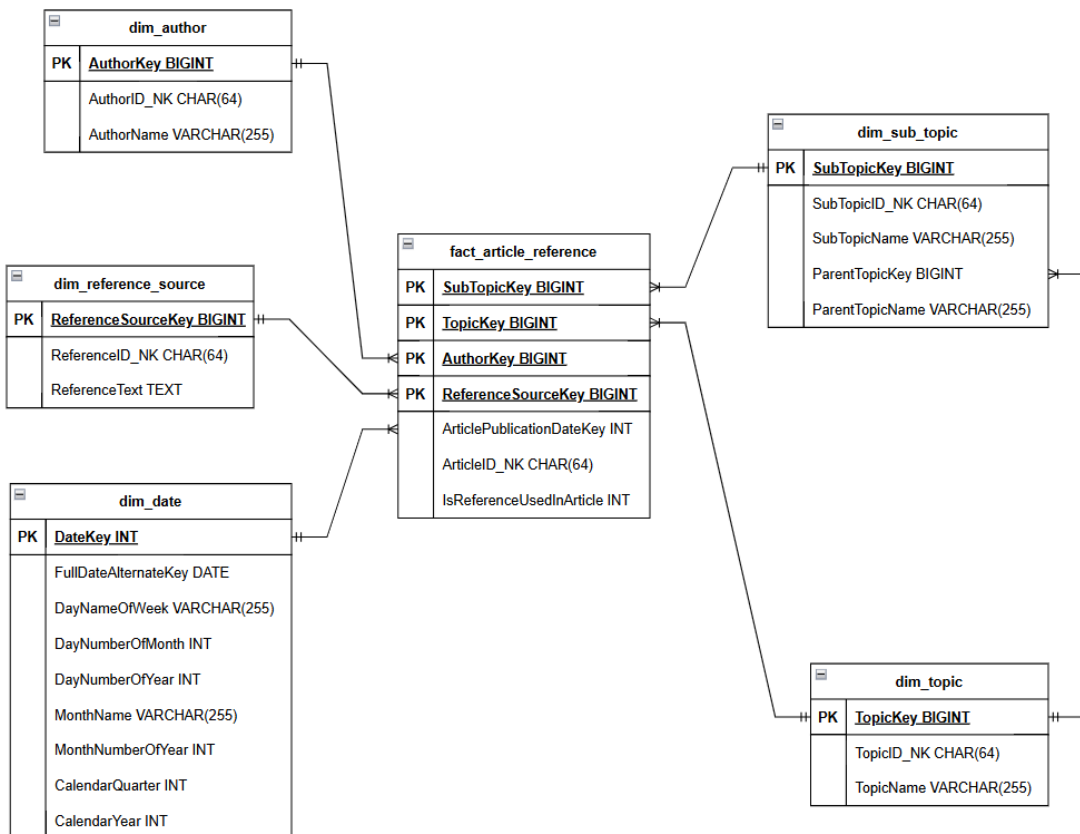
Mô hình vùng Curated được tôi thiết kế theo **Galaxy Schema**, với 5 fact chính. Mỗi fact liên kết với các dim giống như một **Star Schema**



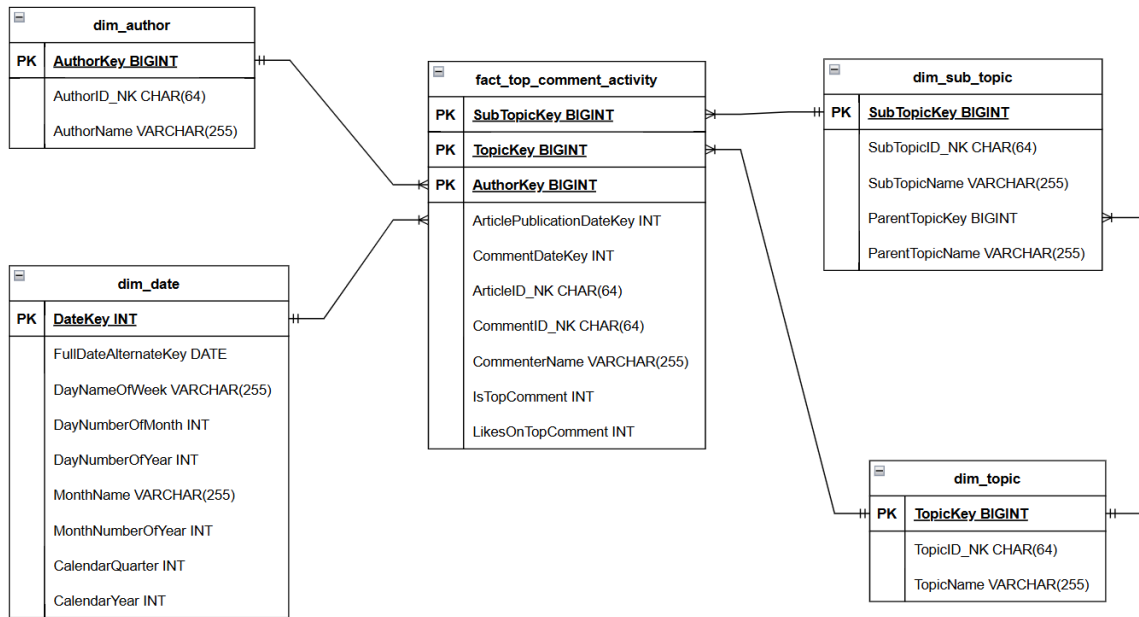
Hình 2.10: Mô hình cho fact article publication



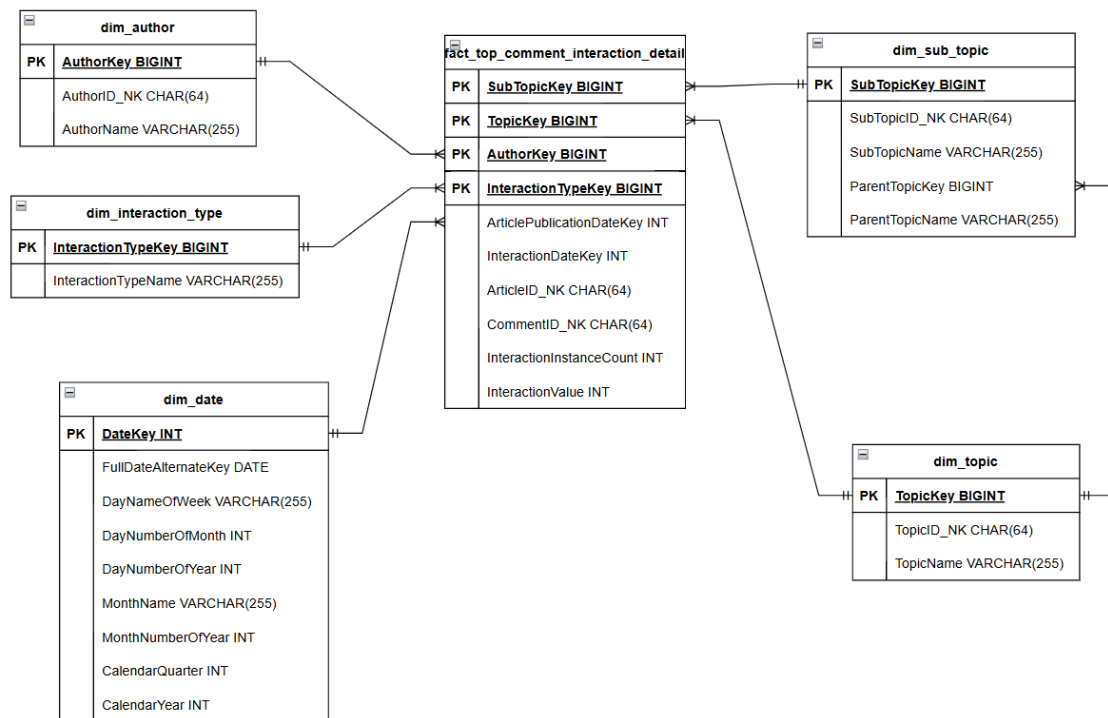
Hình 2.11: Mô hình cho fact article keyword



Hình 2.12: Mô hình cho fact article reference



Hình 2.13: Mô hình cho fact comment activity



Hình 2.14: Mô hình cho fact top comment interaction detail

Bảng 2.11: Mô tả các trường của bảng dim_date

Tên trường	Kiểu dữ liệu	Ý nghĩa
DateKey	INT	Khóa chính (PK) của bảng, khóa thay thế duy nhất cho mỗi ngày (ví dụ: 20250522).
FullDateAlternateKey	DATE	Ngày đầy đủ (ví dụ: `2025-05-22`). Khóa tự nhiên, dùng để join với dữ liệu nguồn.
DayNameOfWeek	STRING	Tên của ngày trong tuần (ví dụ: `Thứ Năm`).
DayNumberOfMonth	INT	Ngày trong tháng (1-31).
DayNumberOfYear	INT	Ngày trong năm (1-366).
MonthName	STRING	Tên tháng (ví dụ: `Tháng Năm`).
MonthNumberOfYear	INT	Tháng trong năm (1-12).
CalendarQuarter	INT	Quý trong năm (1, 2, 3, 4).
CalendarYear	INT	Năm theo lịch.

Bảng dim_date: Bảng chiều cung cấp các thuộc tính thời gian để phân tích dữ liệu theo ngày, tháng, quý, năm. Cần script riêng để tạo và điền dữ liệu.

Bảng 2.12: Mô tả các trường của bảng dim_author

Tên trường	Kiểu dữ liệu	Ý nghĩa
AuthorKey	BIGINT	Khóa chính (PK), khóa thay thế duy nhất cho mỗi tác giả. Được tạo bằng cách hash nhất quán từ AuthorID_NK (hoặc AuthorName) và chuyển đổi sang số nguyên BIGINT.
AuthorID_NK	STRING	Khóa tự nhiên (Natural Key) của tác giả, từ AuthorID của bảng `authors` (clean).
AuthorName	STRING	Tên của tác giả.

Bảng dim_author: Bảng chiều cung cấp thông tin về tác giả bài viết để phân tích hiệu suất, đóng góp. Nguồn: bảng `authors` (clean).

Bảng 2.13: Mô tả các trường của bảng `dim_topic`

Tên trường	Kiểu dữ liệu	Ý nghĩa
TopicKey	BIGINT	Khóa chính (PK), khóa thay thế duy nhất cho mỗi chủ đề. Được tạo bằng cách hash nhất quán từ TopicID_NK (hoặc TopicName) và chuyển đổi sang số nguyên BIGINT.
TopicID_NK	STRING	Khóa tự nhiên (Natural Key) của chủ đề, từ TopicID của bảng <code>`topics`</code> (clean).
TopicName	STRING	Tên của chủ đề chính.

Bảng `dim_topic`: Bảng chiều cung cấp thông tin về chủ đề chính của bài viết để phân tích xu hướng. Nguồn: bảng ``topics`` (clean).

Bảng 2.14: Mô tả các trường của bảng `dim_sub_topic`

Tên trường	Kiểu dữ liệu	Ý nghĩa
SubTopicKey	BIGINT	Khóa chính (PK), khóa thay thế duy nhất cho mỗi chủ đề phụ. Được tạo bằng cách hash nhất quán từ SubTopicID_NK (hoặc SubTopicName kết hợp ParentTopicKey) và chuyển đổi sang số nguyên BIGINT.
SubTopicID_NK	STRING	Khóa tự nhiên (Natural Key) của chủ đề phụ, từ SubTopicID của bảng <code>`subtopics`</code> (clean).
SubTopicName	STRING	Tên của chủ đề phụ.
ParentTopicKey	BIGINT	Khóa ngoại (FK) trỏ đến <code>dim_topic.TopicKey</code> . Giá trị -1 nếu không có chủ đề cha hoặc không áp dụng.
ParentTopicName	STRING	Tên của chủ đề chính (denormalized từ <code>dim_topic</code> để tiện truy vấn).

Bảng `dim_sub_topic`: Bảng chiều cung cấp thông tin chi tiết về chủ đề phụ.

Nguồn: `subtopics` (clean) và join với `dim\_topic` (curated).

Bảng 2.15: Mô tả các trường của bảng dim_keyword

Tên trường	Kiểu dữ liệu	Ý nghĩa
KeywordKey	BIGINT	Khóa chính (PK), khóa thay thế duy nhất cho mỗi từ khóa. Được tạo bằng cách hash nhất quán từ KeywordID_NK (hoặc KeywordText) và chuyển đổi sang số nguyên BIGINT.
KeywordID_NK	STRING	Khóa tự nhiên (Natural Key) của từ khóa, từ KeywordID của bảng `keywords` (clean).
KeywordText	STRING	Nội dung của từ khóa.

Bảng dim_keyword: Bảng chiều cung cấp danh sách các từ khóa đã được gán cho bài viết. Nguồn: bảng `keywords` (clean).

Bảng 2.16: Mô tả các trường của bảng dim_reference_source

Tên trường	Kiểu dữ liệu	Ý nghĩa
ReferenceSourceKey	BIGINT	Khóa chính (PK), khóa thay thế duy nhất cho mỗi nguồn tham khảo. Được tạo bằng cách hash nhất quán từ ReferenceID_NK (hoặc ReferenceText) và chuyển đổi sang số nguyên BIGINT.
ReferenceID_NK	STRING	Khóa tự nhiên (Natural Key) của nguồn tham khảo, từ ReferenceID của bảng `references_table` (clean).
ReferenceText	STRING	Tên/nội dung của nguồn tham khảo.

Bảng dim_reference_source: Bảng chiều cung cấp danh sách các nguồn tham khảo được trích dẫn. Nguồn: bảng `references\_table` (clean).

Bảng 2.17: Mô tả các trường của bảng `dim_interaction_type`

Tên trường	Kiểu dữ liệu	Ý nghĩa
InteractionTypeKey	BIGINT	Khóa chính (PK), khóa thay thế duy nhất cho mỗi loại tương tác. Được tạo bằng cách hash nhất quán từ InteractionTypeName và chuyển đổi sang số nguyên BIGINT.
InteractionTypeName	STRING	Tên của loại tương tác (ví dụ: `Thích`, `Vui`). Khóa tự nhiên, trích xuất từ giá trị duy nhất trong `comment_interactions` (clean).

Bảng `dim_interaction_type`: Bảng chiều phân loại các loại tương tác trên bình luận. Nguồn: giá trị duy nhất từ `comment\_interactions` (clean).

Bảng 2.18: Mô tả các trường của bảng `fact_article_publication`

Tên trường	Kiểu dữ liệu	Ý nghĩa
PublicationDateKey	INT	Thành phần của khóa tổ hợp định danh bản ghi sự kiện. Đồng thời là Khóa ngoại (FK) trỏ đến <code>dim_date.DateKey</code> (chỉ phần ngày). Giá trị -1 nếu không map được.
ArticlePublicationTimestamp	TIMESTAMP	Thời điểm đầy đủ bài viết được xuất bản. Degenerate Dimension/Measure. Dùng để phân vùng.

Tên trường	Kiểu dữ liệu	Ý nghĩa
AuthorKey	BIGINT	Thành phần của khóa tổ hợp định danh bản ghi sự kiện. Đồng thời là Khóa ngoại (FK) trỏ đến dim_author.AuthorKey. Giá trị -1 nếu không map được.
TopicKey	BIGINT	Thành phần của khóa tổ hợp định danh bản ghi sự kiện. Đồng thời là Khóa ngoại (FK) trỏ đến dim_topic.TopicKey. Giá trị -1 nếu không map được.
SubTopicKey	BIGINT	Thành phần của khóa tổ hợp định danh bản ghi sự kiện. Đồng thời là Khóa ngoại (FK) trỏ đến dim_sub_topic.SubTopicKey. Giá trị -1 nếu không map được hoặc không áp dụng.
ArticleID_NK	STRING	Thành phần của khóa tổ hợp định danh bản ghi sự kiện. Là ID gốc của bài viết từ `clean` (Degenerate Dimension).
ArticleTitle	STRING	Tiêu đề bài viết. Degenerate Dimension.
ArticleDescription	STRING	Mô tả ngắn của bài viết. Degenerate Dimension.
PublishedArticleCount	INT	Measure: Số lượng bài viết được xuất bản (luôn là 1).
OpinionCount	INT	Measure: Tổng số ý kiến/bình luận ban đầu (từ y_kien gốc).
WordCountInMainContent	INT	Measure: Số từ trong nội dung chính.

Tên trường	Kiểu dữ liệu	Ý nghĩa
CharacterCountInMainContent	INT	Measure: Số ký tự trong nội dung chính.
EstimatedReadTimeMinutes	FLOAT	Measure: Thời gian đọc ước tính (phút).
TaggedKeywordCountInArticle	INT	Measure: Số lượng từ khóa riêng biệt được gán cho bài viết.
ReferenceSourceCountInArticle	INT	Measure: Số lượng nguồn tham khảo riêng biệt được trích dẫn.

Bảng fact_article_publication: Bảng sự kiện trung tâm ghi nhận các số liệu và thuộc tính cơ bản mỗi khi một bài viết được xuất bản.

Bảng 2.19: Mô tả các trường của bảng fact_article_keyword

Tên trường	Kiểu dữ liệu	Ý nghĩa
ArticlePublicationDateKey	INT	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) trỏ đến dim_date.DateKey (ngày xuất bản bài viết). Giá trị -1 nếu không map được.
ArticleID_NK	STRING	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) đến ArticleID_NK của bài viết trong bảng fact_article_publication hoặc articles (clean).
KeywordKey	BIGINT	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) trỏ đến dim_keyword.KeywordKey. Giá trị -1 nếu không map được.

Tên trường	Kiểu dữ liệu	Ý nghĩa
AuthorKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_author.AuthorKey của tác giả bài viết. Giá trị -1 nếu không map được.
TopicKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_topic.TopicKey của chủ đề bài viết. Giá trị -1 nếu không map được.
SubTopicKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_sub_topic.SubTopicKey của chủ đề phụ bài viết. Giá trị -1 nếu không map được hoặc không áp dụng.
IsKeywordTaggedToArticle	INT	Measure: Cờ đánh dấu sự tồn tại của mỗi quan hệ (luôn là 1).

Bảng fact_article_keyword: Bảng sự kiện ghi nhận mỗi quan hệ giữa mỗi bài viết và từng từ khóa đã được gán (tag) cho nó.

Bảng 2.20: Mô tả các trường của bảng fact_article_reference

Tên trường	Kiểu dữ liệu	Ý nghĩa
ArticlePublicationDateKey	INT	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) trỏ đến dim_date.DateKey (ngày xuất bản bài viết). Giá trị -1 nếu không map được.
ArticleID_NK	STRING	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) đến ArticleID_NK của bài viết.

Tên trường	Kiểu dữ liệu	Ý nghĩa
ReferenceSourceKey	BIGINT	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) trỏ đến dim_reference_source.ReferenceSourceKey. Giá trị -1 nếu không map được.
AuthorKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_author.AuthorKey. Giá trị -1 nếu không map được.
TopicKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_topic.TopicKey. Giá trị -1 nếu không map được.
SubTopicKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_sub_topic.SubTopicKey. Giá trị -1 nếu không map được hoặc không áp dụng.
IsReferenceUsedInArticle	INT	Measure: Cờ đánh dấu sự tồn tại của mỗi quan hệ (luôn là 1).

Bảng fact_article_reference: Bảng sự kiện ghi nhận mối quan hệ giữa mỗi bài viết và từng nguồn tham khảo được trích dẫn.

Bảng 2.21: Mô tả các trường của bảng fact_top_comment_activity

Tên trường	Kiểu dữ liệu	Ý nghĩa
ArticlePublicationDateKey	INT	Khóa ngoại (FK) trỏ đến dim_date.DateKey (ngày xuất bản bài viết). Giá trị -1 nếu không map được.

Tên trường	Kiểu dữ liệu	Ý nghĩa
CommentDateKey	INT	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) trỏ đến dim_date.DateKey (trong ETL này, nó bằng ArticlePublicationDateKey). Giá trị -1 nếu không map được.
ArticleID_NK	STRING	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) đến ArticleID_NK của bài viết.
CommentID_NK	STRING	Thành phần của khóa tổ hợp định danh bản ghi. Là ID gốc của bình luận từ `clean`.
AuthorKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_author.AuthorKey. Giá trị -1 nếu không map được.
TopicKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_topic.TopicKey. Giá trị -1 nếu không map được.
SubTopicKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_sub_topic.SubTopicKey. Giá trị -1 nếu không map được hoặc không áp dụng.
CommenterName	STRING	Tên người bình luận. Degenerate Dimension.
IsTopComment	INT	Measure: Cờ đánh dấu đây là một bình luận `top` được ghi nhận (luôn là 1).
LikesOnTopComment	INT	Measure: Tổng số lượt thích mà bình luận `top` này nhận được.

Bảng fact_top_comment_activity: Bảng sự kiện ghi nhận các số liệu chính liên quan đến các bình luận `top` được crawl.

Bảng 2.22: Mô tả các trường của bảng fact_top_comment_interaction_detail

Tên trường	Kiểu dữ liệu	Ý nghĩa
ArticlePublicationDateKey	INT	Khóa ngoại (FK) trỏ đến dim_date.DateKey (ngày xuất bản bài viết). Giá trị -1 nếu không map được.
InteractionDateKey	INT	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) trỏ đến dim_date.DateKey (trong ETL này, nó bằng ArticlePublicationDateKey). Giá trị -1 nếu không map được.
ArticleID_NK	STRING	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) đến ArticleID_NK của bài viết.
CommentID_NK	STRING	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) đến CommentID_NK của bình luận.
InteractionTypeKey	BIGINT	Thành phần của khóa tổ hợp định danh bản ghi. Đồng thời là Khóa ngoại (FK) trỏ đến dim_interaction_type.InteractionTypeKey. Giá trị -1 nếu không map được.
AuthorKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_author.AuthorKey. Giá trị -1 nếu không map được.
TopicKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_topic.TopicKey. Giá trị -1 nếu không map được.
SubTopicKey	BIGINT	Khóa ngoại (FK) theo ngữ cảnh, trỏ đến dim_sub_topic.SubTopicKey. Giá trị -1 nếu không map được hoặc không áp dụng.

Tên trường	Kiểu dữ liệu	Ý nghĩa
InteractionInstanceCount	INT	Measure: Số lần loại tương tác này xuất hiện trên bình luận (luôn là 1).
InteractionValue	INT	Measure: Số lượt của loại tương tác đó (ví dụ: số lượt `Thích`).

Bảng fact_top_comment_interaction_detail: Bảng sự kiện ghi nhận chi tiết từng loại tương tác (Thích, Vui,...) trên các bình luận `top`.

Ngoài ba vùng chính kể trên, khi có các nhu cầu phân tích đặc thù hoặc cần cung cấp dữ liệu theo một góc nhìn nghiệp vụ cụ thể cho người dùng cuối, một **lớp ngữ nghĩa (Semantic Layer)** có thể được phát triển. Lớp này thường bao gồm các views được xây dựng sẵn, tổng hợp hoặc ánh xạ dữ liệu từ vùng Curated hoặc Clean, giúp đơn giản hóa việc truy cập và hiểu dữ liệu. Tuy nhiên, trong phạm vi và giai đoạn hiện tại của dự án, lớp ngữ nghĩa này chưa được triển khai. Sự phân chia này không chỉ giúp tổ chức dữ liệu một cách logic mà còn tạo điều kiện cho việc áp dụng các chiến lược quản trị dữ liệu, bảo mật và tối ưu hóa chi phí lưu trữ một cách hiệu quả.

2.2.4 Data procesing

Xử lý dữ liệu là một cầu phần then chốt trong kiến trúc lakehouse tin tức, đóng vai trò ETL (Extract, Transform, Load) dữ liệu giữa các vùng khác nhau. Để đảm nhiệm vai trò này, dự án đã lựa chọn **Apache Spark [2]** làm công nghệ xử lý chính, đặc biệt khi sử dụng thông qua API Python là **PySpark**.

1. Lý do lựa chọn Apache Spark (PySpark) cho xử lý dữ liệu

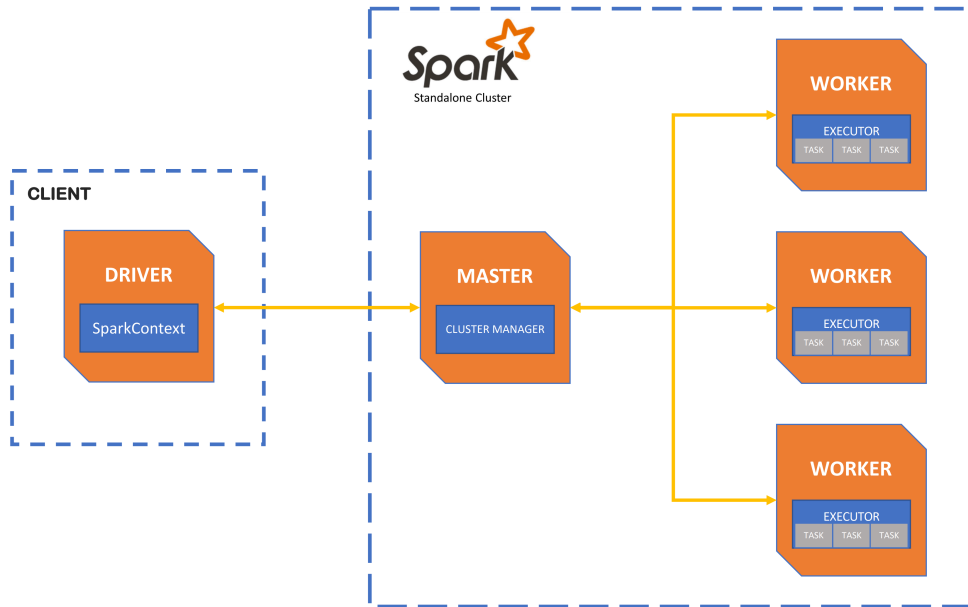
Apache Spark, với giao diện PySpark, được chọn làm công cụ xử lý chính trong dự án lakehouse tin tức này vì những khả năng vượt trội sau:

- **Khả năng làm việc hiệu quả với các định dạng dữ liệu đa dạng:** PySpark cung cấp các API trực quan và mạnh mẽ để đọc và ghi dữ liệu từ nhiều nguồn và định dạng khác nhau. Spark có khả năng làm việc tốt với JSONL, cũng như định dạng file parquet.

- **Tương thích và khai thác hiệu quả bảng Apache Iceberg:** Apache Iceberg là định dạng bảng mở được chọn cho lakehouse này để quản lý siêu dữ liệu và đảm bảo tính nhất quán. PySpark tích hợp mượt mà với Iceberg thông qua Spark SQL và DataFrame API. Nó cho phép thực hiện các thao tác `CREATE TABLE`, `SELECT`, `INSERT OVERWRITE`, `MERGE INTO` và các tính năng nâng cao như *schema evolution*, *time travel* trực tiếp trên các bảng Iceberg. Điều này giúp quản lý dữ liệu tin tức một cách linh hoạt và đáng tin cậy.
- **Năng lực xử lý nhanh và khối lượng dữ liệu lớn:** Spark được thiết kế cho tính toán phân tán, có khả năng xử lý song song các tác vụ trên một cụm máy tính. Kiến trúc của Spark, bao gồm việc tối ưu hóa truy vấn thông qua Catalyst Optimizer và thực thi trong bộ nhớ (in-memory processing) khi có thể, cho phép nó xử lý hàng terabytes dữ liệu một cách hiệu quả và nhanh chóng. Điều này cực kỳ quan trọng đối với một hệ thống tin tức, nơi dữ liệu mới được cập nhật liên tục với khối lượng lớn.
- **Sức mạnh và sự linh hoạt của PySpark:**
 - **API Python phong phú:** PySpark cung cấp một API Python toàn diện, bao gồm DataFrame API rất giống với Pandas, giúp các nhà khoa học dữ liệu và kỹ sư dữ liệu quen thuộc với Python có thể nhanh chóng phát triển các ứng dụng xử lý dữ liệu phức tạp.
 - **Khả năng mở rộng logic nghiệp vụ:** Cho phép định nghĩa các hàm tùy chỉnh (UDFs - User-Defined Functions) bằng Python để áp dụng các logic nghiệp vụ phức tạp, bao gồm cả việc tích hợp các thư viện Python khác vào quy trình xử lý phân tán của Spark.
 - **Hệ sinh thái rộng lớn:** Spark có một cộng đồng người dùng và nhà phát triển lớn mạnh, cùng với một hệ sinh thái các thư viện và công cụ hỗ trợ phong phú, giúp dễ dàng tìm kiếm giải pháp và tích hợp với các công nghệ khác.

Những yếu tố này làm cho PySpark trở thành công cụ chính giúp chuyển đổi dữ liệu thô thành dữ liệu có giá trị, tuân thủ theo data model đã thiết kế trong lakehouse tin tức.

2. Thiết lập và cơ chế hoạt động của cụm Spark Standalone



Hình 2.15: Kiến trúc spark cluster ở Standalone mode

Để triển khai Spark cho dự án, tôi sẽ thiết lập một cụm ở chế độ **standalone**. Đây là chế độ triển khai đơn giản nhất của Spark, không yêu cầu các trình quản lý tài nguyên bên ngoài như YARN hay Mesos. Cụm standalone bao gồm các thành phần chính với vai trò như sau:

- **Master Node:** Nút Master hoạt động như một người quản lý và điều phối tài nguyên cho toàn bộ cụm Spark. Khi một ứng dụng PySpark (ví dụ, một script Python sử dụng `SparkSession`) được gửi đến cụm (thông qua lệnh `spark-submit`), Master Node sẽ tiếp nhận yêu cầu này. Nó chịu trách nhiệm thương lượng tài nguyên với các Worker Node và phân công các *task* (tác vụ con của một job) cho các *Executor* trên các Worker Node để thực thi. Master Node cũng theo dõi trạng thái của các Worker Node và các ứng dụng đang chạy.
- **Worker Nodes:** Các Worker Node là nơi các tác vụ tính toán của ứng dụng Spark được thực thi. Mỗi Worker Node chạy một hoặc nhiều tiến trình *Executor*. Executor là các tiến trình được Master Node khởi tạo theo yêu cầu của *Driver Program* (chương trình chính của ứng dụng Spark, thường chạy trên client hoặc trên một trong các node của cụm). Mỗi Executor sẽ nhận các task từ Driver, thực thi chúng trên dữ liệu của mình (một phần của RDD/DataFrame), và trả kết quả về cho Driver hoặc ghi xuống hệ thống lưu

trữ. Executors cũng chịu trách nhiệm cache dữ liệu trong bộ nhớ để tăng tốc các truy cập sau này.

2.2.5 Data Consumption and Delivery

Khi đã có một lakehouse được xây dựng vững chắc để lưu trữ và quản lý dữ liệu tin tức đã qua xử lý, bước tiếp theo và vô cùng quan trọng là khai thác hiệu quả khối lượng dữ liệu đó. Dữ liệu trong lakehouse cần được truy cập, phân tích và phân phối đến các đối tượng người dùng khác nhau, từ các nhà phân tích dữ liệu, nhà khoa học dữ liệu đến các ứng dụng nghiệp vụ và hệ thống báo cáo.

1. Query Engine

Để cho phép người dùng và các ứng dụng tương tác và truy vấn dữ liệu một cách linh hoạt từ lakehouse (cụ thể là các bảng Apache Iceberg), dự án này sử dụng **Trino** (trước đây là PrestoSQL) [12] làm công cụ truy vấn chính.

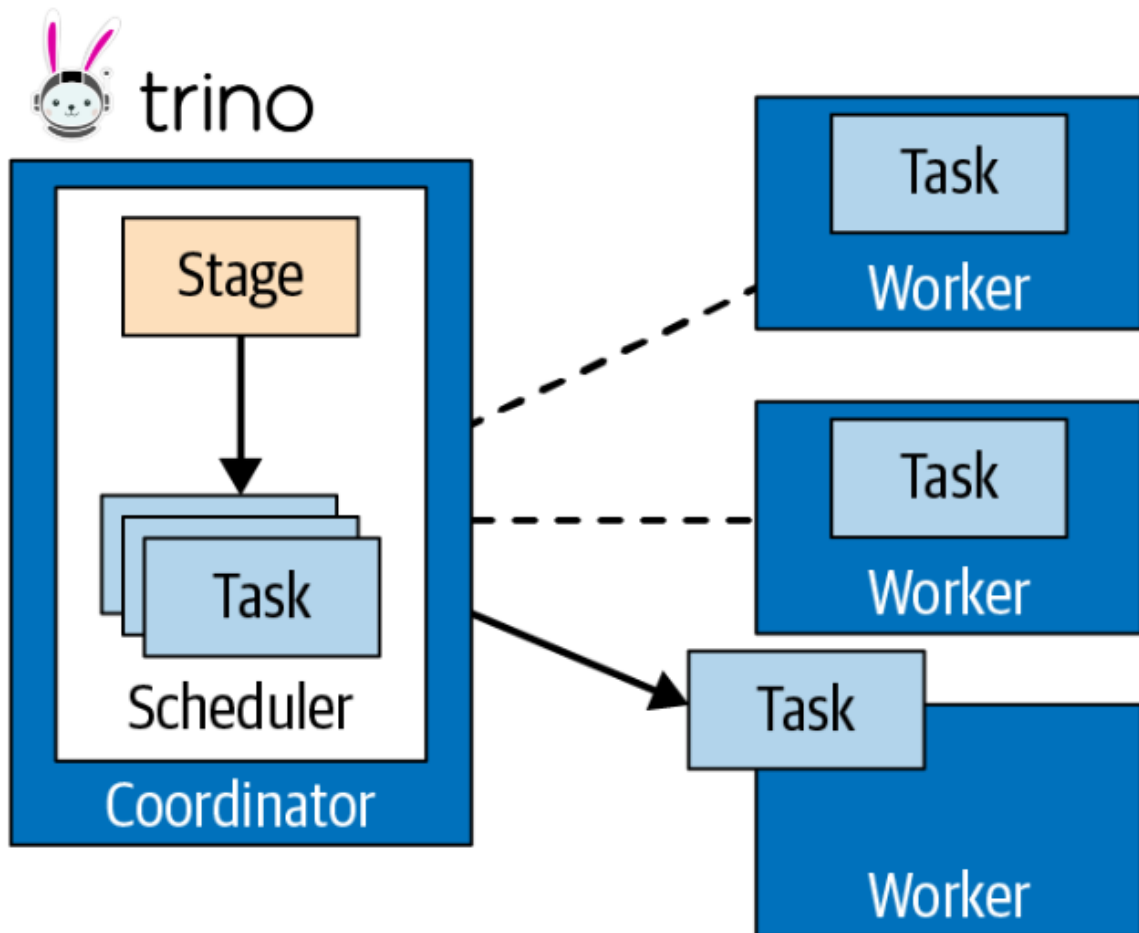
Trino là một hệ thống truy vấn SQL phân tán, mã nguồn mở, được thiết kế để chạy các truy vấn phân tích tương tác nhanh chóng trên các nguồn dữ liệu lớn. Một trong những đặc điểm kiến trúc quan trọng của Trino, tương tự như Apache Spark, là **sự tách biệt giữa tính toán (compute) và lưu trữ (storage)**. Điều này có nghĩa là Trino không sở hữu dữ liệu mà nó truy vấn; thay vào đó, nó kết nối tới các hệ thống lưu trữ dữ liệu hiện có thông qua các connector.

Những điểm mạnh nổi bật của Trino bao gồm:

- **Truy vấn đa nguồn:** Trino có khả năng thực thi một câu lệnh SQL duy nhất để truy vấn dữ liệu từ nhiều nguồn khác nhau cùng một lúc. Điều này được thực hiện thông qua hệ thống connector phong phú của Trino (Iceberg, Hive, MySQL, PostgreSQL, Kafka, Elasticsearch, v.v.).
- **Tối ưu hóa và tạo kế hoạch truy vấn:** Khi một câu lệnh SQL được gửi đến Trino, Coordinator sẽ thực hiện phân tích (parse), tối ưu hóa và tạo ra một kế hoạch thực thi phân tán (distributed query plan). Kế hoạch này bao gồm các giai đoạn (stages) và tác vụ (tasks) sẽ được phân phối cho các Worker Node. Quá trình tối ưu hóa này giúp giảm thiểu lượng dữ liệu cần đọc và xử lý, tăng tốc độ thực thi truy vấn.
- **Hiệu suất cao cho truy vấn tương tác:** Trino được tối ưu cho các truy vấn có độ trễ thấp, rất phù hợp cho các công cụ Business Intelligence (BI),

dashboarding và các nhu cầu phân tích dữ liệu ad-hoc. Kiến trúc MPP (Massively Parallel Processing) của nó cho phép xử lý song song hiệu quả.

- **Hỗ trợ chuẩn SQL ANSI:** Điều này giúp người dùng đã quen với SQL có thể dễ dàng làm việc với Trino mà không cần học một ngôn ngữ truy vấn mới.
- **Khả năng mở rộng:** Cả Coordinator và Worker trong Trino đều có thể được mở rộng để đáp ứng nhu cầu truy vấn tăng cao.



Hình 2.16: Kiến trúc phân tán Trino

Một cụm Trino hoạt động theo kiến trúc phân tán, bao gồm hai loại node chính:

- **Coordinator (Nút điều phối):**
 - **Vai trò tổng quan:** Coordinator là "bộ não" của cụm Trino. Nó chịu trách nhiệm tiếp nhận các câu lệnh SQL từ client (ví dụ: Trino CLI, JDBC/ODBC driver, các công cụ BI). Sau khi nhận được câu lệnh,

Coordinator sẽ:

1. **Phân tích (Parse) câu lệnh SQL:** Kiểm tra cú pháp và ngữ nghĩa.
2. **Phân tích (Analyze) truy vấn:** Xác định các bảng, cột liên quan và yêu cầu siêu dữ liệu từ các connector tương ứng.
3. **Lập kế hoạch truy vấn (Query Planning):** Tạo ra một kế hoạch thực thi logic và vật lý tối ưu. Kế hoạch này được chia thành các giai đoạn (stages), và mỗi giai đoạn lại được chia thành các tác vụ (tasks) song song.
4. **Điều phối thực thi:** Phân công các task cho các Worker Node khả dụng.
5. **Theo dõi tiến trình:** Giám sát trạng thái của các Worker và các task đang chạy.
6. **Tổng hợp kết quả:** Thu thập kết quả từ các Worker Node và trả về cho client.

– Thông thường, chỉ có một Coordinator trong một cụm Trino.

• **Worker:**

– **Vai trò tổng quan:** Các Worker Node là nơi thực thi các tác vụ tính toán thực tế. Chúng nhận các task được giao bởi Coordinator. Mỗi Worker sẽ:

1. **Thực thi tasks:** Xử lý một phần dữ liệu được gán cho task đó. Điều này có thể bao gồm đọc dữ liệu từ nguồn (thông qua connector), lọc, join, tổng hợp, v.v.
2. **Trao đổi dữ liệu trung gian:** Các Worker có thể trao đổi dữ liệu với nhau trong quá trình thực thi các giai đoạn của một truy vấn phức tạp (ví dụ: trong một phép join lớn).
3. **Gửi kết quả:** Gửi kết quả của các task đã hoàn thành về cho Coordinator hoặc cho các Worker khác (nếu là dữ liệu trung gian).

– Một cụm Trino có thể có nhiều Worker Node, và số lượng Worker có thể được tăng hoặc giảm để điều chỉnh năng lực xử lý của cụm.

Khi một truy vấn được thực thi, client gửi yêu cầu đến Coordinator. Coordinator lập kế hoạch và giao việc cho các Worker. Các Worker đọc dữ liệu trực tiếp từ các data source (các file Parquet của bảng Iceberg trên MinIO), xử lý dữ liệu song song, và gửi kết quả về Coordinator để tổng hợp lại trước khi trả về cho

client.

2. Business Intelligence Engine

Trong bối cảnh dữ liệu ngày càng trở thành tài sản quý giá, việc khai thác hiệu quả nguồn dữ liệu để phục vụ cho việc ra quyết định và xây dựng chiến lược là một yêu cầu then chốt. Với mục đích khám phá sâu, phân tích đa chiều và trực quan hóa dữ liệu một cách linh hoạt, việc lựa chọn và triển khai một công cụ Business Intelligence (BI) là nhu cầu hợp lý và thiết yếu. Các công cụ BI không chỉ giúp tạo ra các báo cáo (reports), dashboards và panels trực quan, mà còn hỗ trợ người dùng ở mọi cấp độ có thể tương tác, đặt câu hỏi và tự tìm kiếm những giá trị từ dữ liệu thô.

Trong khuôn khổ của dự án này, tôi đã lựa chọn **Metabase** [7] làm công cụ BI trung tâm. Metabase là một nền tảng Business Intelligence mã nguồn mở, nổi bật với giao diện thân thiện và khả năng giúp người dùng dễ dàng tiếp cận và phân tích dữ liệu mà không yêu cầu kiến thức kỹ thuật sâu rộng.



Hình 2.17: Công cụ Metabase

Metabase được chọn lựa dựa trên nhiều ưu điểm nổi bật, phù hợp với yêu cầu của dự án:

- **Dễ sử dụng và trực quan:** Metabase cho phép người dùng, kể cả những người không chuyên về kỹ thuật, có thể dễ dàng đặt câu hỏi (questions), tạo bộ lọc, và xây dựng các biểu đồ trực quan hóa dữ liệu thông qua giao diện kéo thả hoặc trình soạn thảo câu hỏi thân thiện.
- **Mã nguồn mở và cộng đồng mạnh mẽ:** Là một dự án mã nguồn mở, Metabase không tốn chi phí bản quyền, đồng thời được hưởng lợi từ sự phát

triển và hỗ trợ của một cộng đồng người dùng và nhà phát triển lớn mạnh toàn cầu.

- **Khả năng tạo Dashboard linh hoạt:** Người dùng có thể dễ dàng kết hợp nhiều câu hỏi và biểu đồ khác nhau vào một dashboard tổng hợp, giúp theo dõi các chỉ số quan trọng (KPIs) và nắm bắt bức tranh toàn cảnh về hoạt động.
- **Hỗ trợ SQL Mode cho người dùng nâng cao:** Bên cạnh giao diện trực quan, Metabase vẫn cung cấp chế độ soạn thảo SQL cho phép các nhà phân tích dữ liệu hoặc người dùng có kỹ năng viết các truy vấn phức tạp và tùy chỉnh theo nhu cầu.
- **Quản lý quyền truy cập và chia sẻ dễ dàng:** Metabase cho phép quản trị viên thiết lập quyền truy cập chi tiết cho từng người dùng hoặc nhóm người dùng đối với các nguồn dữ liệu, câu hỏi, và dashboards, đảm bảo tính bảo mật và phù hợp.
- **Khả năng nhúng (Embedding):** Các biểu đồ và dashboards từ Metabase có thể được nhúng vào các ứng dụng hoặc trang web khác, mở rộng khả năng chia sẻ thông tin.
- **Triển khai và cài đặt đơn giản:** So với nhiều công cụ BI khác, Metabase có quy trình cài đặt và cấu hình ban đầu tương đối nhanh chóng và không phức tạp.

Kiến trúc kết nối và hoạt động giữa Metabase và Trino:

Trong kiến trúc hệ thống của dự án, Metabase không trực tiếp thực thi các truy vấn tính toán nặng lên nguồn dữ liệu gốc. Thay vào đó, Metabase đóng vai trò là giao diện người dùng cuối (front-end) cho việc khám phá dữ liệu và trực quan hóa, trong khi **Trino** (như đã được đề cập ở phần trước) đảm nhận vai trò của một **query engine** (bộ máy truy vấn) mạnh mẽ và phân tán.

Quy trình tương tác và xử lý dữ liệu diễn ra như sau:

1. **Người dùng tương tác với Metabase:** Người dùng cuối sử dụng giao diện của Metabase để đặt câu hỏi, lựa chọn các trường dữ liệu, áp dụng bộ lọc, hoặc làm mới một dashboard đã có.
2. **Metabase tạo và gửi truy vấn đến Trino:** Dựa trên thao tác của người dùng, Metabase sẽ biên dịch yêu cầu đó thành một câu lệnh SQL (hoặc sử dụng câu lệnh SQL do người dùng tự viết trong SQL Mode). Câu lệnh SQL

này sau đó được Metabase gửi đến Trino để thực thi.

3. **Trino thực thi truy vấn:** Trino nhận câu lệnh SQL từ Metabase và tiến hành thực thi truy vấn này trên các nguồn dữ liệu đã được cấu hình (ví dụ: data lake, kho dữ liệu). Trino có khả năng truy vấn liên nguồn (federated queries), tối ưu hóa và thực thi các tác vụ tính toán phức tạp một cách hiệu quả nhờ kiến trúc phân tán của mình.
4. **Trino trả kết quả về cho Metabase:** Sau khi thực thi xong, Trino trả về tập kết quả (result set) cho Metabase.
5. **Metabase trực quan hóa kết quả:** Metabase nhận dữ liệu kết quả từ Trino và sử dụng các thư viện trực quan hóa của mình để hiển thị thông tin dưới dạng biểu đồ, bảng biểu, hoặc các dạng trực quan khác mà người dùng đã chọn.

Sự phân tách vai trò này mang lại nhiều lợi ích:

- **Tận dụng sức mạnh tính toán của Trino:** Toàn bộ gánh nặng xử lý truy vấn phức tạp, tổng hợp dữ liệu lớn được chuyển cho Trino, giúp đảm bảo hiệu suất và tốc độ phản hồi nhanh chóng, ngay cả với khối lượng dữ liệu lớn.
- **Giảm tải cho Metabase:** Metabase có thể tập trung vào việc cung cấp trải nghiệm người dùng tốt nhất cho việc hiển thị và tương tác dữ liệu, trở nên nhẹ nhàng và linh hoạt hơn.
- **Khả năng truy cập dữ liệu đa dạng:** Thông qua Trino, Metabase gián tiếp có khả năng truy cập và phân tích dữ liệu từ nhiều hệ thống lưu trữ khác nhau mà Trino hỗ trợ, tạo nên một giải pháp BI thống nhất trên nền tảng dữ liệu đa dạng.
- **Tối ưu hóa chi phí và tài nguyên:** Sử dụng Metabase (mã nguồn mở) kết hợp với Trino (cũng là mã nguồn mở và có khả năng mở rộng linh hoạt) giúp tối ưu hóa chi phí bản quyền và cho phép điều chỉnh tài nguyên tính toán phù hợp với nhu cầu thực tế.

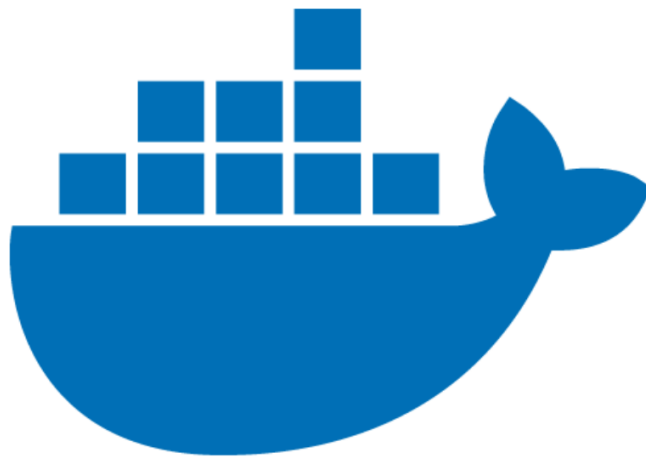
Bằng cách này, Metabase và Trino tạo thành một cặp đôi mạnh mẽ, nơi Trino đóng vai trò "bộ não" xử lý dữ liệu, còn Metabase là "gương mặt" thân thiện giúp người dùng khám phá và hiểu dữ liệu một cách dễ dàng và hiệu quả.

2.2.6 Common Services

Trong việc xây dựng một nền tảng dữ liệu hiện đại và linh hoạt, việc tích hợp các dịch vụ chung (Common Services) đóng vai trò then chốt để đảm bảo tính ổn định, khả năng mở rộng, và hiệu quả vận hành. Các dịch vụ này tạo nên một bộ khung vững chắc, hỗ trợ cho các thành phần xử lý, lưu trữ và phân tích dữ liệu hoạt động một cách trơn tru. Dự án này tận dụng một số công cụ mã nguồn mở hàng đầu cho các dịch vụ chung như sau:

1. Containerization với Docker

Containerization, hay công nghệ đóng gói ứng dụng bằng container, là một phương pháp tiên tiến để phát triển, triển khai và vận hành ứng dụng một cách nhất quán trên nhiều môi trường khác nhau. Trong dự án này, **Docker** [4] được lựa chọn làm nền tảng containerization chính để triển khai và quản lý vòng đời của toàn bộ các thành phần mã nguồn mở được sử dụng (ví dụ: Trino, Metabase, Airflow, Nessie, Prometheus, Grafana, v.v.).



Hình 2.18: Công cụ Docker

Lý do và lợi ích của việc sử dụng Docker:

- **Môi trường nhất quán:** Docker đảm bảo rằng mỗi ứng dụng và các thành phần phụ thuộc của nó (thư viện, tệp cấu hình) được đóng gói vào một *Docker image* duy nhất. Image này sau đó được chạy dưới dạng một *Docker container*, mang lại sự đồng nhất giữa môi trường phát triển (development), kiểm thử (testing) và sản phẩm (production), qua đó giảm thiểu đáng kể lỗi phát sinh do sự khác biệt môi trường ("It works on my machine" problem).

- **Triển khai nhanh chóng và dễ dàng:** Việc triển khai các ứng dụng phức tạp trở nên đơn giản hơn nhiều. Chỉ cần có Docker runtime, các container có thể được khởi chạy từ image tương ứng một cách nhanh chóng mà không cần lo lắng về việc cài đặt thủ công các phụ thuộc.
- **Cách ly tài nguyên và ứng dụng:** Mỗi container hoạt động trong một môi trường biệt lập, có namespace riêng cho process, network, và filesystem. Điều này giúp các ứng dụng không ảnh hưởng lẫn nhau, tăng cường tính ổn định và bảo mật.
- **Quản lý phụ thuộc hiệu quả:** Docker cho phép định nghĩa rõ ràng các phiên bản của thư viện và runtime cho từng ứng dụng, tránh xung đột phiên bản giữa các thành phần khác nhau của hệ thống.
- **Khả năng mở rộng (Scalability):** Các ứng dụng được container hóa có thể dễ dàng được nhân bản (replicated) để đáp ứng nhu cầu tải tăng cao, thường được kết hợp với các công cụ điều phối container như Kubernetes hoặc Docker Swarm (tuy nhiên, phạm vi dự án này có thể chỉ tập trung vào Docker cho việc triển khai đơn giản hơn).
- **Tối ưu hóa tài nguyên:** So với máy ảo truyền thống (VMs), container nhẹ hơn do chia sẻ kernel của hệ điều hành máy chủ, dẫn đến việc sử dụng tài nguyên (CPU, memory) hiệu quả hơn và thời gian khởi động nhanh hơn.

Trong dự án, việc sử dụng Docker giúp chuẩn hóa quy trình triển khai, đơn giản hóa việc nâng cấp và bảo trì các dịch vụ mã nguồn mở, đồng thời tạo điều kiện thuận lợi cho việc tự động hóa và tích hợp liên tục (CI/CD) nếu cần.

2. Quản lý Metadata Catalog với Nessie cho Iceberg

Trong một hệ sinh thái Lakehouse hiện đại, việc quản lý metadata một cách hiệu quả là cực kỳ quan trọng để đảm bảo tính nhất quán, khả năng khám phá dữ liệu và quản trị dữ liệu (data governance). Đặc biệt khi sử dụng định dạng bảng mở như Apache Iceberg, một catalog mạnh mẽ là điều cần thiết. Dự án này sử dụng **Project Nessie** [9] làm catalog tập trung để quản lý metadata cho các bảng Apache Iceberg.



Hình 2.19: Công cụ quản lý catalog tập trung Nessie

Vai trò và lợi ích của Nessie kết hợp với Iceberg:

- **Quản lý metadata giao dịch:** Nessie cung cấp khả năng thực hiện các thay đổi metadata (như tạo bảng, thay đổi schema, cập nhật snapshot của Iceberg) dưới dạng các giao dịch nguyên tử (atomic transactions). Điều này đảm bảo rằng metadata luôn ở trạng thái nhất quán.
- **Hỗ trợ ngữ nghĩa Git-cho-Dữ liệu:** Đây là một trong những ưu điểm vượt trội của Nessie. Nó cho phép:
 - Branching: Tạo các nhánh (branches) riêng biệt của catalog để thử nghiệm các thay đổi dữ liệu, phát triển ETL, hoặc thực hiện các phân tích mà không ảnh hưởng đến nhánh chính (main/production).
 - Tagging: Gắn thẻ (tags) cho các phiên bản cụ thể của dữ liệu (ví dụ: "end-of-month-report", "v1.0-dataset"), giúp dễ dàng truy cập lại các snapshot lịch sử quan trọng.
 - Merging: Trộn các thay đổi từ một nhánh vào nhánh khác sau khi đã được kiểm thử và xác nhận.
 - Commits: Mỗi thay đổi metadata được ghi nhận như một commit, tạo ra một lịch sử phiên bản rõ ràng.
- **Đảm bảo tính nhất quán dữ liệu giữa các engine:** Nessie hoạt động như một "single source of truth" cho metadata của Iceberg. Điều này cho phép query engine như Trino và processing engine như Spark cùng làm việc trên một tập dữ liệu nhất quán, nhìn thấy cùng một phiên bản của bảng tại một

thời điểm hoặc một commit cụ thể. Ví dụ, một pipeline ETL ghi dữ liệu vào bảng Iceberg thông qua Spark, và người dùng cuối truy vấn dữ liệu đó gần như ngay lập tức thông qua Trino, tất cả đều dựa trên metadata được quản lý bởi Nessie.

- **Khả năng tái tạo dữ liệu:** Với khả năng truy cập các phiên bản lịch sử của metadata, người dùng có thể "du hành thời gian" để truy vấn trạng thái của bảng tại một commit hoặc tag cụ thể, rất hữu ích cho việc debug, audit, hoặc tái tạo các phân tích.

Bằng cách sử dụng Nessie, dự án có được một giải pháp quản lý metadata mạnh mẽ, mang lại sự linh hoạt, khả năng quản trị phiên bản và tính nhất quán cao cho các bảng dữ liệu Iceberg, từ đó nâng cao độ tin cậy và hiệu quả của toàn bộ Data platform.

3. Điều phối Pipeline Dữ liệu với Apache Airflow

Để tự động hóa, lập lịch và giám sát các luồng công việc dữ liệu (data pipelines) phức tạp, việc sử dụng một công cụ điều phối (orchestration tool) là không thể thiếu. Dự án này lựa chọn **Apache Airflow** [1] làm nền tảng điều phối pipeline. Airflow là một công cụ mã nguồn mở mạnh mẽ cho phép người dùng định nghĩa, lập lịch và theo dõi các luồng công việc dưới dạng mã Python



Hình 2.20: Công cụ điều phối luồng Airflow

Vai trò và lợi ích của Apache Airflow:

- **Định nghĩa Pipeline bằng Code:** Các luồng công việc trong Airflow được định nghĩa dưới dạng các **Directed Acyclic Graphs (DAGs)** bằng Python. Điều này mang lại sự linh hoạt cao, cho phép quản lý phiên bản, kiểm thử và tùy biến pipeline một cách dễ dàng.
- **Lập lịch linh hoạt:** Airflow cung cấp cơ chế lập lịch mạnh mẽ để tự động

kích hoạt các DAGs theo thời gian định sẵn hoặc theo sự kiện.

- **Quản lý phụ thuộc tác vụ:** Bên trong một DAG, các tác vụ (tasks) có thể được định nghĩa với các mối quan hệ phụ thuộc rõ ràng, đảm bảo rằng một tác vụ chỉ chạy khi các tác vụ điều kiện tiên quyết của nó đã hoàn thành thành công.
- **Giao diện người dùng (UI) trực quan:** Airflow cung cấp một giao diện web phong phú để theo dõi trạng thái của các DAG runs, tiến độ của các tasks, xem logs, và quản lý các kết nối, biến số.
- **Khả năng mở rộng và tùy biến cao:** Với kiến trúc dựa trên các *Operators* (ví dụ: BashOperator, PythonOperator, TrinoOperator, SparkSubmitOperator, DockerOperator) và *Plugins*, Airflow có thể dễ dàng được mở rộng để tương tác với hầu hết mọi loại hệ thống và dịch vụ. Trong dự án này, Airflow có thể điều phối các tác vụ như:
 - Kích hoạt các job Spark để xử lý và biến đổi dữ liệu trên Iceberg.
 - Kích hoạt Docker container để tạo môi trường cho Spark driver
 - Điều phối và lập lịch các quy trình crawl dữ liệu
 - Điều phối và lập lịch các quy trình ETL dữ liệu
- **Xử lý lỗi và thử lại:** Airflow cho phép cấu hình cơ chế thử lại tự động cho các tác vụ bị lỗi, cũng như định nghĩa các hành động cần thực hiện khi có lỗi xảy ra (ví dụ: gửi thông báo).
- **Logging tập trung:** Logs từ tất cả các tác vụ được thu thập và có thể truy cập dễ dàng qua UI, giúp cho việc gỡ lỗi trở nên thuận tiện.

Việc tích hợp Airflow giúp dự án tự động hóa các quy trình dữ liệu phức tạp, từ thu thập, xử lý, đến chuẩn bị dữ liệu cho phân tích, đảm bảo tính tin cậy, đúng hạn và dễ dàng giám sát toàn bộ vòng đời dữ liệu.

4. Giám sát Hệ thống và Pipeline với Prometheus và Grafana

Để đảm bảo sự ổn định, hiệu năng và chất lượng của toàn bộ nền tảng dữ liệu cũng như các pipeline đang vận hành, một hệ thống giám sát toàn diện là yếu tố cốt lõi. Dự án này triển khai giải pháp giám sát kết hợp giữa **Prometheus** [3] để thu thập và lưu trữ metrics, và **Grafana** [6] để trực quan hóa và cảnh báo.



Hình 2.21: Công cụ Monitoring hiệu quả

Prometheus:

- **Thu thập Metrics:** Prometheus là một hệ thống giám sát mã nguồn mở hoạt động theo mô hình kéo (pull model), nơi nó định kỳ truy cập các *endpoints* được cấu hình (thường là các services, applications, servers) để thu thập các số liệu hiện tại. Nhiều công cụ mã nguồn mở (bao gồm Docker, Trino, Airflow, Nessie nếu có exporter) cung cấp sẵn metrics ở định dạng Prometheus.
- **Time Series Database - TSDB:** Các metrics thu thập được lưu trữ trong một cơ sở dữ liệu chuỗi thời gian được tối ưu hóa, cho phép truy vấn và phân tích hiệu quả.
- **Ngôn ngữ truy vấn PromQL:** Prometheus cung cấp một ngôn ngữ truy vấn mạnh mẽ là PromQL (Prometheus Query Language) để người dùng có thể lựa chọn, tổng hợp và thực hiện các phép toán trên dữ liệu metrics.
- **Cơ chế cảnh báo:** Thông qua thành phần Alertmanager, Prometheus cho phép định nghĩa các quy tắc cảnh báo dựa trên các biểu thức PromQL. Khi một điều kiện cảnh báo được kích hoạt, Alertmanager có thể gửi thông báo qua nhiều kênh khác nhau (email, Slack, PagerDuty, v.v.).

Grafana:

- **Trực quan hóa Dữ liệu:** Grafana là một nền tảng mã nguồn mở hàng đầu cho việc trực quan hóa và phân tích dữ liệu metrics. Nó cho phép người dùng tạo ra các dashboards tương tác và đa dạng từ nhiều nguồn dữ liệu khác nhau.

- **Kết nối với Prometheus:** Grafana tích hợp chặt chẽ với Prometheus như một nguồn dữ liệu, cho phép truy vấn metrics bằng PromQL trực tiếp từ giao diện Grafana và hiển thị chúng dưới dạng biểu đồ, đồ thị, bảng, gauges, v.v.
- **Tạo Dashboard tùy chỉnh:** Người dùng có thể dễ dàng xây dựng các dashboards tùy chỉnh để theo dõi các chỉ số quan trọng. Ví dụ:
 - **Giám sát tài nguyên hệ thống:** CPU, bộ nhớ, ổ đĩa, băng thông mạng của các máy chủ và các container Docker.
 - **Giám sát chất lượng pipeline dữ liệu:** Trạng thái các DAGs và tasks trong Airflow (thành công, thất bại, thời gian chạy), số lượng bản ghi được xử lý, độ trễ dữ liệu.
- **Quản lý cảnh báo:** Grafana cũng có thể hiển thị và quản lý các cảnh báo được tạo ra từ Prometheus Alertmanager.

Lợi ích của giải pháp giám sát kết hợp:

- **Phát hiện sớm vấn đề:** Giúp nhanh chóng xác định các sự cố về hiệu năng, lỗi hệ thống hoặc vấn đề về chất lượng dữ liệu, từ đó giảm thiểu thời gian chết (downtime) và ảnh hưởng đến người dùng.
- **Tối ưu hóa hiệu năng:** Cung cấp thông tin chi tiết về việc sử dụng tài nguyên và hiệu suất của các thành phần, làm cơ sở cho việc tối ưu hóa và điều chỉnh cấu hình.
- **Đảm bảo độ tin cậy và ổn định:** Theo dõi liên tục giúp duy trì sự ổn định của hệ thống và các pipeline dữ liệu.
- **Nâng cao khả năng hiểu biết hệ thống:** Cung cấp cái nhìn sâu sắc về cách hệ thống hoạt động và tương tác với nhau.

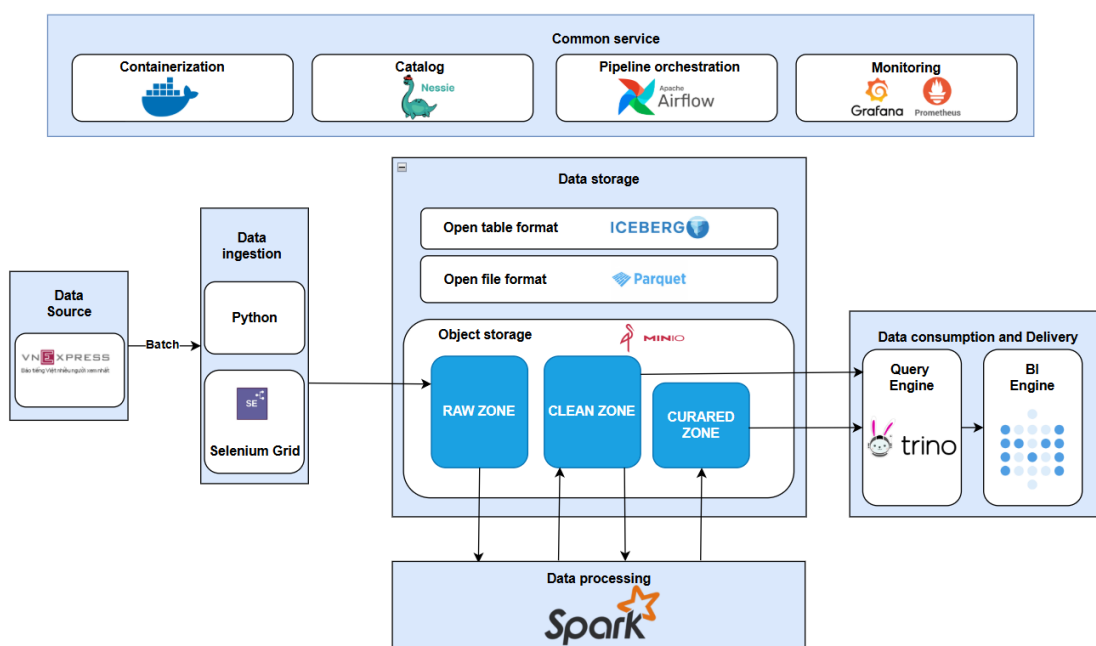
Bằng việc kết hợp Prometheus và Grafana, dự án thiết lập một hệ thống giám sát mạnh mẽ, cho phép theo dõi toàn diện từ hạ tầng cơ sở đến các ứng dụng và quy trình dữ liệu, đảm bảo hoạt động hiệu quả và tin cậy.

Chương 3

Lập trình xây dựng hệ thống Lakehouse

3.1 Kiến trúc toàn bộ dự án

Sau khi phân tích và thiết kế, tôi đưa ra kiến trúc toàn bộ dự án.



Hình 3.1: Kiến trúc tổng thể dự án

Mã nguồn dự án ở lưu trữ remote tại repo [Github](#), với cấu trúc cây thư mục sau:

```

-
├─ Apache_Spark
│   ├── apps
│   ├── docker_client
│   └─ spark.yml
├─ MinIO
│   └─ MinIO.yml
├─ Monitoring
│   ├── alertmanager
│   ├── grafana
│   ├── monitoring.yml
│   └─ prometheus
├─ Nessie
│   ├── nessie.sh
│   └─ nessie.yml
├─ Trino
│   ├── etc
│   └─ trino.yml
├─ Web_scraping
│   ├── CrawlJob
│   ├── CrawlPackage
│   ├── SeleniumPackage
│   └─ chrome.yml
├─ airflow
│   ├── airflow.yml
│   ├── config
│   ├── dags
│   ├── logs
│   └─ plugins
├─ docker-compose-main.yml
└─ metabase
    └─ metabase.yml

```

Hình 3.2: Cấu trúc cây thư mục

3.2 Lập trình và xây dựng quá trình thu thập dữ liệu

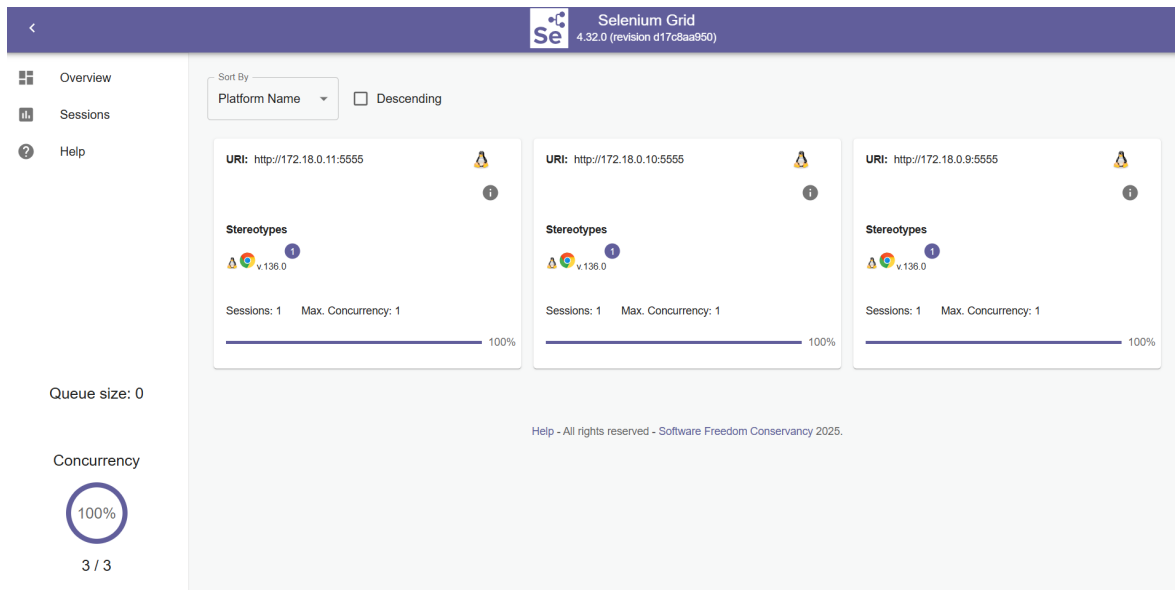
Quá trình thu thập dữ liệu là bước đầu tiên và quan trọng trong dự án, được thực hiện bằng ngôn ngữ Python kết hợp với thư viện Selenium để tự động hóa việc trích xuất tin tức từ trang web VnExpress, như đã đề cập trong phần phân tích và thiết kế.

Phạm vi dữ liệu thu thập kéo dài từ năm 2021 đến thời điểm hiện tại. Dữ liệu được lấy từ 11 chuyên mục được xem là có giá trị cao cho việc phân tích, bao gồm:

- Thời sự
- Góc nhìn
- Thế giới
- Kinh doanh
- Bất động sản
- Giáo dục
- Pháp luật
- Sức khỏe
- Đời sống
- Du lịch
- Khoa học

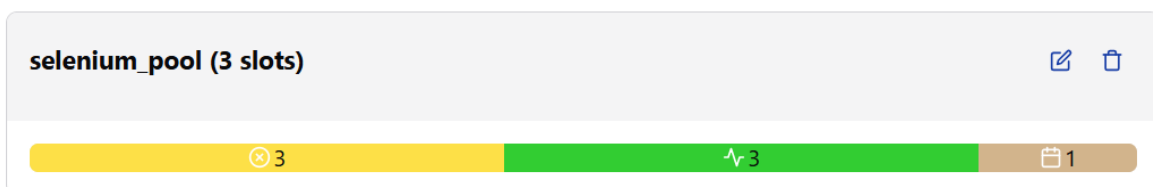
Tổng khối lượng dữ liệu thô thu thập được hơn 1 GB. Nếu thực hiện tuần tự, thời gian để hoàn tất quá trình này ước tính mất hơn 300 giờ. Đây là một thách thức lớn về mặt thời gian, đòi hỏi một giải pháp tối ưu để tăng tốc độ thu thập.

Để giải quyết vấn đề này, một kiến trúc thu thập dữ liệu song song đã được thiết kế và triển khai bằng cách sử dụng Selenium Grid. Mô hình này cho phép thực thi nhiều phiên trình duyệt cùng một lúc, giúp rút ngắn đáng kể thời gian chờ, còn chỉ khoảng 150 giờ lấy dữ liệu. Cụm Selenium Grid được cấu hình với một Hub làm trung tâm điều phối và ba Node thực thi (Chrome browser). Hub nhận các yêu cầu crawl và phân phối chúng đến các Node rảnh rỗi.



Hình 3.3: Mô hình Selenium Grid với 1 Hub và 3 Node đang hoạt động

Để quản lý và điều phối các tác vụ crawl một cách hiệu quả, hệ thống sử dụng Apache Airflow, được triển khai ở chế độ CeleryExecutor mode. Nhằm đồng bộ hóa số lượng tác vụ song song với năng lực của cụm Selenium Grid, một *pool* trong Airflow đã được cấu hình với giới hạn là 3 tác vụ. Cơ chế này đảm bảo rằng tại mỗi thời điểm, chỉ có tối đa 3 tác vụ crawl được chạy đồng thời, tương ứng với 3 Node của Selenium Grid, giúp tối ưu hóa việc sử dụng tài nguyên và tránh tình trạng quá tải.



Hình 3.4: Cấu hình *Selenium Pool* trên Airflow giới hạn 3 tác vụ song song

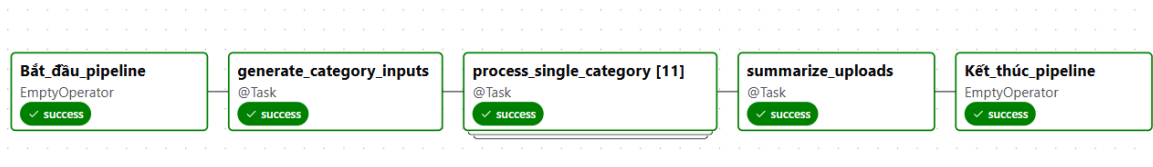
Ngoài ra, tôi cũng viết một package riêng dành cho việc crawl dữ liệu. Đó là các hàm mà tôi viết để có thể tái sử dụng cho cùng một hành động nào đó.

3.2.1 Crawl dữ liệu trong 1 ngày bất kì

Để đáp ứng nhu cầu thu thập dữ liệu cho một ngày cụ thể hoặc chạy lại (backfill) dữ liệu cho những ngày bị thiếu, một luồng công việc (pipeline) tự động đã được xây dựng trên Apache Airflow. Pipeline này cho phép người vận

hành chủ động kích hoạt việc thu thập dữ liệu với một tham số đầu vào là ngày thực thi theo định dạng `yyyymmdd`. Nếu không được cung cấp, hệ thống sẽ mặc định lấy ngày hiện tại (tức ngày 06/06/2025).

Hình 3.5 minh họa giao diện đồ thị của luồng công việc (DAG - Directed Acyclic Graph) này. Toàn bộ quá trình được tự động hóa và bao gồm các bước chính sau:

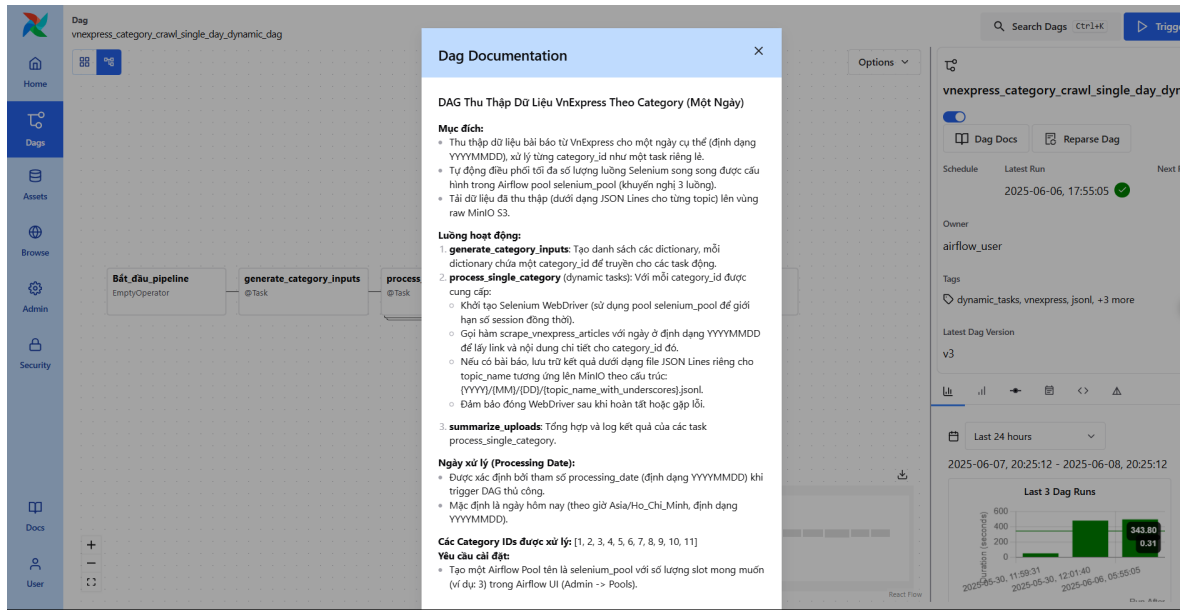


Hình 3.5: Pipeline trên Airflow cho tác vụ crawl dữ liệu theo ngày

- **generate_category_inputs:** Khi pipeline được kích hoạt, tác vụ đầu tiên sẽ khởi tạo và tạo ra một danh sách chứa 11 chuyên mục mục tiêu cần thu thập.
- **process_single_category[11]:** Đây là bước xử lý cốt lõi. Tận dụng cơ chế *dynamic task mapping* của Airflow, hệ thống tự động nhân bản tác vụ xử lý thành 11 tác vụ con chạy song song. Mỗi tác vụ con đảm nhiệm việc crawl toàn bộ tin tức của một chuyên mục trong ngày đã định. Kiến trúc này giúp khai thác triệt để hiệu năng của cụm Selenium Grid đã được thiết lập ở phần trước.
- **summarize_uploads:** Sau khi tất cả 11 tác vụ song song hoàn thành, một tác vụ tổng hợp sẽ được thực thi. Nhiệm vụ của nó là thu thập siêu dữ liệu (metadata) từ các bước trước, ghi nhận lại số lượng bài báo đã thu thập, kiểm tra lỗi và thông báo kết quả cuối cùng.

Cách thiết kế này không chỉ giúp thu thập lại dữ liệu một cách linh hoạt mà còn đảm bảo hiệu suất xử lý luôn ở mức cao nhất nhờ vào việc song song hóa các tác vụ độc lập.

Ngoài ra, tôi cũng đã viết một tài liệu làm cho Dag trở nên tường minh hơn, việc viết tài liệu là một thực hành tốt khi nó cung cấp metadata cho Data platform.



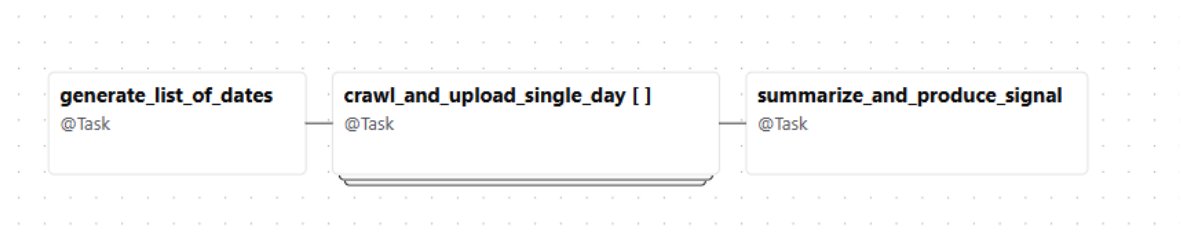
Hình 3.6: Tài liệu cho dag crawl dữ liệu cho một ngày

3.2.2 Crawl dữ liệu trong một khoảng ngày bất kì

Để xử lý việc thu thập dữ liệu lịch sử trên quy mô lớn, một pipeline có khả năng cấu hình theo khoảng thời gian **vnexpress_daily_crawl_range_day_dag** đã được phát triển. Đây là luồng công việc chính và được sử dụng thường xuyên nhất cho các tác vụ *backfill* dữ liệu lớn, có thể kéo dài nhiều ngày hoặc thậm chí nhiều tháng.

Điểm khác biệt cốt lõi của pipeline này so với pipeline ở mục trước nằm ở chiến lược song song hóa: thay vì xử lý song song các chuyên mục trong một ngày, nó sẽ xử lý song song **toàn bộ các ngày** trong khoảng thời gian được người dùng chỉ định.

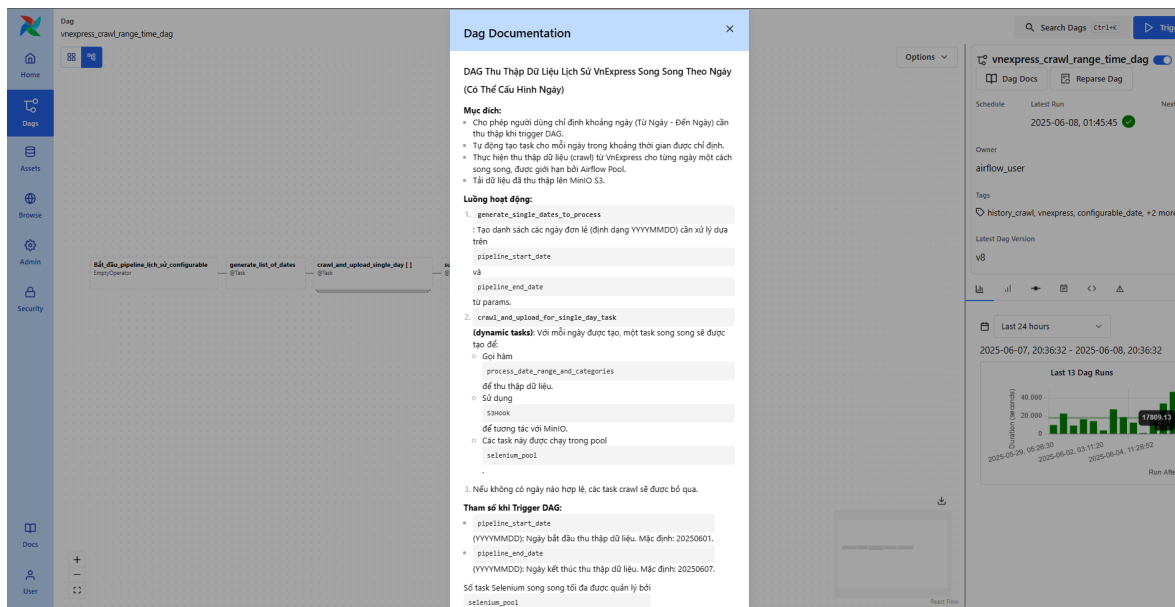
Ngoài ra, đây cũng là dag chính được thiết lập chạy hằng ngày, với mặc định lấy dữ liệu 7 ngày gần nhất. Lý do cần lấy khoảng ngày như vậy, là vì lượng tương tác các bài báo sẽ cần một khoảng thời gian để tăng lên.



Hình 3.7: Luồng DAG xử lý song song nhiều ngày trong một khoảng thời gian

1. **generate_list_of_dates:** Dựa trên hai tham số đầu vào, tác vụ này tạo ra một danh sách chứa tất cả các ngày cần xử lý.
2. **crawl_and_upload_single_day:** Sử dụng cơ chế dynamic task mapping, hệ thống sẽ lặp qua danh sách ngày đã tạo. Với mỗi ngày, nó sẽ kích hoạt một phiên chạy mới của `vnexpress_crawl_single_day_dag`. Các phiên chạy này được đưa vào hàng đợi và thực thi song song, giới hạn bởi dung lượng của *Selenium Pool* đã định nghĩa.
3. **summarize_and_produce_signal:** Sau khi tất cả các DAG con đã hoàn thành, tác vụ cuối cùng này sẽ chạy để tổng hợp lại kết quả từ toàn bộ các ngày, cung cấp một báo cáo cuối cùng về tác vụ. Đặc biệt, khi mà task này được lên lịch tự động.

Giao diện quản lý của Airflow trong Hình 3.8 cung cấp một cái nhìn tổng quan, bao gồm cả phần tài liệu hướng dẫn được tích hợp sẵn để người vận hành dễ dàng sử dụng.



Hình 3.8: Tài liệu cho dag crawl dữ liệu khoảng ngày bất kì

3.3 Lập trình và xây dựng phần lưu trữ cho Lakehouse

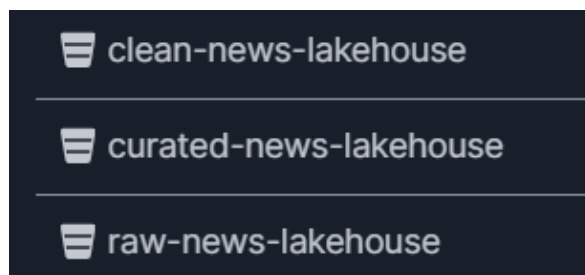
Nền tảng lưu trữ là thành phần xương sống của bất kỳ hệ thống dữ liệu nào. Đối với dự án Lakehouse này, giải pháp lưu trữ đối tượng (object storage) mã nguồn mở MinIO được lựa chọn làm nền tảng chính, như đã phân tích ở các chương trước. Trong khuôn khổ dự án, một cụm MinIO cluster gồm 4 container

được triển khai để mô phỏng một môi trường lưu trữ phân tán, có khả năng chịu lỗi và mở rộng.

3.3.1 Kiến trúc lưu trữ đa vùng

Để quản lý dữ liệu một cách khoa học và hiệu quả qua các giai đoạn xử lý, kiến trúc lưu trữ đa vùng được áp dụng. Dữ liệu sẽ được phân tách và lưu trữ tại 3 vùng riêng biệt, tương ứng với 3 bucket trên MinIO, mỗi vùng có một vai trò và đặc tính riêng:

- **Vùng Thô (Raw Zone):** Là nơi tiếp nhận và lưu trữ dữ liệu gốc, không qua chỉnh sửa từ nguồn (VnExpress). Dữ liệu ở đây có tính bất biến và là nguồn chân lý cho toàn bộ hệ thống.
- **Vùng Sạch (Clean Zone):** Dữ liệu từ vùng Raw sau khi trải qua quá trình ETL (Trích xuất, Chuyển đổi, Tải) đầu tiên sẽ được lưu tại đây. Tại vùng này, dữ liệu đã được làm sạch, chuẩn hóa cấu trúc, định kiểu và loại bỏ các thông tin dư thừa.
- **Vùng Tinh Chế (Curated Zone):** Đây là vùng chứa dữ liệu đã sẵn sàng cho mục đích phân tích. Dữ liệu ở đây đã được tổng hợp, làm giàu và cấu trúc lại theo các mô hình kinh doanh (business models) cụ thể, phục vụ trực tiếp cho các công cụ BI, dashboard.



Hình 3.9: Ba bucket trên MinIO tương ứng với 3 vùng của Lakehouse

3.3.2 Chi tiết cấu trúc và định dạng tệp qua các vùng

1. Vùng raw

Dữ liệu từ quá trình crawl được lưu trực tiếp vào vùng Raw dưới định dạng JSONL. Mỗi tệp chứa nhiều đối tượng JSON, mỗi đối tượng trên một dòng, giúp cho việc đọc và xử lý streaming trở nên hiệu quả. Cấu trúc lưu trữ được phân cấp

theo `raw-news-lakehouse/năm/tháng/ngày/topic.jsonl`, giúp dễ dàng truy vấn và quản lý dữ liệu theo thời gian.

The screenshot shows the 'raw-news-lakehouse' interface. At the top, it says 'Created on: Sun, May 18 2025 02:10:11 (GMT+7)', 'Access: PRIVATE', and '1.0 GiB - 15631 Objects'. Below this is a breadcrumb path: 'raw-news-lakehouse / 2025 / 05 / 01'. The main table lists files with columns 'Name', 'Last Modified', and 'Size'.

Name	Last Modified	Size
Bất động_sản.jsonl	Wed, May 28 2025 07:42 (GMT+7)	15.6 KiB
Du_lịch.jsonl	Wed, May 28 2025 07:49 (GMT+7)	65.9 KiB
Đời_sống.jsonl	Wed, May 28 2025 07:48 (GMT+7)	42.9 KiB
Giáo_dục.jsonl	Wed, May 28 2025 07:44 (GMT+7)	28.7 KiB
Góc_nhìn.jsonl	Wed, May 28 2025 07:36 (GMT+7)	8.0 KiB
Khoa_học.jsonl	Wed, May 28 2025 07:50 (GMT+7)	37.2 KiB
Kinh_doanh.jsonl	Wed, May 28 2025 07:41 (GMT+7)	78.0 KiB
Pháp_luật.jsonl	Wed, May 28 2025 07:43 (GMT+7)	40.7 KiB
Sức_khỏe.jsonl	Wed, May 28 2025 07:47 (GMT+7)	109.6 KiB
Thế_gới.jsonl	Wed, May 28 2025 07:40 (GMT+7)	127.3 KiB
Thời_sự.jsonl	Wed, May 28 2025 07:36 (GMT+7)	76.6 KiB

Hình 3.10: Cấu trúc thư mục và định dạng tệp tại vùng Raw

2. Vùng Clean và Curated

Sau quá trình xử lý bằng Apache Spark, dữ liệu từ vùng Raw được chuyển đổi và lưu vào vùng Clean, và sau đó là vùng Curated.

The image shows two side-by-side screenshots of Lakehouse interfaces. The left screenshot is for 'clean-news-lakehouse' (Created on: Fri, Jun 13 2025 18:29:02 (GMT+7), Access: PRIVATE, 15.5 KiB - 10 Objects). It shows a directory listing for 'clean-news-lakehouse / nessie_clean_news_warehouse / news_clean_db' with various JSON files. The right screenshot is for 'curated-news-lakehouse' (Created on: Sat, May 24 2025 22:13:48 (GMT+7), Access: PRIVATE, 75.6 KiB - 16 Objects). It shows a directory listing for 'curated-news-lakehouse / nessie_curated_news_warehouse / news_curated_db' with various JSON files.

Hình 3.11: Đại diện cho các bảng có trong 2 vùng clean và curated

Tại hai vùng này, có hai sự thay đổi quan trọng về mặt kỹ thuật:

1. **Chuyển đổi định dạng tệp:** Dữ liệu được chuyển đổi từ JSONL sang định dạng cột **Apache Parquet**. Parquet tối ưu cho các truy vấn phân tích (OLAP) nhờ khả năng nén dữ liệu hiệu quả và chỉ đọc những cột cần thiết (column projection), giúp tăng tốc độ truy vấn lên đáng kể.
2. **Áp dụng định dạng bảng mở:** Hệ thống sử dụng **Apache Iceberg** để quản lý dữ liệu. Iceberg tạo ra một lớp trừu tượng phía trên các tệp Parquet, tổ chức chúng thành các bảng có cấu trúc rõ ràng.



Hình 3.12: Cấu trúc bảng Iceberg tại vùng Clean và Curated với thư mục data và metadata

Cấu trúc của một bảng Iceberg bao gồm hai thư mục chính là data (chứa các tệp Parquet) và metadata (chứa các thông tin về schema, snapshots, manifests...). Chính lớp metadata này mang lại các tính năng mạnh mẽ cho Lakehouse như giao tác ACID, du hành thời gian (time travel), và phát triển schema một cách an toàn.

3.4 Lập trình và xây dựng phần ETL dữ liệu

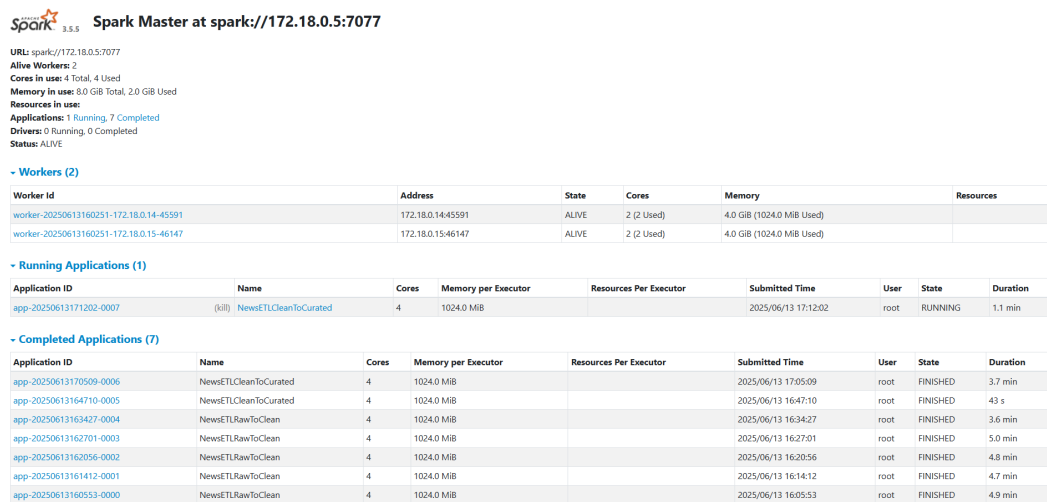
Quá trình ETL (Extract - Transform - Load) là trung tâm của hệ thống Lakehouse, đóng vai trò biến đổi dữ liệu thô thành những thông tin có giá trị và sẵn sàng cho việc phân tích. Nhiệm vụ chính của tầng này là áp dụng các quy tắc logic để xử lý và định hình dữ liệu theo các mô hình đã được thiết kế ở các vùng (zone) khác nhau.

3.4.1 Kiến trúc và Công nghệ sử dụng

1. Apache Spark cho xử lý phân tán

Như đã phân tích, Apache Spark được lựa chọn làm công cụ xử lý chính nhờ khả năng xử lý dữ liệu lớn tốc độ cao, hệ sinh thái API phong phú (Spark SQL, DataFrame API) và khả năng mở rộng linh hoạt. Trong dự án này, một cụm Spark được triển khai ở chế độ độc lập (Standalone Mode), bao gồm:

- **1 Master Node:** Đóng vai trò quản lý tài nguyên và điều phối công việc cho toàn bộ cụm.
- **2 Worker Nodes:** Chịu trách nhiệm thực thi các tác vụ (tasks) xử lý dữ liệu thực tế.



Spark Master at spark://172.18.0.5:7077

URL: spark://172.18.0.5:7077
 Alive Workers: 2
 Cores in use: 4 Total, 4 Used
 Memory in use: 8.0 GiB Total, 2.0 GiB Used
 Resources in use:
 Applications: 1 Running, 7 Completed
 Drivers: 0 Running, 0 Completed
 Status: ALIVE

- Workers (2)

Worker Id	Address	State	Cores	Memory	Resources
worker-20250613160251-172.18.0.14-45591	172.18.0.14:45591	ALIVE	2 (2 Used)	4.0 GiB (1024.0 MiB Used)	
worker-20250613160251-172.18.0.15-46147	172.18.0.15:46147	ALIVE	2 (2 Used)	4.0 GiB (1024.0 MiB Used)	

- Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20250613171202-0007	(kill) NewsETLCleanToCurated	4	1024.0 MiB		2025/06/13 17:12:02	root	RUNNING	1.1 min

- Completed Applications (7)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20250613170509-0006	NewsETLCleanToCurated	4	1024.0 MiB		2025/06/13 17:05:09	root	FINISHED	3.7 min
app-20250613164710-0005	NewsETLCleanToCurated	4	1024.0 MiB		2025/06/13 16:47:10	root	FINISHED	43 s
app-20250613163427-0004	NewsETLRawToClean	4	1024.0 MiB		2025/06/13 16:34:27	root	FINISHED	3.6 min
app-20250613162701-0003	NewsETLRawToClean	4	1024.0 MiB		2025/06/13 16:27:01	root	FINISHED	5.0 min
app-20250613162056-0002	NewsETLRawToClean	4	1024.0 MiB		2025/06/13 16:20:56	root	FINISHED	4.8 min
app-20250613161412-0001	NewsETLRawToClean	4	1024.0 MiB		2025/06/13 16:14:12	root	FINISHED	4.7 min
app-20250613160553-0000	NewsETLRawToClean	4	1024.0 MiB		2025/06/13 16:05:53	root	FINISHED	4.9 min

Hình 3.13: Giao diện quản lý của Spark Master cho thấy 2 Worker đã sẵn sàng

2. Cơ chế kích hoạt Job với Airflow và Docker

Để tự động hóa và lên lịch cho các tác vụ ETL, hệ thống kết hợp giữa Airflow và Docker. Thay vì cài đặt Spark trực tiếp trên các node của Airflow, một giải pháp linh hoạt hơn được áp dụng:

1. **Xây dựng một Docker Image riêng (Gateway Image):** Một image được tạo ra chứa đầy đủ môi trường cần thiết, bao gồm Python, các thư viện phụ thuộc và Spark client. Image này đóng vai trò như một "gateway" hay một node client có khả năng gửi yêu cầu `spark-submit`.
2. **Sử dụng DockerOperator của Airflow:** Trong các DAG của Airflow, `DockerOperator` được sử dụng để khởi chạy một container từ Gateway

Image nói trên. Lệnh `spark-submit` sẽ được thực thi bên trong container này, gửi tác vụ đến cụm Spark đã triển khai để xử lý.

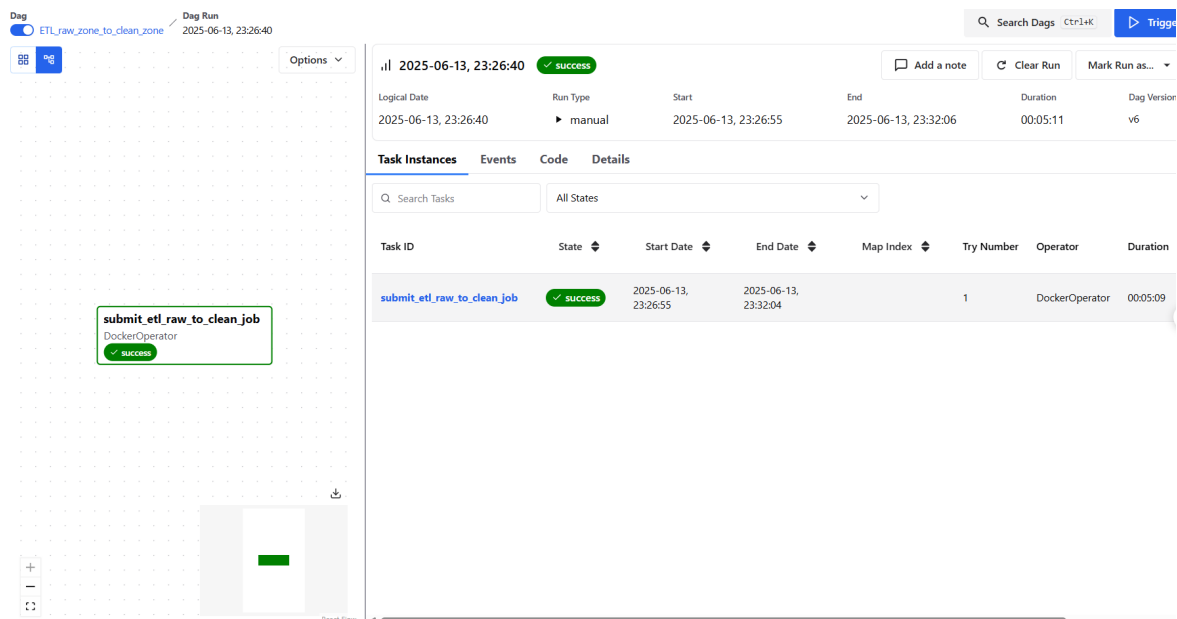
Cách tiếp cận này giúp tách biệt hoàn toàn môi trường của Airflow và Spark, Airflow đóng vai trò là một công cụ điều phối, còn spark cluster là nơi thực sự chạy mã nguồn, đồng thời tăng tính ổn định cho toàn hệ thống.

3.4.2 Các luồng xử lý ETL chính

Hai luồng ETL chính được xây dựng tương ứng với quá trình dịch chuyển dữ liệu giữa các vùng.

1. Giai đoạn Raw-to-Clean

Đây là bước làm sạch và chuẩn hóa dữ liệu đầu tiên. Pipeline này đọc dữ liệu JSONL từ vùng Raw, thực hiện các phép biến đổi như: ép kiểu dữ liệu (từ text sang số, ngày tháng), xử lý giá trị thiếu (null), loại bỏ các bản ghi trùng lặp và chọn lọc các trường cần thiết. Dữ liệu sau khi xử lý được lưu vào vùng Clean dưới định dạng Parquet và được quản lý bởi bảng Iceberg.

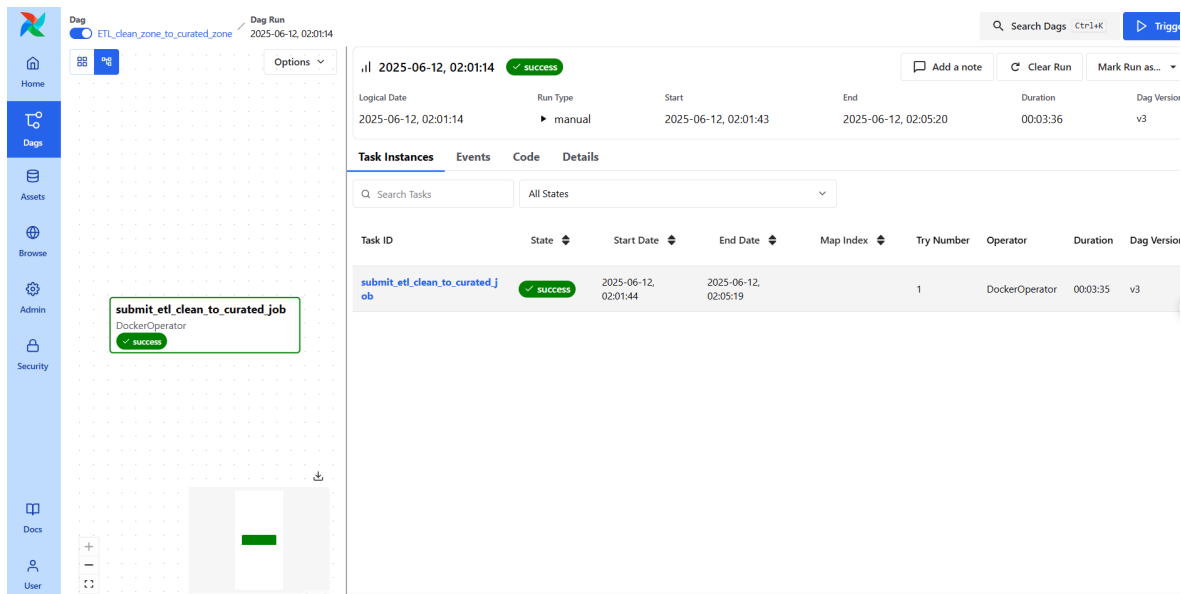


Hình 3.14: DAG trên Airflow cho pipeline ETL từ vùng Raw sang Clean

2. Giai đoạn Clean-to-Curated

Pipeline này chịu trách nhiệm áp dụng các logic nghiệp vụ phức tạp hơn. Nó đọc dữ liệu đã được làm sạch từ vùng Clean, thực hiện các phép nối (join) giữa

các bộ dữ liệu khác nhau, tính toán các chỉ số mới, và tổng hợp dữ liệu theo các chiều phân tích (ví dụ: tổng hợp số lượng bài báo theo ngày, theo chuyên mục). Kết quả cuối cùng là các bảng dữ liệu trong vùng Curated, đã được tối ưu và sẵn sàng cho các công cụ BI và phân tích.



Hình 3.15: DAG trên Airflow cho pipeline ETL từ vùng Clean sang Curated

3.4.3 Các kỹ thuật tối ưu hóa Spark được áp dụng

Trong quá trình phát triển các tác vụ ETL, nhiều kỹ thuật tối ưu hóa của Spark đã được áp dụng để tăng hiệu suất và giảm thiểu thời gian xử lý:

- **Caching:** Lưu các DataFrame trung gian thường xuyên được tái sử dụng vào bộ nhớ. Điều này giúp Spark không phải tính toán lại các DataFrame này từ đầu ở các bước sau, đặc biệt hữu ích trong các thuật toán lặp hoặc các job có nhiều action.
- **Broadcast Join:** Khi thực hiện join một bảng lớn với một bảng nhỏ, kỹ thuật này gửi một bản sao của bảng nhỏ đến tất cả các Worker Node. Việc này giúp tránh được quá trình shuffle dữ liệu tốn kém của bảng lớn, từ đó tăng tốc độ join lên đáng kể.
- **Coalesce và Repartition:** Được sử dụng để kiểm soát số lượng phân vùng (partition) của dữ liệu đầu ra. Sau các phép lọc, số lượng partition có thể không còn tối ưu. Coalesce() được dùng để giảm số lượng partition một cách hiệu quả (không gây shuffle) nhằm giải quyết "vấn đề tệp nhỏ" (small

files problem) trước khi ghi dữ liệu xuống Lakehouse.

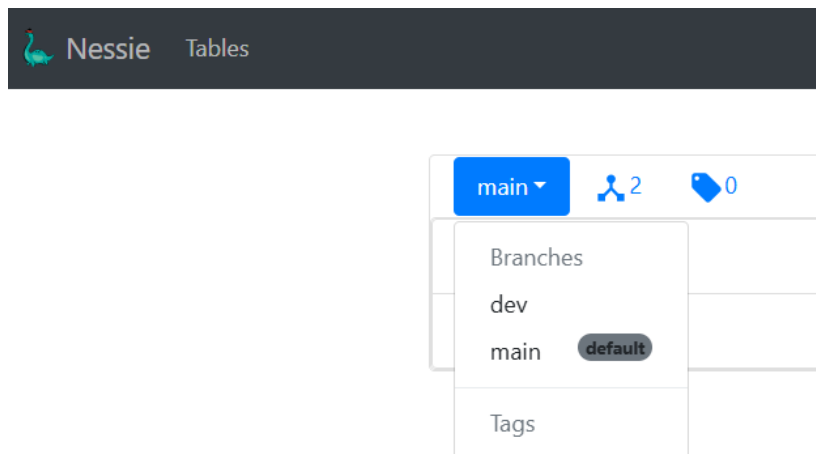
3.5 Lập trình và xây dựng các dịch vụ chung khác

Ngoài các thành phần cốt lõi như lưu trữ và xử lý, một hệ thống dữ liệu hiện đại cần các dịch vụ phụ trợ để đảm bảo tính toàn vẹn, khả năng quan sát và quản lý hiệu quả. Trong dự án này, ngoài Airflow và Docker, tôi còn sử dụng hai nhóm dịch vụ quan trọng được triển khai là Project Nessie cho quản lý phiên bản dữ liệu và bộ đôi Prometheus-Grafana cho giám sát hệ thống.

3.5.1 Project Nessie - Quản lý phiên bản dữ liệu

Như đã phân tích, Nessie được tích hợp vào Lakehouse để đóng vai trò là một catalog giao dịch (transactional catalog), mang lại các tính năng tương tự Git cho dữ liệu.

Cơ chế hoạt động của Nessie dựa trên việc quản lý các con trỏ (pointers) trỏ đến các tệp tin metadata của Apache Iceberg. Trong khi Iceberg chịu trách nhiệm theo dõi trạng thái của một bảng (schema, snapshots, các tệp dữ liệu), Nessie sẽ quản lý lịch sử các commit của những con trỏ này. Điều này cho phép thực hiện các giao tác nguyên tử (atomic transactions) trên nhiều bảng cùng một lúc, một tính năng mà Iceberg đơn lẻ không hỗ trợ.



Hình 3.16: Giao diện người dùng của Nessie hiển thị các nhánh

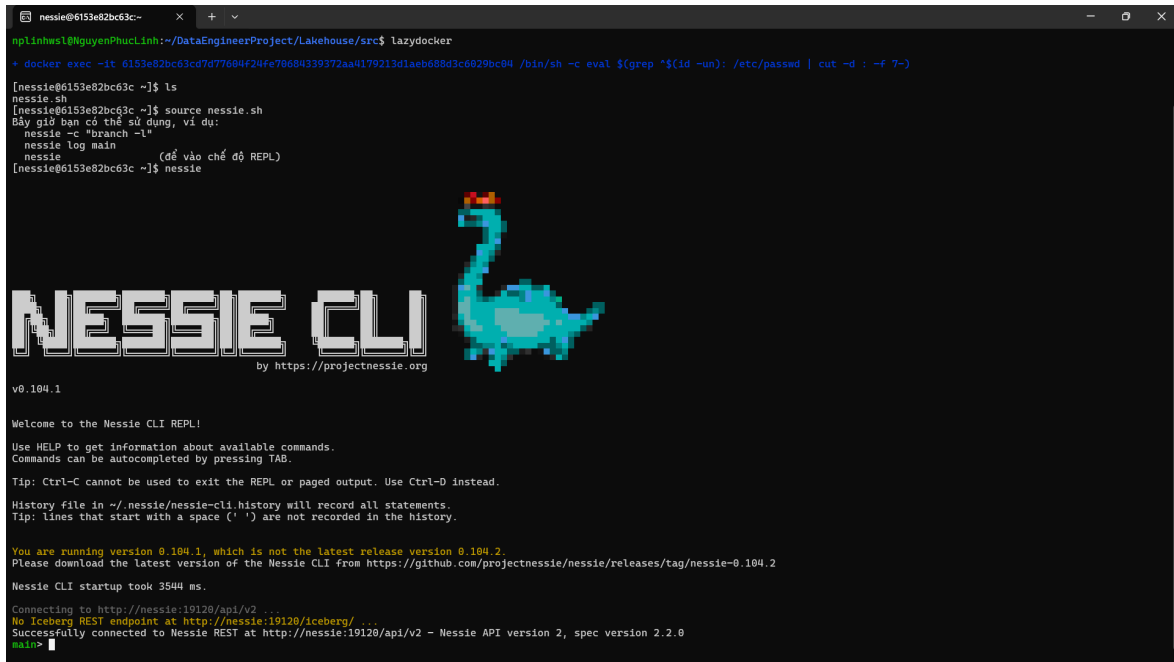
main ▾ nessie / commits		
Iceberg overwrite against news_curated_db.fact_top_comment_interaction_detail root committed on 7 minutes ago	 49345cb9	<>
Iceberg overwrite against news_curated_db.fact_top_comment_activity root committed on 7 minutes ago	 f15323e5	<>
Iceberg overwrite against news_curated_db.fact_article_reference root committed on 8 minutes ago	 dbbaa20b	<>
Iceberg overwrite against news_curated_db.fact_article_keyword root committed on 8 minutes ago	 ae34d268	<>
Iceberg overwrite against news_curated_db.fact_article_publication root committed on 8 minutes ago	 cf9cd3a5	<>
Iceberg overwrite against news_curated_db.dim_interaction_type root committed on 9 minutes ago	 7a81c90d	<>
Iceberg overwrite against news_curated_db.dim_reference_source root committed on 9 minutes ago	 5d170260	<>
Iceberg overwrite against news_curated_db.dim_keyword root committed on 9 minutes ago	 86f81fd7	<>

Hình 3.17: Giao diện người dùng của Nessie hiển thị các commit

Việc mang ngữ nghĩa của Git vào thế giới dữ liệu mở ra các khả năng vận hành mạnh mẽ:

- **Phân nhánh (Branching):** Cho phép các kỹ sư dữ liệu tạo ra các nhánh riêng (ví dụ: dev) để thử nghiệm các quá trình ETL hoặc phân tích mà không ảnh hưởng đến nhánh sản phẩm chính (main).
- **Hợp nhất (Merging):** Sau khi các thay đổi trên nhánh riêng đã được kiểm tra và xác thực, chúng có thể được hợp nhất trở lại nhánh chính một cách an toàn và nguyên tử.
- **Khả năng tái lập:** Bằng cách tham chiếu đến một commit hash cụ thể, người dùng có thể "du hành thời gian"(time travel) để truy vấn chính xác trạng thái của toàn bộ Lakehouse tại một thời điểm trong quá khứ.

Việc tương tác và quản lý các hoạt động này trong dự án được thực hiện chủ yếu qua giao diện dòng lệnh `nessie-cli`.



```

nessie@6153e82bc63c:~$ docker exec -it 6153e82bc63c /bin/sh -c eval $(grep "$(id -un): /etc/passwd | cut -d : -f 7-)
nessie@6153e82bc63c:~$ ls
nessie.sh
nessie@6153e82bc63c:~$ source nessie.sh
Đây giờ bạn có thể sử dụng, ví dụ:
nessie -c "branch -l"
nessie log main
nessie                               (để vào chế độ REPL)
nessie@6153e82bc63c:~$ nessie

NESSIE CLI
by https://projectnessie.org

v0.104.1

Welcome to the Nessie CLI REPL!

Use HELP to get information about available commands.
Commands can be auto-completed by pressing TAB.

Tip: Ctrl-C cannot be used to exit the REPL or paged output. Use Ctrl-D instead.

History file in ~/.nessie/nessie-cli.history will record all statements.
Tip: Lines that start with a space (' ') are not recorded in the history.

You are running version 0.104.1, which is not the latest release version 0.104.2.
Please download the latest version of the Nessie CLI from https://github.com/projectnessie/nessie/releases/tag/nessie-0.104.2

Nessie CLI startup took 3544 ms.

Connecting to http://nessie:19120/api/v2 ...
No Iceberg REST endpoint at http://nessie:19120/iceberg/ ...
Successfully connected to Nessie REST at http://nessie:19120/api/v2 - Nessie API version 2, spec version 2.2.0
main>

```

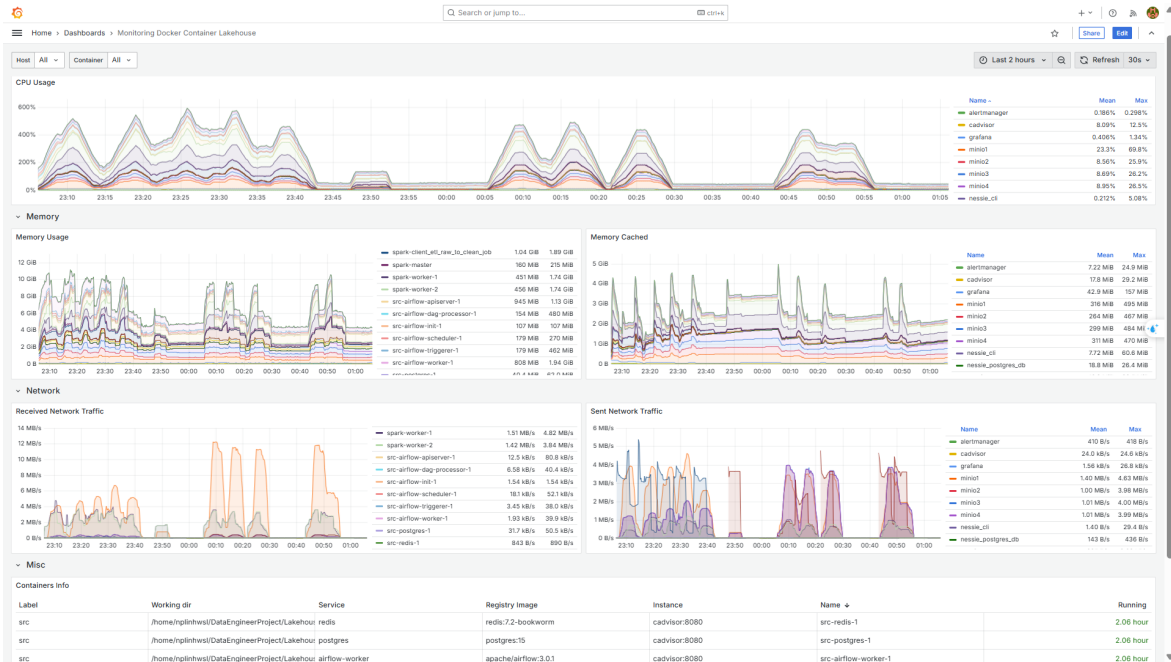
Hình 3.18: Tương tác với Nessie thông qua giao diện dòng lệnh (CLI)

3.5.2 Giám sát hệ thống với Prometheus và Grafana

Để đảm bảo hệ thống hoạt động ổn định và hiệu quả, một giải pháp giám sát (monitoring) toàn diện là không thể thiếu. Bộ đôi Prometheus và Grafana được lựa chọn, trong đó Prometheus chịu trách nhiệm thu thập và lưu trữ các chỉ số (metrics), còn Grafana đảm nhiệm việc trực quan hóa chúng thành các biểu đồ và dashboard.

1. Giám sát tài nguyên hệ thống

Để thu thập các chỉ số về tài nguyên của các container (CPU, RAM, Network I/O), công cụ **cAdvisor** (Container Advisor) của Google được triển khai. cAdvisor sẽ tự động phát hiện tất cả các container đang chạy trên một node và xuất ra các chỉ số hiệu năng của chúng. Prometheus được cấu hình để tự động thu thập (scrape) các chỉ số này từ cAdvisor. Dữ liệu sau đó được trực quan hóa trên Grafana, giúp người quản trị có một cái nhìn tổng quan về "sức khỏe" của toàn bộ hệ thống.



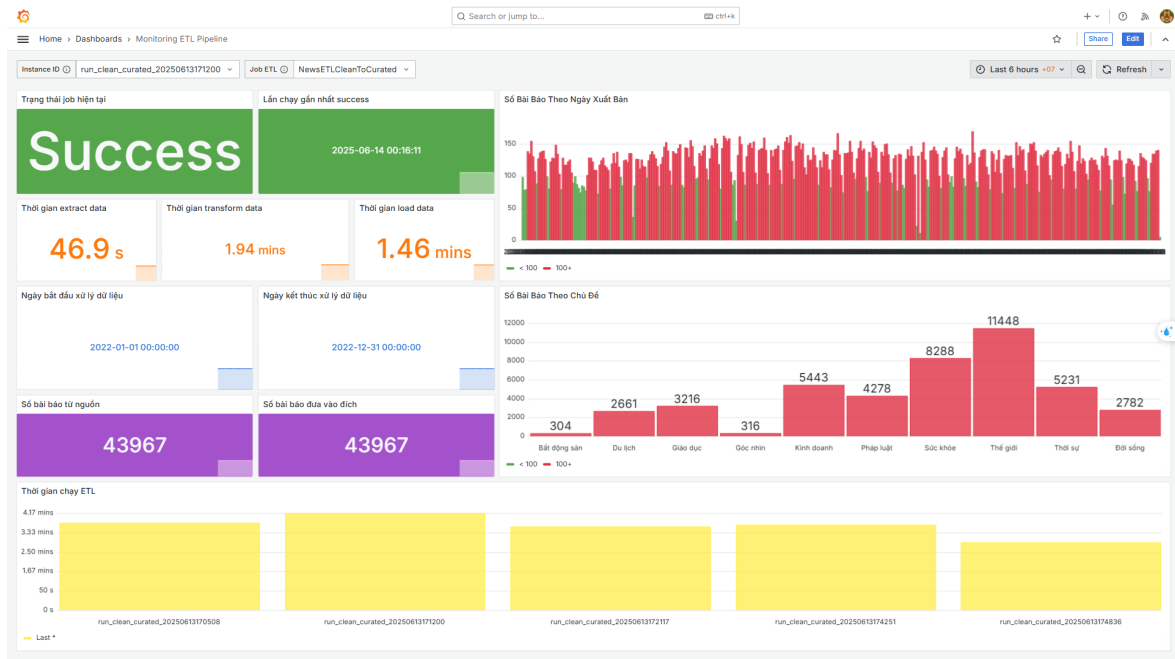
Hình 3.19: Dashboard trên Grafana trực quan hóa tài nguyên các container

2. Giám sát chất lượng pipeline

Việc giám sát các pipeline xử lý theo lô (batch job) đặt ra một thách thức: các job này thường chỉ tồn tại trong một thời gian ngắn và không có một endpoint cố định để Prometheus có thể scrape. Để giải quyết vấn đề này, mô hình **Pushgateway** được áp dụng:

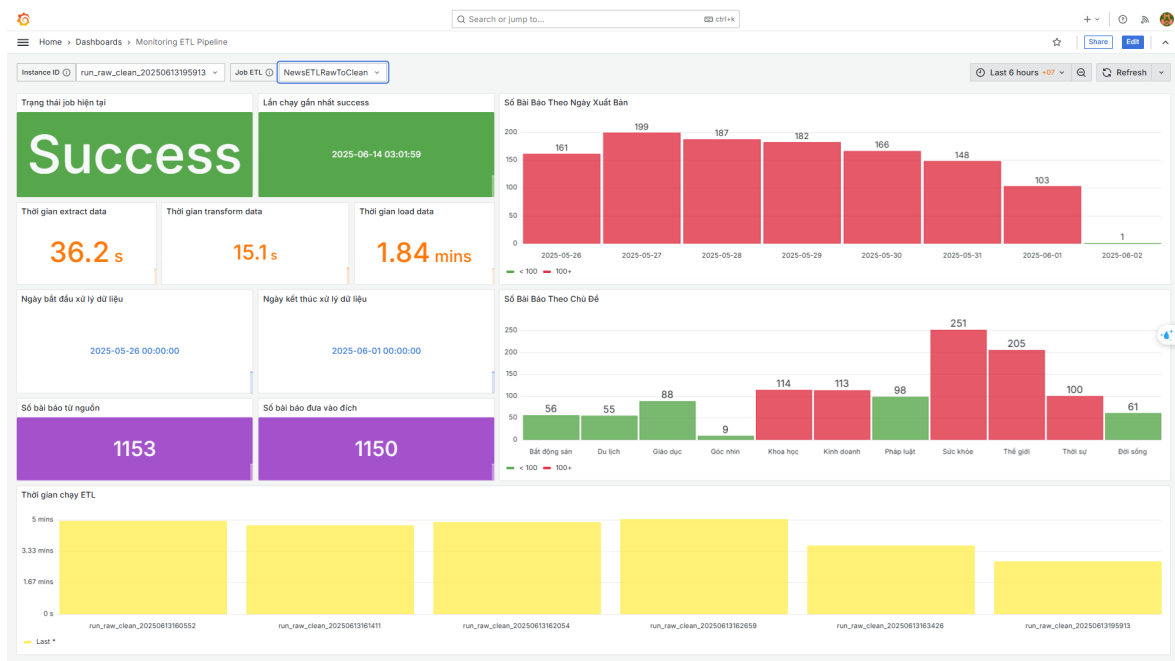
- Tạo chỉ số tùy chỉnh:** Trong mã nguồn Python của các tác vụ ETL, thư viện `prometheus_client` được sử dụng để định nghĩa các chỉ số nghiệp vụ quan trọng dưới dạng các metric.
- Đẩy chỉ số:** Khi một tác vụ ETL kết thúc, nó sẽ chủ động đẩy (push) các giá trị chỉ số vừa thu thập được lên một dịch vụ trung gian gọi là Prometheus Pushgateway.
- Thu thập từ Pushgateway:** Prometheus được cấu hình để định kỳ scrape các chỉ số từ Pushgateway, thay vì từ chính các tác vụ đã kết thúc.

Phương pháp này cho phép theo dõi hiệu suất và chất lượng của cả những pipeline xử lý không thường trực, sau đó được trực quan hóa trên Grafana để dễ dàng phân tích và cảnh báo khi có sự cố.



Hình 3.20: Dashboard trên Grafana theo dõi các chỉ số chất lượng của pipeline lịch sử

Đây là một pipeline từ vùng clean đến vùng curated, và chạy trong 1 năm. Đây thuộc job để xử lý dữ liệu lịch sử. Còn về job chạy hằng ngày một cách tự động, tôi chỉ xử lý dữ liệu trong 7 ngày gần nhất.



Hình 3.21: Dashboard trên Grafana theo dõi các chỉ số chất lượng của pipeline hằng ngày

3.6 Lập trình và xây dựng các dịch vụ khai thác dữ liệu

Sau khi dữ liệu đã được thu thập, xử lý và lưu trữ một cách có tổ chức, bước tiếp theo là cung cấp các công cụ và dịch vụ để người dùng cuối có thể khai thác và tương tác với dữ liệu đó. Trong dự án này, hai dịch vụ chính được triển khai cho mục đích này là Trino làm cổng truy vấn và Metabase làm nền tảng trực quan hóa.

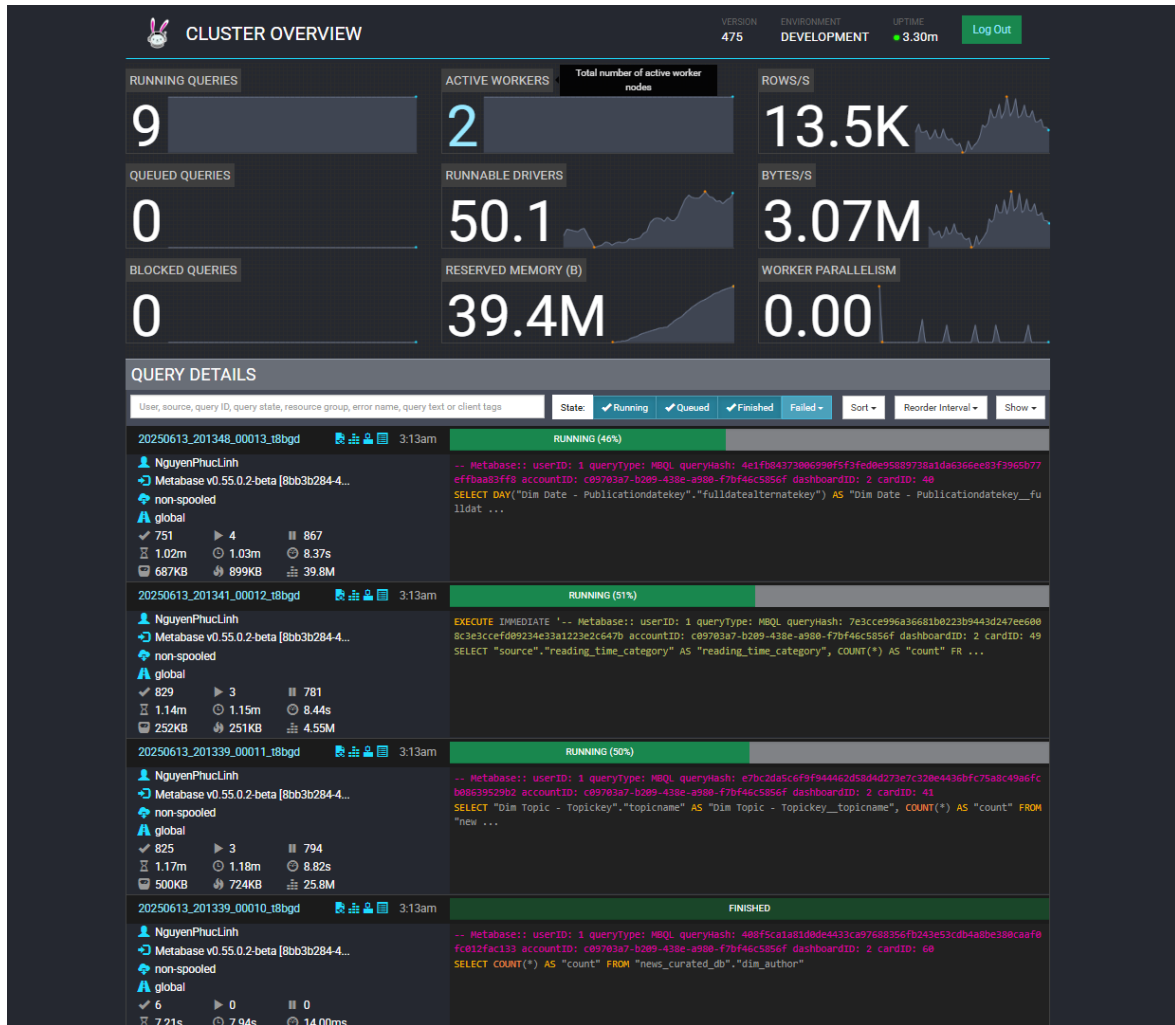
3.6.1 Trino - Cổng truy vấn SQL phân tán

Để cho phép các nhà phân tích dữ liệu, kỹ sư dữ liệu và các ứng dụng khác có thể truy vấn dữ liệu trong Lakehouse một cách hiệu quả, Trino (trước đây là PrestoSQL) được lựa chọn làm công cụ truy vấn SQL phân tán. Ưu điểm lớn nhất của Trino là kiến trúc tách biệt giữa tính toán và lưu trữ, cho phép nó truy vấn dữ liệu tại chỗ mà không cần phải di chuyển dữ liệu vào một hệ thống độc quyền.

Trong dự án, một cụm Trino được triển khai với:

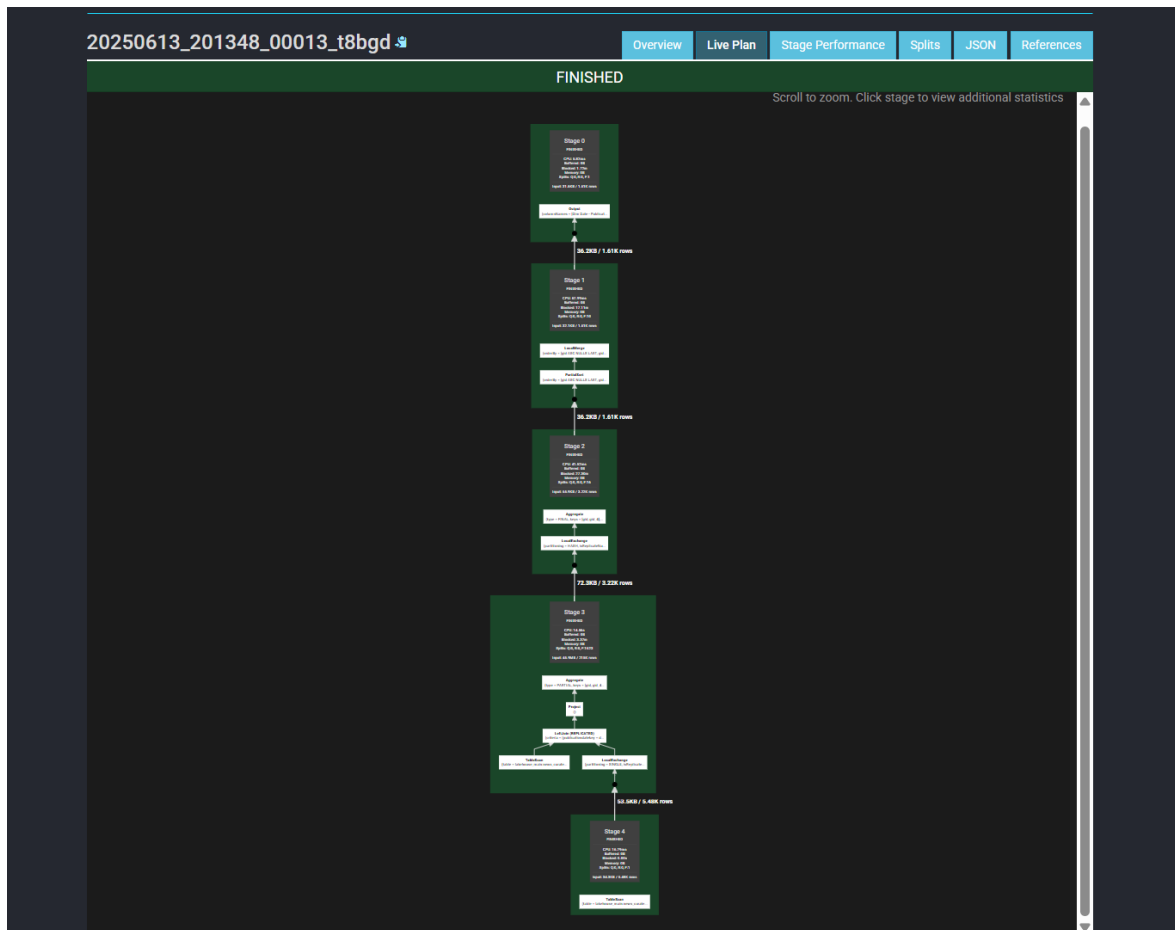
- **Một Coordinator:** Chịu trách nhiệm phân tích (parse) câu lệnh SQL, tối ưu hóa kế hoạch truy vấn (query plan), và điều phối công việc cho các Worker.
- **Hai Workers:** Chịu trách nhiệm thực thi các tác vụ được giao bởi Coordinator, trực tiếp đọc dữ liệu từ MinIO, xử lý và trả kết quả về.

Giao diện người dùng của Trino cung cấp khả năng theo dõi các truy vấn đang chạy, xem lại lịch sử các truy vấn đã thực thi, và quan trọng nhất là phân tích kế hoạch thực thi tối ưu.



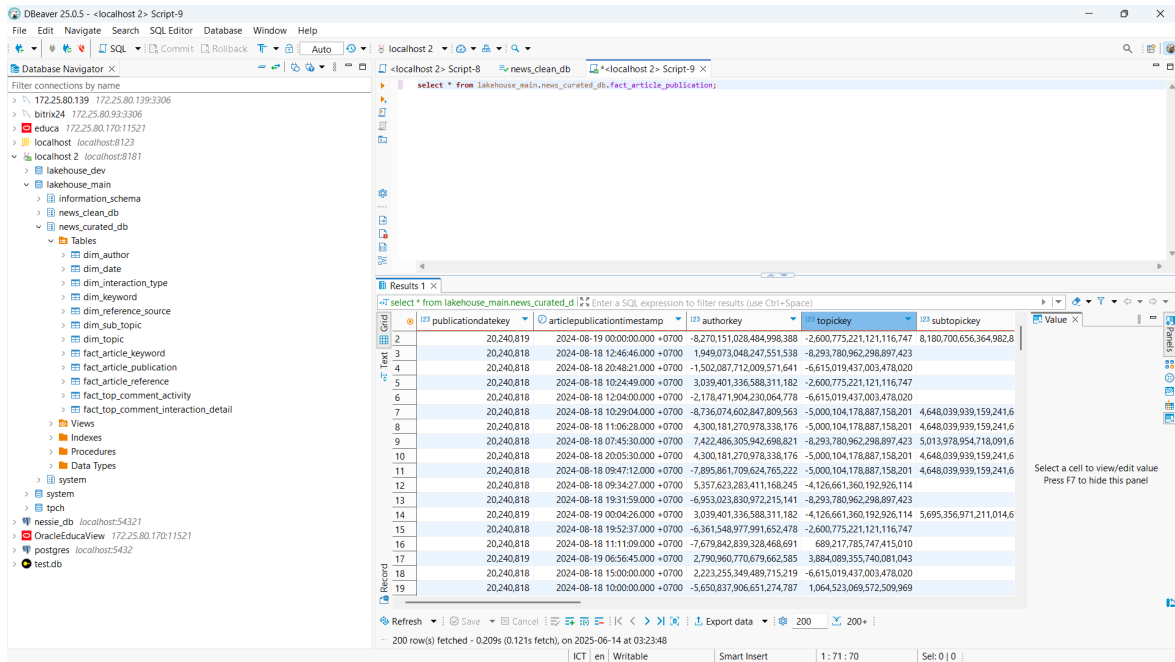
Hình 3.22: Giao diện quản lý của Trino hiển thị lịch sử truy vấn

Một trong những sức mạnh của Trino là bộ tối ưu hóa dựa trên chi phí (Cost-Based Optimizer). Nó sẽ phân tích câu lệnh SQL và tạo ra một cây thực thi hiệu quả nhất, bao gồm các bước như đẩy các điều kiện lọc xuống nguồn (predicate pushdown), sắp xếp lại thứ tự join, v.v. Người dùng có thể xem lại cây tối ưu này để hiểu rõ cách Trino xử lý truy vấn của mình.



Hình 3.23: Cây thực thi được Trino tối ưu hóa cho một câu lệnh SQL phức tạp

Bằng cách cung cấp một cổng truy cập SQL chuẩn (thông qua giao thức JDBC/ODBC), Trino đóng vai trò như một "cổng giao tiếp SQL vạn năng" cho Lakehouse, cho phép một hệ sinh thái rộng lớn các công cụ client có thể kết nối và khai thác dữ liệu. Một ví dụ điển hình là việc sử dụng DBeaver, một công cụ quản trị cơ sở dữ liệu phổ biến, để duyệt và truy vấn các bảng Iceberg một cách dễ dàng.



Hình 3.24: Sử dụng DBeaver để kết nối đến Lakehouse thông qua Trino

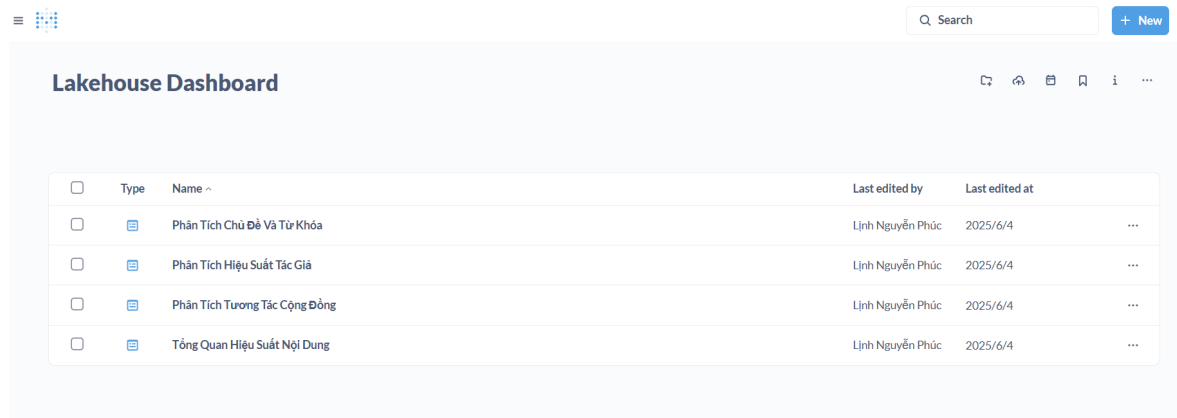
3.6.2 Metabase - Nền tảng Business Intelligence

Để hoàn thiện tầng khai thác dữ liệu, tôi đã lựa chọn Metabase làm công cụ trực quan hóa và phân tích. Là một nền tảng BI mã nguồn mở, Metabase đặc biệt mạnh mẽ trong việc cho phép người dùng cuối (end-user), kể cả những người không chuyên về SQL, có thể tự mình khám phá dữ liệu, tạo báo cáo và xây dựng các dashboard tương tác một cách dễ dàng.

Để khai thác dữ liệu từ Lakehouse, tôi đã cấu hình Metabase để kết nối thông qua Trino. Kiến trúc kết nối mà tôi triển khai, **Metabase** → **Trino** → **MinIO/Iceberg**, là một mô hình tiêu biểu và hiệu quả. Trong kiến trúc này, Metabase gửi các yêu cầu phân tích dưới dạng câu lệnh SQL đến Trino. Cụm Trino sẽ chịu trách nhiệm xử lý các truy vấn nặng, sau đó trả về một tập kết quả đã được tính toán. Metabase chỉ cần nhận kết quả này và thực hiện công việc cuối cùng là trực quan hóa. Cách tiếp cận này giúp tôi tận dụng được sức mạnh xử lý phân tán của Trino, giải phóng tài nguyên cho Metabase và đảm bảo giao diện người dùng luôn mượt mà, phản hồi nhanh chóng.

Đây chính là cầu nối cuối cùng, biến dữ liệu đã được tinh chế trong vùng Curated thành những thông tin chi tiết có giá trị. Để hiện thực hóa khả năng này, tôi đã thiết kế và xây dựng 4 dashboard phân tích chính, mỗi dashboard tập trung

vào một khía cạnh khác nhau của dữ liệu.



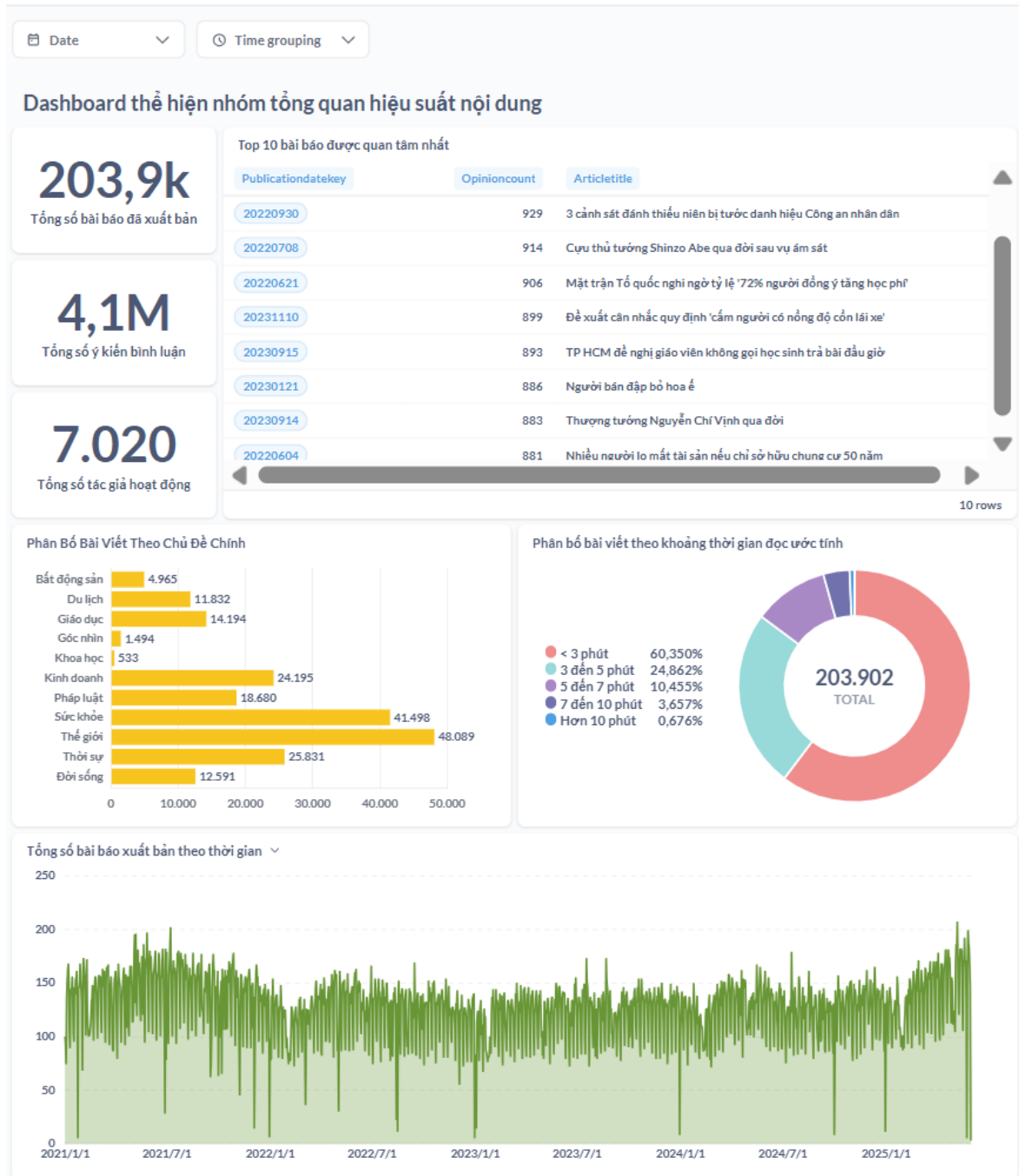
☐	Type	Name ^	Last edited by	Last edited at	
☐		Phân Tích Chủ Đề Và Từ Khóa	Linh Nguyễn Phúc	2025/6/4	...
☐		Phân Tích Hiệu Suất Tác Giả	Linh Nguyễn Phúc	2025/6/4	...
☐		Phân Tích Tương Tác Cộng Đồng	Linh Nguyễn Phúc	2025/6/4	...
☐		Tổng Quan Hiệu Suất Nội Dung	Linh Nguyễn Phúc	2025/6/4	...

Hình 3.25: Giao diện chính của Lakehouse Dashboard trên Metabase

Dưới đây là mô tả chi tiết về 4 dashboard mà tôi đã xây dựng:

1. Dashboard Tổng Quan Hiệu Suất Nội Dung:

1. Tổng Quan Hiệu Suất Nội Dung



Hình 3.26: Dashboard Tổng Quan Hiệu Suất Nội Dung

Trong dashboard đầu tiên này, tôi hướng đến việc cung cấp một bức tranh toàn cảnh về hiệu suất và quy mô của toàn bộ nội dung. Nó đóng vai trò như một trung tâm điều khiển, giúp người xem nhanh chóng nắm bắt các chỉ số

cốt lõi và xu hướng xuất bản chung. Để đạt được mục tiêu này, tôi đã thiết kế các thành phần sau:

- **Các chỉ số hiệu suất chính (KPIs):** Hiển thị ba con số tổng quan quan trọng nhất là tổng số bài báo, tổng số bình luận, và tổng số tác giả hoạt động.
- **Top 10 bài báo được quan tâm nhất:** Bảng xếp hạng các bài báo dựa trên số lượng bình luận để nhanh chóng xác định các nội dung có sức ảnh hưởng lớn.
- **Phân bố bài viết theo Chủ Đề Chính:** Biểu đồ cột thể hiện số lượng bài viết cho từng chuyên mục, giúp tôi đánh giá sự tập trung trong chiến lược nội dung.
- **Phân bố bài viết theo thời gian đọc ước tính:** Biểu đồ tròn phân tích độ dài bài viết, qua đó hé lộ thói quen tiêu thụ nội dung của độc giả.
- **Xu hướng xuất bản theo thời gian:** Biểu đồ miền thể hiện mật độ xuất bản bài báo từ năm 2021 đến nay để theo dõi sự tăng trưởng về số lượng. Kết quả là một cái nhìn 360 độ, giúp ban biên tập có thể đánh giá chiến lược nội dung tổng thể và tìm ra các cơ hội phát triển.

2. Dashboard Phân Tích Hiệu Suất Tác Giả:

2. Phân Tích Hiệu Suất Tác Giả



Hình 3.27: Dashboard Phân Tích Hiệu Suất Tác Giả

Dashboard thứ hai được tôi xây dựng để đánh giá và so sánh hiệu suất của các tác giả một cách đa chiều, từ năng suất đến khả năng tạo tương tác. Với bộ lọc theo Chủ đề và Thời gian, dashboard này trở thành một công cụ quản lý và định hướng nội dung mạnh mẽ. Các phân tích chính trong dashboard này gồm:

- **Top 10 tác giả có nhiều bài báo nhất:** Đo lường **năng suất** sản xuất nội dung.
- **Top 10 tác giả nhận được trung bình lượng bình luận cao nhất:** Đo lường **hiệu quả tương tác** trên mỗi bài viết, tìm ra những cây bút "chất lượng hơn số lượng".
- **Top 10 tác giả nhận được tổng lượng bình luận cao nhất:** Đo lường

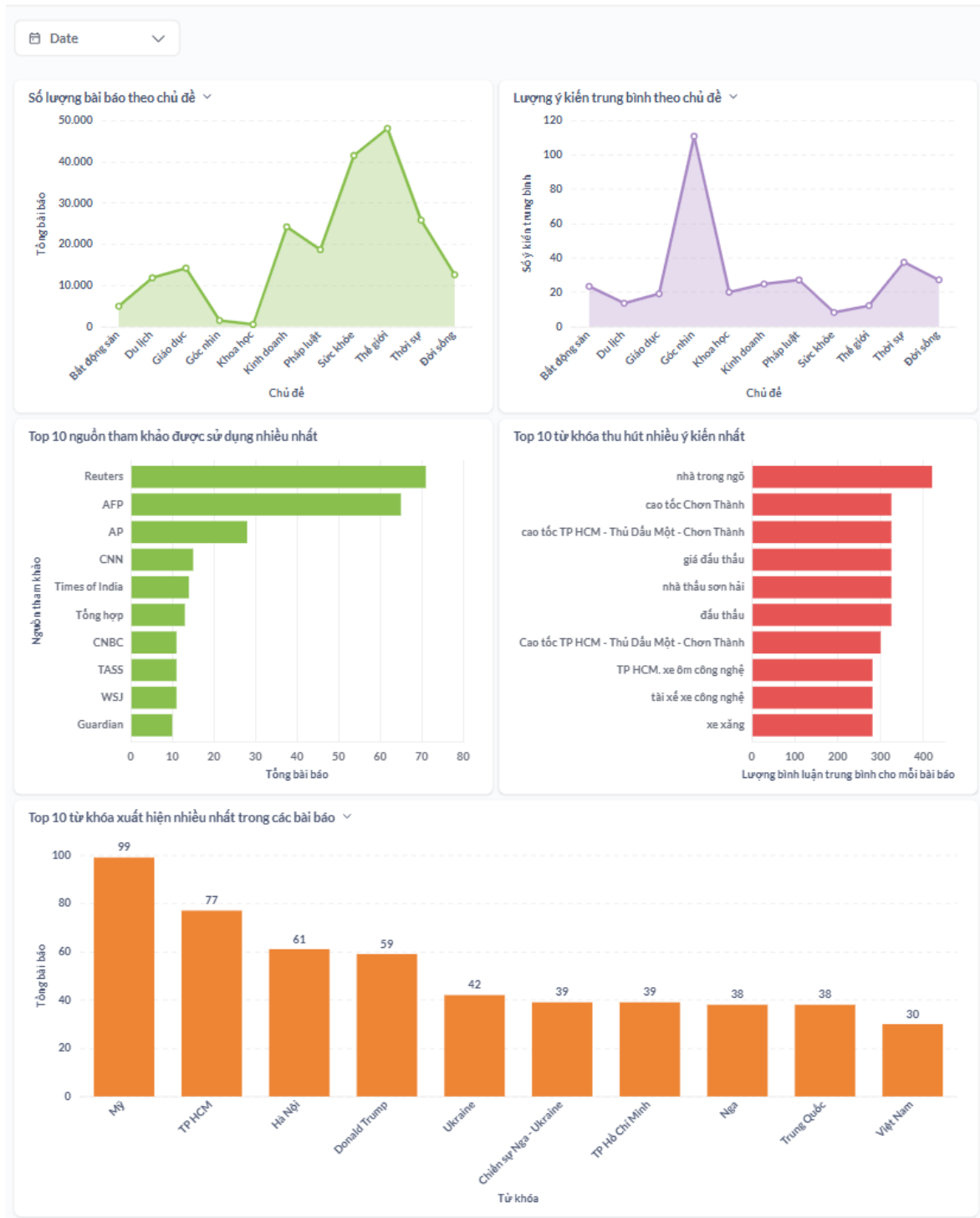
tầm ảnh hưởng tương tác tổng thể, kết hợp giữa năng suất và hiệu quả.

- **Top 10 từ khóa mà nhiều nhà báo thường dùng:** Phân tích **hành vi** và **lĩnh vực chuyên môn** chung của các tác giả, cho thấy các mảng nội dung trọng tâm của tòa soạn.

Thông qua đó, ban biên tập có thể đánh giá công bằng hiệu suất, phát hiện tài năng và điều phối nguồn lực một cách hiệu quả.

3. Dashboard Phân Tích Chủ Đề Và Từ Khóa:

3. Phân Tích Chủ Đề Và Từ Khóa



Hình 3.28: Dashboard Phân Tích Chủ Đề Và Từ Khóa

Với dashboard này, tôi đi sâu vào phân tích "cái gì"— tức là nội dung nào đang được sản xuất và nội dung nào thực sự thu hút độc giả. Mục tiêu của

tôi là tìm ra các chủ đề "nóng" để định hình chiến lược nội dung chi tiết hơn.

- **Phân tích Số lượng và Tương tác theo Chủ đề:** Tôi đặt hai biểu đồ này cạnh nhau để so sánh giữa **sản lượng** và **mức độ thu hút** của từng chủ đề, giúp tìm ra các chủ đề tiềm năng hoặc kém hiệu quả.
- **Top nguồn tham khảo được sử dụng:** Phân tích các nguồn tin được trích dẫn để đánh giá tính đa dạng và nguồn gốc thông tin.
- **Top từ khóa thu hút và xuất hiện nhiều nhất:** Tôi thiết kế hai biểu đồ này như một cặp phân tích mạnh mẽ. Bằng cách so sánh giữa những gì được viết nhiều nhất và những gì được quan tâm nhất, tôi có thể phát hiện ra các "mỏ vàng" nội dung.

Đây là công cụ không thể thiếu mà tôi xây dựng để hỗ trợ các biên tập viên chuyên mục trong việc ra quyết định đầu tư nội dung.

4. Dashboard Phân Tích Tương Tác Cộng Đồng:

4. Phân Tích Tương Tác Cộng Đồng



Hình 3.29: Dashboard Phân Tích Tương Tác Cộng Đồng

Dashboard cuối cùng chuyển hướng phân tích từ nội dung sang cộng đồng độc giả. Ở đây, tôi khám phá "tầng tương tác thứ hai"— các bình luận và phản ứng xoay quanh chúng.

- **Các chỉ số tổng quan về bình luận:** Thống kê quy mô hoạt động của cộng đồng, cho thấy sự tham gia của cả những "độc giả thầm lặng".
- **Phân bố các loại tương tác:** Phân tích cảm xúc và hình thức phản hồi

của người dùng đối với các bình luận, với "Thích" là hình thức phổ biến nhất.

- **Top 10 người bình luận nhiều nhất:** Xếp hạng này giúp tôi nhận diện những "người dùng quyền lực" (power users) có ảnh hưởng lớn nhất trong cộng đồng.
- **Phân bố bình luận theo chủ đề bài báo:** Biểu đồ này kết nối hoạt động của cộng đồng với loại nội dung, chỉ ra những chủ đề "chạm" đến độc giả và khuyến khích họ bày tỏ quan điểm.

Từ những phân tích này, tôi có thể cung cấp những insight quý giá về cộng đồng độc giả, giúp ban biên tập có những chiến lược phù hợp để tăng cường tương tác.

Kết luận

Sau một thời gian tập trung nghiên cứu và làm việc, đồ án tốt nghiệp của tôi đã đạt được những mục tiêu đề ra và thu được các kết quả cụ thể. Phần cuối cùng này sẽ tổng kết lại những thành tựu đó, những kinh nghiệm quý báu đã tích lũy và đề xuất những hướng phát triển tiềm năng cho đề tài trong tương lai.

1. Kết quả đạt được

Đồ án đã thành công trong việc nghiên cứu và xây dựng một nền tảng dữ liệu hoàn chỉnh theo kiến trúc Data Lakehouse hiện đại, ứng dụng cho lĩnh vực tin tức, thời sự. Hệ thống không chỉ là một mô hình lý thuyết mà là một sản phẩm có thể hoạt động, bao trùm toàn bộ vòng đời của dữ liệu với các kết quả cụ thể như sau:

- 1. Xây dựng thành công một hệ thống Data Lakehouse End-to-End:** Đã triển khai và tích hợp thành công một hệ sinh thái công nghệ mã nguồn mở phức tạp bao gồm MinIO, Spark, Airflow, Nessie, Trino, Selenium Grid và Metabase trên một nền tảng tại chỗ (on-premise).
- 2. Tự động hóa toàn bộ luồng dữ liệu:** Hệ thống có khả năng tự động thu thập dữ liệu tin tức hàng ngày, thực hiện các quy trình ETL để làm sạch, chuẩn hóa và làm giàu dữ liệu, đảm bảo dữ liệu luôn được cập nhật và sẵn sàng cho việc phân tích.
- 3. Lưu trữ và quản lý dữ liệu hiệu quả:** Áp dụng kiến trúc lưu trữ đa vùng kết hợp với định dạng bảng mở Apache Iceberg và công cụ quản lý phiên bản Nessie, giúp dữ liệu được tổ chức một cách khoa học, tối ưu về hiệu suất truy vấn và có khả năng quay về các phiên bản lịch sử.
- 4. Cung cấp khả năng khai thác dữ liệu trực quan:** Thông qua Trino và Metabase, hệ thống cung cấp một cổng truy cập dữ liệu mạnh mẽ, cho phép

người dùng từ kỹ thuật đến phi kỹ thuật có thể dễ dàng truy vấn, khám phá và xây dựng các dashboard phân tích chuyên sâu về hiệu suất nội dung, tác giả, và tương tác cộng đồng.

Toàn bộ mã nguồn của dự án đã được tôi lưu trữ và công khai tại kho chứa trên [GitHub](#).

2. Những kiến thức và kỹ năng tích lũy

Quá trình thực hiện đồ án là một cơ hội quý báu để tôi củng cố và phát triển nhiều kiến thức, kỹ năng quan trọng:

- **Về kiến thức chuyên môn:** Đồ án đã giúp tôi hệ thống và nắm vững kiến thức chuyên sâu về các kiến trúc dữ liệu hiện đại, đặc biệt là mô hình Data Lakehouse. Tôi cũng đã hiểu rõ hơn về vai trò, cách hoạt động và sự tương tác giữa các thành phần trong một hệ thống dữ liệu.
- **Về kỹ năng thực hành:** Tôi đã có kinh nghiệm thực tiễn trong việc triển khai và tích hợp một hệ sinh thái công nghệ phức tạp. Kỹ năng làm việc với Linux, Docker, lập trình với Python, Spark và cấu hình các dịch vụ khác đã được nâng cao đáng kể.
- **Về tư duy thiết kế và giải quyết vấn đề:** Đồ án đã rèn luyện cho tôi tư duy thiết kế một hệ thống từ đầu đến cuối, từ việc phân tích yêu cầu, lựa chọn công nghệ phù hợp, cho đến việc tối ưu hóa hiệu năng và đảm bảo tính ổn định của hệ thống.

3. Hướng phát triển của đề tài

Mặc dù đã đạt được những mục tiêu chính, hệ thống vẫn còn nhiều tiềm năng để có thể tiếp tục mở rộng và phát triển trong tương lai:

1. **Mở rộng nguồn và đa dạng hóa dữ liệu:** Hệ thống có thể được phát triển để thu thập dữ liệu từ nhiều nguồn khác như các trang tin tức khác, các nền tảng mạng xã hội (Twitter, Facebook) để có được cái nhìn đa chiều và toàn diện hơn về các xu hướng thông tin.
2. **Ứng dụng Học máy để khai thác sâu hơn:** Nền tảng dữ liệu đã xây dựng là một nguồn tài nguyên lý tưởng để triển khai các mô hình học máy, ví dụ

như:

- **Phân tích tình cảm:** Phân tích các bình luận của độc giả để đánh giá thái độ của cộng đồng đối với các chủ đề, sự kiện.
 - **Mô hình hóa chủ đề:** Tự động phát hiện các chủ đề ẩn hoặc các nhóm tin tức liên quan trong kho dữ liệu.
 - **Xây dựng hệ thống gợi ý:** Gợi ý các bài báo phù hợp cho từng người đọc dựa trên lịch sử đọc của họ.
3. **Nâng cấp theo hướng xử lý thời gian thực:** Tích hợp các công nghệ như Apache Kafka và Spark Streaming để có thể phân tích và cập nhật các dashboard gần như ngay lập tức khi có sự kiện mới xảy ra, thay vì xử lý theo lô hàng ngày.

Danh mục tài liệu tham khảo

- [1] Apache Software Foundation. *Apache Airflow Official Website*. URL: <https://airflow.apache.org/>.
- [2] Apache Software Foundation. *Apache Spark Official Website*. URL: <https://spark.apache.org/>.
- [3] Cloud Native Computing Foundation. *Prometheus — Monitoring system & time series database*. URL: <https://prometheus.io/> (**urlseen** 14/06/2025).
- [4] Docker, Inc. *Docker Official Website*. URL: <https://www.docker.com/>.
- [5] FPT Corporation. *VnExpress — Báo điện tử tiếng Việt*. URL: <https://vnexpress.net/> (**urlseen** 14/06/2025).
- [6] Grafana Labs. *Grafana — Open Source Observability Platform*. URL: <https://grafana.com/> (**urlseen** 14/06/2025).
- [7] Metabase, Inc. *Metabase Official Website*. URL: <https://www.metabase.com/>.
- [8] MinIO, Inc. *MinIO Official Website*. URL: <https://min.io/>.
- [9] Project Nessie. *Project Nessie Official Website*. URL: <https://projectnessie.org/>.
- [10] Selenium Project. *Selenium Grid Official Documentation*. URL: <https://www.selenium.dev/documentation/grid/>.
- [11] Gaurav Ashok Thalpati. *Practical Lakehouse Architecture*. <https://www.oreilly.com/library/view/practical-lakehouse-architecture/9781098153007/>. 2024.
- [12] Trino Software Foundation. *Trino Official Website*. URL: <https://trino.io/>.